

# Sistemi di Calcolo

Cristian Amendola, Giovanni Esposito, Danilo Di Primio

Università degli studi La Sapienza

Ingegneria Informatica e Automatica - 2 anno

Febbraio - Maggio 2026

# Indice

|          |                                                                               |           |
|----------|-------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Debugging</b>                                                              | <b>3</b>  |
| 1.1      | GDB . . . . .                                                                 | 3         |
| 1.2      | Valgrind . . . . .                                                            | 4         |
| <b>2</b> | <b>Teoria</b>                                                                 | <b>5</b>  |
| 2.1      | Cos'è un sistema di Calcolo? . . . . .                                        | 5         |
|          | Hardware . . . . .                                                            | 5         |
|          | Elementi di base di un microprocessore . . . . .                              | 5         |
| 2.2      | Come viene programmato un Sistema di Calcolo . . . . .                        | 8         |
|          | Linguaggio Alto e Basso Livello . . . . .                                     | 8         |
|          | Compilatori e Interpreti . . . . .                                            | 8         |
|          | Formato testo, oggetto ed eseguibile . . . . .                                | 9         |
|          | Disassemblare programmi in codice macchina: objdump . . . . .                 | 10        |
|          | Traduzione in linguaggio macchina programma C . . . . .                       | 10        |
| <b>3</b> | <b>Assembly</b>                                                               | <b>13</b> |
| 3.1      | Funzioni . . . . .                                                            | 13        |
|          | Uso di parametri nelle funzioni . . . . .                                     | 13        |
| 3.2      | Istruzioni . . . . .                                                          | 13        |
|          | Struttura dei dati in memoria . . . . .                                       | 14        |
|          | Accesso in memoria con spiazzamento . . . . .                                 | 15        |
| 3.3      | Gestione del flusso di esecuzione . . . . .                                   | 16        |
|          | Salto incondizionato . . . . .                                                | 16        |
|          | Salto condizionato . . . . .                                                  | 16        |
|          | Registro EFLAGS . . . . .                                                     | 17        |
| 3.4      | Istruzioni di Condizione . . . . .                                            | 18        |
|          | Cicli While-For . . . . .                                                     | 18        |
| 3.5      | Puntatori . . . . .                                                           | 19        |
|          | Accesso in memoria con base,indice e spiazzamento . . . . .                   | 21        |
| 3.6      | Chiamate a Funzione . . . . .                                                 | 22        |
|          | Metodo <i>Push + Pop</i> . . . . .                                            | 26        |
| 3.7      | Conversione del tipo (cast) . . . . .                                         | 29        |
|          | Variabili Locali . . . . .                                                    | 31        |
|          | Espressioni Booleane . . . . .                                                | 34        |
| 3.8      | Allineamento nello Stack . . . . .                                            | 35        |
| 3.9      | Allineamento delle Struct . . . . .                                           | 36        |
| 3.10     | Istruzioni SHIFT . . . . .                                                    | 37        |
| 3.11     | Istruzione IDIV . . . . .                                                     | 38        |
| 3.12     | Istruzione CMOV . . . . .                                                     | 38        |
| 3.13     | Ottimizzazione dei programmi . . . . .                                        | 39        |
|          | Analisi prestazionale del programma . . . . .                                 | 39        |
|          | Legge di Amdahl . . . . .                                                     | 40        |
| 3.14     | Gestione dinamica della memoria in Assembly con <code>malloc</code> . . . . . | 40        |
|          | Gestione interna in assembly . . . . .                                        | 41        |
| 3.15     | Profilazione delle prestazioni . . . . .                                      | 43        |
|          | Profilazione Manuale . . . . .                                                | 43        |
|          | Profilazione Automatica . . . . .                                             | 44        |
|          | Assetti chiave della ottimizzazione . . . . .                                 | 44        |
|          | Tecniche di ottimizzazione del codice . . . . .                               | 45        |

|          |                                                   |           |
|----------|---------------------------------------------------|-----------|
| 3.16     | Come vengono eseguiti i programmi . . . . .       | 47        |
|          | Terminazione dei programmi . . . . .              | 48        |
|          | Gestire la terminazione di un programma . . . . . | 48        |
|          | Tipologie di Errore . . . . .                     | 49        |
| <b>4</b> | <b>Approfondimento C</b>                          | <b>50</b> |
| 4.1      | Funzioni di Libreria . . . . .                    | 50        |
|          | Assert . . . . .                                  | 50        |
|          | Funzioni per la gestione di Stringhe . . . . .    | 50        |
|          | Gestione della Memoria . . . . .                  | 53        |
|          | Funzioni di Ordinamento e Ricerca . . . . .       | 54        |
|          | Funzioni Matematiche . . . . .                    | 56        |
|          | Funzioni per la Gestione degli Stream . . . . .   | 56        |
| 4.2      | Esecuzione dei programmi . . . . .                | 60        |
|          | Processi . . . . .                                | 60        |
| 4.3      | Multiprogrammazione . . . . .                     | 62        |
|          | Funzioni per la gestione dei processi . . . . .   | 63        |
|          | Sincronizzazione dei processi . . . . .           | 65        |
|          | Caricamento di programmi . . . . .                | 66        |
| 4.4      | Thread . . . . .                                  | 68        |
|          | Posix Thread . . . . .                            | 69        |
|          | Come la CPU esegue le istruzioni . . . . .        | 70        |
|          | Esecuzione con Pipelining . . . . .               | 72        |
|          | Interruzioni . . . . .                            | 74        |
| 4.5      | Segnali . . . . .                                 | 76        |
|          | Gestione dei segnali . . . . .                    | 77        |
| 4.6      | Gestione della memoria . . . . .                  | 79        |
|          | Frammentazione della memoria . . . . .            | 79        |
|          | Memoria virtuale . . . . .                        | 80        |
|          | Paginazione della memoria . . . . .               | 80        |
|          | Paginazione con bit di validità . . . . .         | 82        |
|          | Memory swapping . . . . .                         | 82        |
|          | Allocazione della memoria . . . . .               | 83        |
|          | Altri miglioramenti . . . . .                     | 84        |
|          | Cache e uso ottimizzato della memoria . . . . .   | 85        |
| 4.7      | Cache e associatività . . . . .                   | 86        |
| 4.8      | File System . . . . .                             | 88        |
|          | File e Directory . . . . .                        | 88        |
|          | Struttura gerarchica dei File System . . . . .    | 89        |
|          | Path / Percorso . . . . .                         | 90        |
|          | Organizzazione interna del File System . . . . .  | 91        |
|          | Tabella dei descrittori . . . . .                 | 94        |
|          | Gestione (Mount) dei Dispositivi . . . . .        | 95        |
|          | Link . . . . .                                    | 96        |
|          | Variabili di ambiente . . . . .                   | 96        |
| 4.9      | Domande d'esame . . . . .                         | 97        |

# Capitolo 1

## Debugging

### 1.1 GDB

Uno dei requisiti necessari per utilizzare gli strumenti di debbuging è l'inclusione delle informazioni di debbuging durante la fase di compilazione. Per compilare un programma C con le informazioni di debug, bisogna compilare in questo modo:

```
gcc -g programma.c -o programma
```

In modo analogo per includere del codice assembly da noi scritto, bisogna aggiungere (Esempio caso codice assembly IA32) nella riga di compilazione:

```
gcc -g -m32 programma.c -o programma
```

Potrebbe capitare che il debug non funzioni, in questo caso è consigliato di definire in gdb i comandi custom, per farlo bisogna editare il file `.gdbinit` all'interno della home utente (Esempio sulla VM BIAR):

Andare nella directory: `/home/studente/.gdbinit`

Inserire nel file il seguente contenuto:

```
define go
    start
    layout src
    layout regs
    focus cmd
end
```

Il comando `go` ci permette di visualizzare diverse informazioni durante l'uso di GDB. Per avviare il debugging su un eseguibile "e" (dopo aver compilato): `→ gdb ./e`

Comandi principali di GDB:

- **go** → lancia il debug.
- **run** → rilancia l'esecuzione con eventuali aggiornamenti da specificare dopo (`run [argomento]`).
- **quit** → per uscire da gdb.
- **cont** → riprende l'esecuzione normalmente dopo un breakpoint.
- **step** → esegue una singola istruzione (non si ferma alle assegnazioni)
- **next** → esegue la funzione fino al return "finale" .
- **finish** → esegue fino alla fine della funzione in cui siamo, non funziona correttamente in codice assembly.
- **break** → inserisce un breakpoint, esempio `b e.c:20` inserisce un breakpoint alla linea 20 del file "e.c", per avere informazioni sui breakpoint attivi invece dovremmo usare `info break`, inoltre si può posizionare un break condizionale: `break e.c:20 if x == 0`, quindi in questo caso sarà posizionato il break solamente se x sarà uguale a 0, altrimenti continuerà normalmente.
- **print:**
  - **print \$eax** → stampa il contenuto di un registro.

- **print x** → stampa il contenuto della variabile x.
- **x addr** → stampa il contenuto della memoria all'indirizzo addr.
- **x\$eax + 4** → stampa il contenuto della memoria dato dall'espressione (in questo caso + 4 posizioni).
- **x/nfu addr** → stampa il contenuto della memoria all'indirizzo addr.
- **start** → si posiziona all'inizio del main, non c'è bisogno di utilizzarlo quando si usa go.
- **where** → mostra la posizione corrente nell'esecuzione.
- **list** → mostra il contenuto del file sorgente che contiene la funzione corrente.
- **backtrace** → ti dà l'elenco dei frame attivi.
- **refresh** → esegue il refresh dell'output del debugger.

## 1.2 Valgrind

Per utilizzare Valgrind avremo bisogno di compilare il nostro file allo stesso modo di gdb. Viene utilizzato per identificare molti errori in C, come:

- Accessi invalidi alla memoria (segmentation fault)
- uso di variabili non inizializzate
- mancata de-allocazione di blocchi allocati dinamicamente (memory leak)

La riga di codice per avviare Valgrind è la seguente:

```
valgrind --leak-check=full ./programma
```

Per terminare la esecuzione della macchina virtuale stando su VS:

```
sudo shutdown -h now
```

# Capitolo 2

## Teoria

### 2.1 Cos'è un sistema di Calcolo?

Un **sistema di calcolo** consiste di software e hardware che operano insieme per supportare l'esecuzione di programmi. In generale un sistema di calcolo è un qualsiasi **sistema programmabile**. Un sistema di calcolo può essere formato da un **singolo nodo**, cioè insieme di parti hardware strettamente connesse tra loro, oppure da **più nodi** connessi mediante una rete di comunicazione e viene anche chiamato **cluster**.

#### Hardware

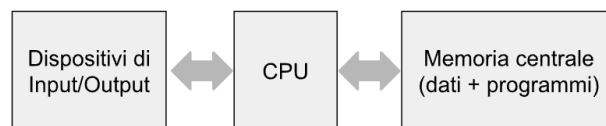


Figura 2.1: Schema di Von Neumann

La memoria centrale contiene dati da elaborare e programmi, i dispositivi di input e output scambiano dati e interagiscono con il mondo esterno, mentre la CPU esegue le istruzioni di programma (Modello di Von Neumann).

La **CPU** è il cuore del Sistema. Esso esegue programmi scritti in linguaggio macchina (codice nativo), rappresentati come sequenze di byte che codificano istruzioni.

I **Bus** sono strutture di interconnessione che collegano le varie componenti del sistema consentendo lo scambio dei dati. I bus sono canali di comunicazione che trasferiscono i dati a blocchi fissi di byte, chiamati word, che è generalmente di 4 byte oppure 8 byte.

L' **I/O Bridge** si occupa di coordinare lo scambio dati fra la CPU e il resto del sistema.

I **Controller e adattatori** interfacciano il sistema verso il mondo esterno, ad esempio acquisendo i movimenti del mouse o i tasti premuti sulla tastiera. La differenza fra i due è che i controller sono saldati sulla scheda madre oppure sono integrati nel dispositivo esterno, mentre gli adattatori sono schede esterne collegate alla scheda madre.

Il **Sistemi di memoria** di un calcolatore è molto articolato. La CPU non comunica direttamente con la memoria principale (RAM), ma ci sono una serie di strati di memorie intermedie chiamate cache che memorizzano i dati più frequentemente acceduti o adiacenti. Le cache hanno tempo di accesso molto inferiori a quelli di una RAM.

#### Elementi di base di un microprocessore

Negli elementi di base di un processore abbiamo le porte logiche che realizzano funzionalità elementari di una CPU. Le tre porte logiche principali sono: AND, OR, NOT e sono rappresentate dai seguenti simboli:

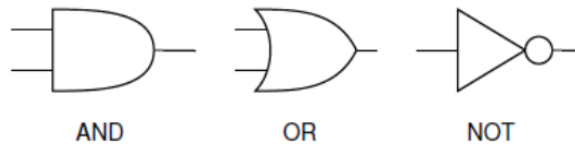


Figura 2.2: Porte Logiche

Analizziamo una porta alla volta, cominciando dalla porta **AND**, realizza un operatore binario che vale 1 se e solo se entrambi i propri argomenti sono 1:

| <b>X</b> | <b>Y</b> | <b>X AND Y</b> |
|----------|----------|----------------|
| 0        | 0        | 0              |
| 0        | 1        | 0              |
| 1        | 0        | 0              |
| 1        | 1        | 1              |

Figura 2.3: Tabella Porta Logica AND

La porta **OR** realizza un operatore binario che vale 1 se e solo se almeno uno dei propri argomenti è 1:

| <b>X</b> | <b>Y</b> | <b>X OR Y</b> |
|----------|----------|---------------|
| 0        | 0        | 0             |
| 0        | 1        | 1             |
| 1        | 0        | 1             |
| 1        | 1        | 1             |

Figura 2.4: Tabella Porta Logica OR

La porta **NOT** realizza un operatore unitario che vale 1 se e solo se il proprio argomento è 0:

| <b>X</b> | <b>NOT X</b> |
|----------|--------------|
| 0        | 1            |
| 1        | 0            |

Figura 2.5: Tabella Porta Logica NOT

Abbiamo anche la possibilità di circuiti combinatori che realizzano funzioni booleane arbitrarie, vediamo un esempio di addizionatore parziale:

| <b>X</b> | <b>Y</b> | <b>somma(X,Y)</b> | <b>riporto(X,Y)</b> |
|----------|----------|-------------------|---------------------|
| 0        | 0        | 0                 | 0                   |
| 0        | 1        | 1                 | 0                   |
| 1        | 0        | 1                 | 0                   |
| 1        | 1        | 0                 | 1                   |

Figura 2.6: Operazioni Logiche

Inoltre un'espressione, gli operatori hanno priorità, dalla maggiore alla minore in questo ordine: NOT, AND, OR.

## 2.2 Come viene programmato un Sistema di Calcolo

### Linguaggio Alto e Basso Livello

Nei linguaggi di programmazione possiamo differenziarli in: alto livello e basso livello. Un linguaggio di **alto livello** forniscono costrutti molto più potenti e sono molto più semplici da imparare, esempi sono: C, Python, ecc... Programmatori più avanzati prediligono l'uso di linguaggi di **livello più basso** per avere un codice più ottimizzato per le prestazioni, esempio è il linguaggio Assembly.

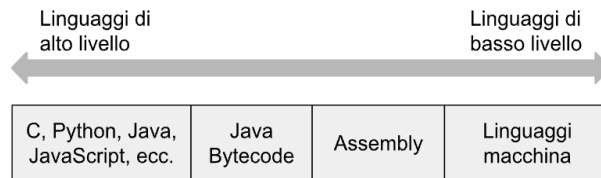


Figura 2.7: Classifica Linguaggi

Il linguaggio macchina è un linguaggio particolare progettato con l'obiettivo di essere facilmente interpretabile dai sistemi elettronici.

### Compilatori e Interpreti

C'è da fare ancora una differenziazione tra i linguaggi, di fatto abbiamo due tipi di linguaggi: compilati e interpretati. Nei linguaggi **compilati** il programma di alto livello viene tradotto in codice macchina in modo da essere eseguito direttamente dalla CPU, questa operazione viene chiamata compilazione; mentre un linguaggio **interpretato** esegue direttamente il programma senza bisogno di compilarlo prima, questi linguaggi vengono chiamati linguaggi di scripting. Esiste anche la possibilità di avere un linguaggio **ibrido**, ovvero il programma viene prima compilato in codice scritto in un linguaggio intermedio per poi venire interpretato. Ogni metodologia comporta i vantaggi e svantaggi:

#### Compilati:

- Vantaggi → il compilatore genera un codice ottimizzato per la piattaforma di calcolo considerata ottenendo migliori prestazioni.
- Svantaggi → il programma non è portatile, è quindi eseguibile solo sulla piattaforma di calcolo su cui è stata compilata.

#### Interpretati:

- Vantaggi → il programma è portatile, inoltre è possibile eseguirlo direttamente senza bisogno di compilarlo.
- Svantaggi → l'esecuzione è più lenta, tuttavia alcuni interpreti, come Java Bytecode, utilizzano una tecnica chiamata Just-in-time compilation (JIT) che consiste nel compilare il programma da eseguire in codice nativo subito prima di essere eseguito.

#### Ibridi:

- Vantaggi → programma portatile.
- Svantaggi → stessi dei linguaggi interpretati.

Noi ci occuperemo del linguaggio C. La compilazione di un programma C può essere scomposta nella compilazione di più moduli C, detti translation unit, ciascuno con estensione ".c".

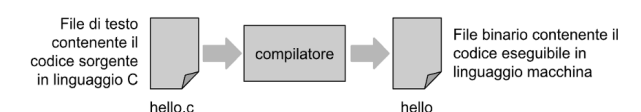


Figura 2.8: Esempio con C

Il processo di compilazione in C è composto in realtà da diversi stadi che vengono effettuati dal sistema di compilazione (compilation toolchain), senza rendercene conto:

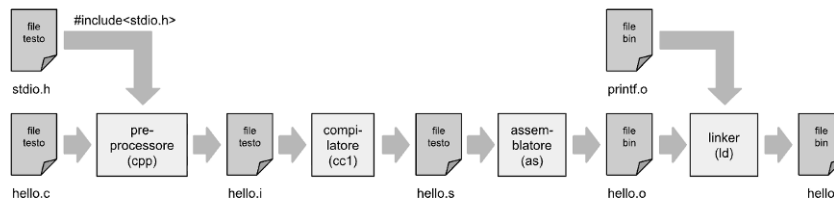


Figura 2.9: Esempio con C

Gli stadi coinvolgono diversi sottoprogrammi che vengono attivati nel sistema di compilazione:

- **Processore** → prende un testo C e lo converte in un altro testo C con le direttive già elaborate (`#include`, `#define`, ecc...). Quindi oltre che leggere il file “hello.c” leggerà anche il file della libreria “stdio.h”. Per intenderci, il comando: `[gcc -E hello.c]`.
- **Processore** → prende il testo C senza direttive, verifica la correttezza sintattica e lo traduce in assembly. Con il comando: `[gcc -S hello.i]`.
- **Processore** → prende un programma in assembly e lo traduce in codice macchina generando un file oggetto binario. Tramite: `[gcc -c hello.s]`
- **Processore** → prende i vari file oggetto e li collega insieme a formare un unico file eseguibile. Nel nostro caso collega insieme “hello.o” contenente il codice macchina e “printf.o” contenente il codice macchina che realizza la funzione “printf” invocata dal programma, dando come risultato il file eseguibile “hello”. Con l'utilizzo del comando: `[gcc hello.o -o hello]`.

Il collegamento può avvenire in due modi possibili: **linking statico**, nel quale il codice oggetto della funzione chiamata viene copiato nel file eseguibile, e **linking dinamico** dove il codice oggetto della funzione chiamata risiede in un file fornito con il sistema operativo, successivamente il collegamento tra la chiamata a funzione ed il codice che lo implementa viene fatto al caricamento del file eseguibile. Esisto in realtà un altro tipo di collegamento di linking chiamato **linking a Runtime**, che avviene durante l'esecuzione del programma, il collegamento tra programma e le librerie necessarie viene effettuato solo nel momento in cui serve realmente quella funzione, mentre il programma è già in funzione, permette di caricare le librerie solo quando necessario. Nei programmi possiamo riscontrare tipi diversi di errore, come gli **errori in fase di esecuzione** (Runtime) che rappresentano problemi che si verificano mentre il programma è in funzione, spesso dovuto all'incompatibilità tra parte software (programma) e hardware oppure a operazioni non valide. Esempio è la divisione per 0.

Abbiamo inoltre anche gli **errori in fase di compilazione** che si verificano durante la fase di compilazione, causati da errori di sintassi o di scrittura del codice, come parentesi mancanti, variabili non dichiarati ecc., e impediscono al programma di avviarsi finché non vengono corretti.

**Instruction set architecture** (ISA) ed è lo standard che definisce come funziona una struttura di calcolo, in modo più preciso un processore. All'interno del documento possiamo trovare:

- *Stato della CPU*
  - Registri → quanti e quali.
  - Memoria
- *Istruzioni*
  - Formato
  - Tipologie → spostamento dati, aritmetico/logiche, controllo.
- *Semantica delle istruzioni* → come l'esecuzione di ciascuna istruzione modifica lo stato.

## Formato testo, oggetto ed eseguibile

Il formato binario ELF (Executable and Linkable Format) è un tipo di formato usato nei sistemi operativi per i file eseguibili, le librerie e i file oggetto. Contiene diverse parti tra cui:

- Informazioni sul programma
- Codice da eseguire
- Dati del programma

- Informazioni per il collegamento con librerie esterne

Non può essere eseguito o linkato su MacOS X. Allo stesso modo un file formato Mach-O non può essere eseguito o linkato in Linux; pertanto, i formati eseguibili non garantiscono la portabilità dei programmi.

## Disassemblare programmi in codice macchina: objdump

Disassemblare un programma in linguaggio macchina consiste nel tradurne le istruzioni nel codice assembly corrispondente in modo che possano essere analizzate da un programmatore.

**Linux: objdump -d (disassemblato)**

In ambiente Linux è possibile disassemblare un file oggetto o un file eseguibile usando il comando `objdump -d`.

Consideriamo il seguente semplice modulo `somma.c`:

```
int somma(int x, int y) {
    return x + y;
}
```

Compilandolo otteniamo un file oggetto `somma.o`:

```
$ gcc -c somma.c
```

che possiamo disassemblare come segue:

```
$ objdump -d somma.o
```

Il comando `objdump` invia l'output sul canale standard (di default è il terminale):

Figura 2.10: Esempio Objdump

## Traduzione in linguaggio macchina programma C

I sistemi di calcolo si basano su un certo numero di astrazioni che forniscono una visione più semplice del funzionamento della macchina. Due delle più importanti astrazioni sono:

- **Memoria** → vista come un grosso array di byte
- **ISA** (Instruction Set Architecture) → definisce:
  - Stato della CPU
  - Formato delle sue istruzioni
  - Effetto che le istruzioni hanno sullo stato

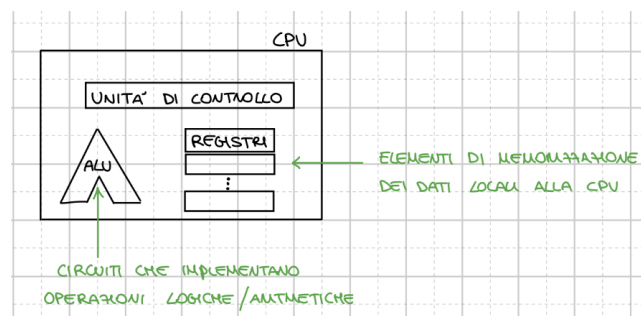


Figura 2.11: ALU

Per tradurre codice di alto livello in codice macchina, i compilatori si basano sulla descrizione astratta della macchina data dalla sua ISA. Le più diffuse sono:

- **IA32** → - IA32 → A, B, C, D, DI, SI, BP, SP, EP, IP, EFLAGS, i primi 7 sono general purpose, ovvero possono essere utilizzati a piacimento seguendo comunque le regole. **BP** e **SP** fanno parte della gestione dello stack. Abbiamo poi il registro **IP** (Instruction Pointer) è un registro non modificabile ma solo leggibile che viene gestito in maniera automatica dalla CPU, inserendo l'indirizzo della prossima istruzione da eseguire in memoria. Infine, abbiamo **EFLAGS**, anch'esso non modificabile che contiene dei flag che descrivono lo stato di funzionamento della CPU. Ciascun registro contiene 32 bit, così distribuiti:

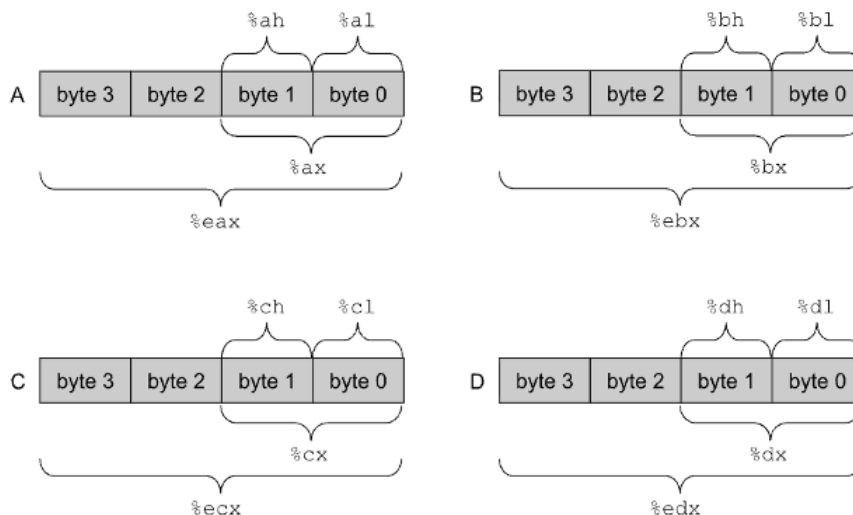


Figura 2.12: Registri IA32

Mentre gli altri registri hanno una suddivisione leggermente diversa:

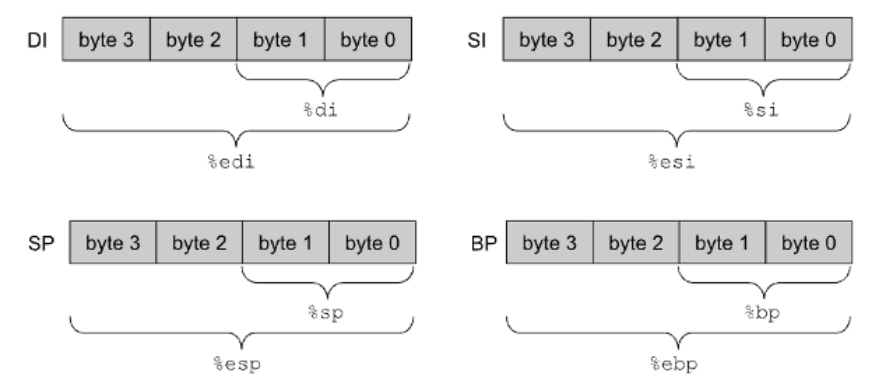


Figura 2.13: Registri IA32

I tipi macchina permettono di rappresentare sia numeri interi che numeri con virgola mobile. Vediamo in cosa corrispondono in C:

| Tipo di dato C | Tipo di dato macchina | Suffisso | Dimensione in byte |
|----------------|-----------------------|----------|--------------------|
| char           | Byte                  | b        | 1                  |
| short          | Word                  | w        | 2                  |
| int            | Double word           | l        | 4                  |
| long           | Double word           | l        | 4                  |
| long long      | non supportato        | -        | 8                  |
| puntatore      | Double word           | l        | 4                  |
| float          | Single precision      | s        | 4                  |
| double         | Double precision      | l        | 8                  |
| long double    | Extended precision    | t        | 12 (10 usati)      |

Figura 2.14: Tipi di Dati

- **X86-64** → descrive le architetture della famiglia di processori x86 a 64 bit, è retrocompatibile con IA32, ovvero le istruzioni di IA32 sono presenti anche nell'x86-64.

Si hanno 11 possibili forme di operandi. Per gli operandi di memoria, ci sono vari modi di indirizzamento che consentono di accedere alla memoria dopo averne calcolato un indirizzo:

| Tipo             | Sintassi                       | Valore Denotato                                | Nome Convenzionale                 |
|------------------|--------------------------------|------------------------------------------------|------------------------------------|
| <i>Immediato</i> | $\$imm$                        | $imm$                                          | immediato                          |
| <i>Registro</i>  | $E$                            | $R[E]$                                         | Registro                           |
| <i>Memoria</i>   | $imm$                          | $M[imm]$                                       | Assoluto                           |
| <i>Memoria</i>   | $E_{base}$                     | $M[R[E_{base}]]$                               | Indiretto                          |
| <i>Memoria</i>   | $imm(E_{base})$                | $M[imm + R[E_{base}]]$                         | Base e spiazzamento                |
| <i>Memoria</i>   | $(E_{base}, E_{indice})$       | $M[imm + R[E_{base}] + R[E_{indice}]]$         | Base e indice                      |
| <i>Memoria</i>   | $imm(E_{base}, E_{indice})$    | $M[imm + R[E_{base}] + R[E_{indice}]]$         | Base, indice e spiazzamento        |
| <i>Memoria</i>   | $(E_{indice}, s)$              | $M[R[E_{indice}] \cdot s]$                     | Indice e scala                     |
| <i>Memoria</i>   | $imm(E_{indice}, s)$           | $M[imm + R[E_{indice}] \cdot s]$               | Base, indice, scala e spiazzamento |
| <i>Memoria</i>   | $imm(E_{base}, E_{indice}, s)$ | $M[R[E_{base}] + R[E_{indice}] \cdot s]$       | Base, indice e scala               |
| <i>Memoria</i>   | $imm(E_{base}, E_{indice}, s)$ | $M[imm + R[E_{base}] + R[E_{indice}] \cdot s]$ | Base, indice, scala e spiazzamento |

Tabella 2.1: Operandi Possibili

Abbiamo anche due modi in cui un processore definisce l'ordine in cui vengono disposti in memoria i byte della rappresentazione di un valore numerico (binari):

- **Big-Endian** → il bit più significativo viene posto all'indirizzo più basso.
- **Little-Endian** → il bit meno significativo viene posto all'indirizzo più basso.

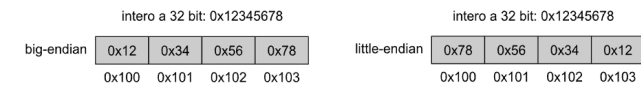


Figura 2.15: Big e Little Endian

L'architettura di IA32 è Little-Endian, mentre sono Big-Endian PowerPC e SPARC. Noi per il momento utilizzeremo i registri A, C, D.

# Capitolo 3

## Assembly

### 3.1 Funzioni

Per definire una funzione utilizziamo una "etichetta"/"label". Una funzione si definisce nel seguente modo →  
nome\_etichetta  
esempio funzione somma:

```
SOMMA:
```

Per rendere una funzione visibile da altri file oggetto usiamo la direttiva ".globl". esempio

```
.globl SOMMA  
SOMMA:
```

esempio prima parte, codice C → Assembly

```
int f(){  
    int x;  
    x = 10;  
    return x;  
}  
→  
.globl f  
f:  
#x -> eax //prende tutto il registro
```

#### Uso di parametri nelle funzioni

```
int f(int x){  
    return x+1;  
}  
→  
int f(int x){  
    int a;  
    a=x;  
    a++;  
    return a;  
}  
→  
.globl f  
f:  
#a->eax  
movl ,%eax  
inc %eax  
ret  
f.c  
f_eq.c  
f.s
```

### 3.2 Istruzioni

Per ogni istruzione dobbiamo definire la taglia dei dati su cui andrà a operare seguendo la sua semantica. In IA32 abbiamo tre tagli possibili:

| Esempio | Suffisso Assembly | Dimensione in byte |
|---------|-------------------|--------------------|
| Char    | b                 | 1                  |
| Short   | w                 | 2                  |
| int, T* | l                 | 4                  |

#### Istruzioni per lo spostamento

S viene spostato in D

```
mov S, D
```

le istruzioni hanno in generale un operando sorgente che specifica un valore di ingresso per l'operazione e un operando destinazione che identifica dove deve essere immagazzinato il risultato

- Operando Sorgente
  - Immediato → operando immagazzinato insieme all'istruzione stessa dichiarato con \$, esempio \$10
  - Registro → operando memorizzato in uno degli 8 registri interni dichiarato con %, esempio %eax
  - Memoria → operando memorizzato in memoria
- Operando Destinazione
  - Registro → il risultato dell'operazione viene memorizzato in uno degli 8 registri interni, sempre dichiarato con %
  - Memoria → il risultato dell'operazione viene memorizzato in memoria

**Regola:** non si usano mai valori immediati per l'operando D

**Regola:** non è possibile usare l'indirizzamento a memoria per entrambi gli operandi

esempio seconda parte, codice C → Assembly

|                                                                                                  |   |                                                                                         |
|--------------------------------------------------------------------------------------------------|---|-----------------------------------------------------------------------------------------|
| <pre>int f(){     int x; // dichiarazione     x = 10; // assegnazione     return x; }  f.c</pre> | → | <pre>.globl f f: #x -&gt; eax //prende tutto il registro movl \$10, %eax ret  f.s</pre> |
|--------------------------------------------------------------------------------------------------|---|-----------------------------------------------------------------------------------------|

esempio con char codice C → Assembly

|                                                                                                                          |   |                                                                                         |
|--------------------------------------------------------------------------------------------------------------------------|---|-----------------------------------------------------------------------------------------|
| <pre>char f(){     char p; // dichiarazione     p = 'a'; // assegnazione     printf("%c\n",p)     return 0; }  f.c</pre> | → | <pre>.globl f f: #x -&gt; al //prende tutto il registro movb \$0x61, %al ret  f.s</pre> |
|--------------------------------------------------------------------------------------------------------------------------|---|-----------------------------------------------------------------------------------------|

## Struttura dei dati in memoria

Per la struttura dei dati in memoria vediamo nello specifico il funzionamento dello StackPointer:

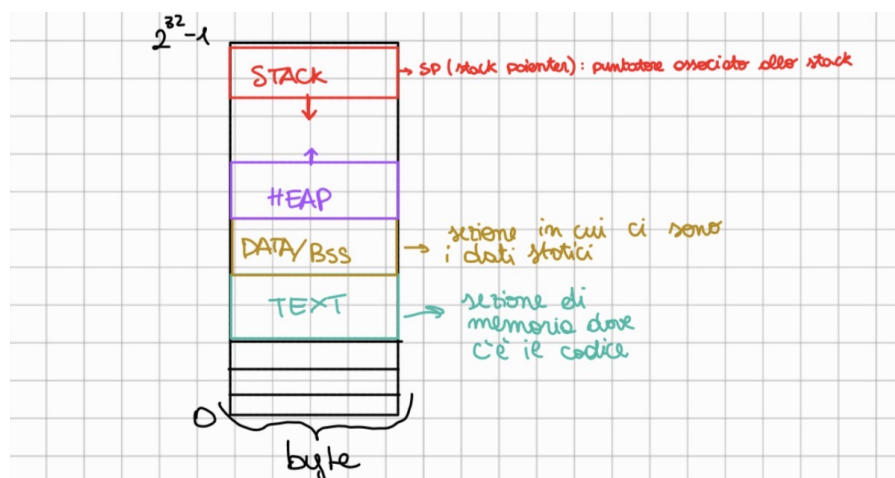


Figura 3.1: StackPointer

Vediamo però cosa accade di preciso nella memoria quando viene chiamata la funzione f:

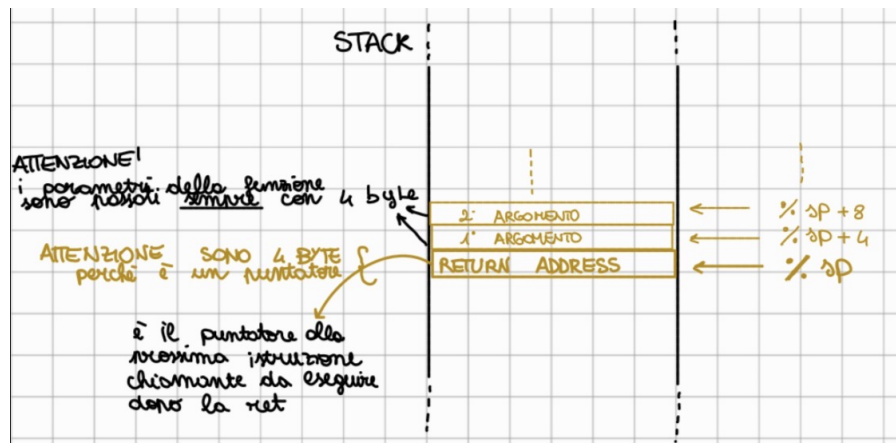


Figura 3.2: StackPointer

**Regola:** tutti i parametri attuali (argomenti) occupano 4 byte ciascuno nello stack pointer indipendentemente dal loro tipo.

## Accesso in memoria con spiazzamento

Per il "passaggio di parametri" in C, abbiamo l'equivalente in assembly in questo modo:

```
d(addr)
```

dove *d* indica lo spiazzamento e *addr* è il puntatore a memoria. Nell'esempio precedente va applicato nel seguente modo:

```
.globl f
f:
    #a -> eax
    movl 4(%esp), %eax
    incl %eax
    ret
```

Quindi avremo che nel registro *eax* sarà salvato il primo parametro passato dalla funzione. In generale avremo che:

```
SP + 4 -> 4(%esp)
```

## Operazioni aritmetico/logiche

```
add S, D | D <- D + S
sub S, D | D <- D - S
mul S, D | D <- D * S senza segno
imul S, D | D <- D * S con segno
and S, D | D <- D & S bit a bit
or S, D | D <- D ^ S bit a bit
xor S, D | D <- -D   viene utilizzato per l'inizializzazione delle variabili,
                    preferito a movl poichè modifica i flag
not S, D | D <- !D
inc D    | D <- D + 1
dec D    | D <- D - 1
neg D    | D <- -D
```

## Moltiplicazione senza segno

```
mul S | D <- D * S
      |   |   |
      |   |   |__ registro 32bit/immediato
%edx:%eax %eax
```

esempio:

|                |          |         |
|----------------|----------|---------|
|                | eax      | edx     |
| movl \$2, %eax | 0...010  | ?       |
| mull \$5       | 0..01010 | 0.....0 |

## Moltiplicazione con segno

```
imul S | D <- D * S
      |   |   |
      |   |   |__ registro 32bit/immediato
%edx:%eax %eax
```

esempio:

|                 |          |          |
|-----------------|----------|----------|
|                 | eax      | edx      |
| movl \$-2, %eax | 11....10 | ?        |
| imull \$5       | 1.....   | 11111111 |

## Divisione

```
div S | D <- D / S
      |   |
      |   |
    eax,edx eax:edx
```

```
idivl r32/imm32 | eax <- edx:eax / S    // quoziente
                 | edx <- edx:eax % S    // modulo
```

### 3.3 Gestione del flusso di esecuzione

## Salto incondizionato

|         |                   |       |
|---------|-------------------|-------|
| ...     |                   | ...   |
| ...     |                   | ...   |
| goto L; |                   | jmp L |
| ...     |                   | ...   |
| ...     |                   | ...   |
| L:      | $\longrightarrow$ | L:    |
| ...     |                   | ...   |
| ...     |                   | ...   |

|   |          |
|---|----------|
| c | Assembly |
|---|----------|

## Salto condizionato

Le istruzioni di salto condizionato in ASM si sviluppano in due fasi:

1. Esecuzione di istruzione aritmetico/logica che modifica EFLAGS
2. Esecuzione di un salto condizionato al valore di EFLAGS

La proprietà valutata in 2. per il salto è detta **condition code (cc)**

esempio:

Risultato R

| Condition Code (cc) | Proprietà di R |
|---------------------|----------------|
| e, z                | $R == 0$       |
| ne, nz              | $R != 0$       |
| Con segno:          |                |
| g                   | $R > 0$        |
| l                   | $R < 0$        |
| ge                  | $R \geq 0$     |
| le                  | $R \leq 0$     |
| Senza segno:        |                |
| a                   | $R > 0$        |
| b                   | $R < 0$        |
| ae                  | $R \geq 0$     |
| be                  | $\leq 0$       |

Il salto condizionato è implementato da:

```
jcc L
```

esempio:

```
subl %eax, %ecx    # R = ecx - eax
je    L            # eax = ecx -> R==0
```

esempio:

```
subl %eax, %ecx
jge    L            # ecx >= eax -> R>=0
```

## Registro EFLAGS

È un registro contenente diversi flag (bit) dedicati a rappresentare lo stato della CPU a seguito dell'esecuzione dell'ultima istruzione:

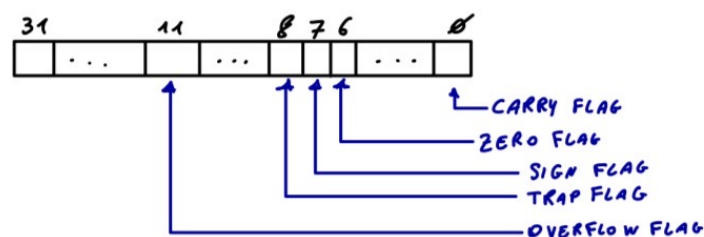


Figura 3.3: EFLAGS

- **Bit 0** → il *carry flag* serve principalmente a indicare un riporto (carry) o un prestito (borrow) nelle operazioni aritmetiche senza segno.
- **Bit 6** → la *zero flag* ha come valore 1 se l'operazione precedente ha avuto come risultato zero, altrimenti varrà 0.
- **Bit 7** → il *sign flag* quando l'operazione ha come risultato negativo vale 1 altrimenti 0
- **Bit 8** → il *trap flag* viene usato dal debugger
- **Bit 11** → l'*overflow flag* indica quando si verifica un overflow nelle operazioni aritmetiche con numeri con segno. Questo succede quando il risultato non può essere rappresentato con il numero di bit disponibile.

7) 8) usato dal debugger 11) per i risultati non rappresentabili nei nostri bit allora il flag varrà 1

### 3.4 Istruzioni di Condizione

|                                                    |   |                                                                          |   |                                                           |
|----------------------------------------------------|---|--------------------------------------------------------------------------|---|-----------------------------------------------------------|
| <pre> if(c&gt;a){     istr 1; } istr 2; f.c </pre> | → | <pre> r = c - a; if(r&lt;=0)goto L; istr 1; L:     istr 2; f_eq.c </pre> | → | <pre> subl %eax, %ecx jle L #istr 1 L: #istr 2 f.s </pre> |
|----------------------------------------------------|---|--------------------------------------------------------------------------|---|-----------------------------------------------------------|

In questo caso nel codice assembly abbiamo "sporcato" il registro `%ecx`. Per non "sporcare" il registro con SUB usiamo *CMP*, che ha la stessa funzione della sottrazione ma non salva in nessun registro.

```
CMP S, D | D - S
```

Ma altera i valori di EFLAGS esattamente come SUB. Avendo quindi un codice corretto in questo modo:

|                                                                           |   |                                                                                         |   |                                                                           |
|---------------------------------------------------------------------------|---|-----------------------------------------------------------------------------------------|---|---------------------------------------------------------------------------|
| <pre> if(test){     blocco A; }else{     blocco B; } blocco C; f.c </pre> | → | <pre> if(!test)goto E; blocco A goto F; E:     blocco B; F:     blocco C; f_eq.c </pre> | → | <pre> CMP S, D jcc E #blocco A jmp F E: #blocco B F: #blocco C f.s </pre> |
|---------------------------------------------------------------------------|---|-----------------------------------------------------------------------------------------|---|---------------------------------------------------------------------------|

Inoltre esiste un'altra funzione simile chiamata *Test* se dobbiamo fare una comparazione a 0, che è equivalente:

```

cmp 0, D | D - 0
test D, D | D & D

```

### Cicli While-For

Vedremo ora come fare i cicli in Assembly, cominciando con il *while*. Consideriamo lo schema generale dell'istruzione, se il test effettuato è vero, viene eseguito il blocco e si ritorna al test, altrimenti si prosegue con l'istruzione successiva al while, vediamo un esempio di traduzione in Assembly:

|                                                         |   |                                                                            |   |                                                                                   |
|---------------------------------------------------------|---|----------------------------------------------------------------------------|---|-----------------------------------------------------------------------------------|
| <pre> while(test){     blocco A; } blocco B; f.c </pre> | → | <pre> L: if(!test) goto E; blocco A; goto L; E:     blocco B f_eq.c </pre> | → | <pre> L:     cmp S, D     jcc E     blocco A     jmp L E:     blocco B f.s </pre> |
|---------------------------------------------------------|---|----------------------------------------------------------------------------|---|-----------------------------------------------------------------------------------|

Quindi non c'è una vera e propria traduzione in Assembly per il while ma occorrerà "smontare" il codice C, in un equivalente in grado di farci giostrare bene i jump, condizionali e non. Vediamo un esempio:

|                                                                                                                                                                          |                   |                                                                                                                                                                                      |                   |                                                                                                                                                                                                       |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>int sum(int n){     int a = 0;     int c = n;     while(c&gt;0){         a = a + c;         c--;     }     return a; }</pre> <p style="text-align: center;">f.c</p> | $\longrightarrow$ | <pre>int sum(int n){     int a = 0;     int c = n; L:     if(c&lt;=0) goto E;     a = a + c;     c--;     goto L; E:     return a; }</pre> <p style="text-align: center;">f_eq.c</p> | $\longrightarrow$ | <pre>.global sum sum:     xorl %eax, %eax     movl 4(%esp), %ecx L:     testl \$0, %ecx     jle E     addl %ecx, %eax     decl %ecx     jmp L E:     ret</pre> <p style="text-align: center;">f.s</p> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Consideriamo invece lo schema generale di una istruzione **for**. Si segue dapprima l'inizializzazione e poi si effettua il test, se il test risulta vero allora viene eseguito il blocco, altrimenti si prosegue con l'istruzione successiva al for. In Assembly può essere tradotta in questo modo:

|                                                                                                    |                   |                                                                                                             |
|----------------------------------------------------------------------------------------------------|-------------------|-------------------------------------------------------------------------------------------------------------|
| <pre>for(init; test; update){     blocco A } blocco B</pre> <p style="text-align: center;">f.c</p> | $\longrightarrow$ | <pre>init; while(test){     blocco A     update; } blocco B</pre> <p style="text-align: center;">f_eq.c</p> |
|----------------------------------------------------------------------------------------------------|-------------------|-------------------------------------------------------------------------------------------------------------|

Quindi passeremo dal for al while corrispondente che siamo già in grado di tradurre. Vediamolo meglio con un esempio:

|                                                                                                                                                                                                                                  |                   |                                                                                                                                                                                                                                                                             |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>unsigned fact(unsigned n){     unsigned a = 1;     unsigned c;     unsigned d = n;     c = 2; L:     if(c&gt;d) goto E;     a = a * c;     c++;     goto L; E:     return a; }</pre> <p style="text-align: center;">f.c</p> | $\longrightarrow$ | <pre>.globl fact fact:     movl \$1, %eax     movl \$2, %ecx     movl 4(%esp), %edx L:     cmpl %edx, %ecx     # poichè senza segno     # usiamo ja e non jcc     ja E     imull %ecx, %eax     incl %ecx     jmp L E:     ret</pre> <p style="text-align: center;">f.s</p> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Quindi per un ciclo for procederemo in questo modo: *for*  $\rightarrow$  *while*  $\rightarrow$  *if*.

### 3.5 Puntatori

Un puntatore in C di tipo  $T^*$  contiene un indirizzo di memoria dove è conservato un valore di *tipo*  $T$ . In Assembly per indirizzare una locazione di memoria il cui indirizzo è contenuto in un registro usiamo l'operatore

|                                              |                   |                                                            |
|----------------------------------------------|-------------------|------------------------------------------------------------|
| <pre> int *a; . . . '(): *a = 0;  f.c </pre> | $\longrightarrow$ | <pre> #int *a -&gt; eax . . . movl \$0, (%eax)  f.s </pre> |
|----------------------------------------------|-------------------|------------------------------------------------------------|

Vediamolo in modo grafico:

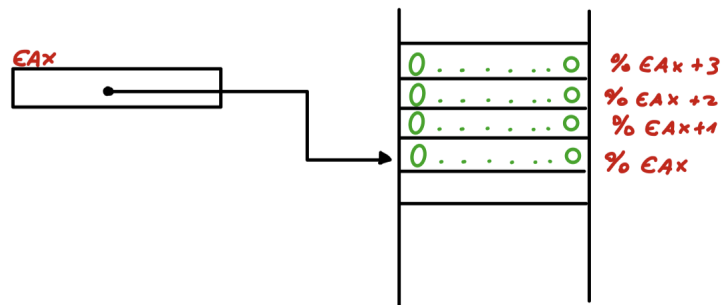


Figura 3.4: Come occupa EAX la memoria in caso di int

Inoltre vale anche: `int c = *a;  $\rightarrow$  movl(%eax), %ecx`

Nel caso dei puntatori la "l" del tipo di dato dipende dalla dimensione del tipo del puntatore (`sizeof(T)`).

Esempio:

|                                               |                   |                                                              |
|-----------------------------------------------|-------------------|--------------------------------------------------------------|
| <pre> char *p; . . . char a = *p;  f.c </pre> | $\longrightarrow$ | <pre> #char *p -&gt; edx . . . movb (%edx), %eax  f.s </pre> |
|-----------------------------------------------|-------------------|--------------------------------------------------------------|

Quando accediamo al parametro passato per valore ad una funzione, stiamo in effetti usando un puntatore con l'operando '()'. Per capirci meglio:

|                                                   |                   |                                      |
|---------------------------------------------------|-------------------|--------------------------------------|
| <pre> void f(int x){     int a = x; }  f.c </pre> | $\longrightarrow$ | <pre> movl 4(%esp), %eax  f.s </pre> |
|---------------------------------------------------|-------------------|--------------------------------------|

In questi casi noi abbiamo sempre usato dei puntatori verso lo stackpointer:

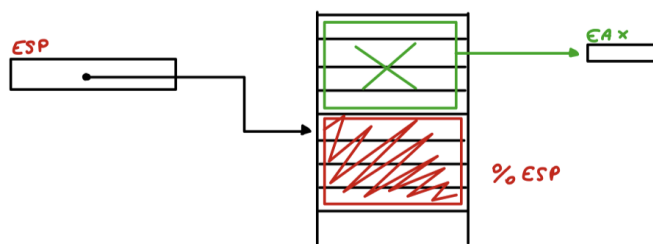


Figura 3.5: Memoria Graficamente

Con lo stesso ragionamento possiamo capire come tradurre gli array in Assembly, molto semplicemente basta accedere alla posizione di memoria corrispondente alla posizione dell'array:

```
int a[7] = 12;                                movl $12, 28(%eax)
f.c                                             → f.s
```

Vediamolo meglio però con un esempio:

```
void f(char *x){                               movb 4(%esp), %eax
    char c = x[1];                             movb 1(%eax), %ecx
}                                             →
f.c                                             f.s
```

**Importante:** in una istruzione IA32 è possibile l'accesso a memoria per al più un operando, inoltre l'operatore '()' può essere usato solo sui registri e questi registri devono contenere indirizzi validi di memoria.

Vediamo il tutto con un esempio:

```
void times2(short *p){                         void times2(short *p){           .globl times2
    *p = *p * 2;                               short *a = p;                   times2:
}                                                short cx = *a;                  movb 4(%esp), %eax
                                                *a += cx;                      #essendo short non c'è
                                                                              #bisogno della 'e'
                                                                              movw (%eax), %cx
                                                                              #equivalente fare:
                                                                              #imulw $2, %ecx
                                                                              addw $cx, $cx
                                                                              movw $cx, (%eax)
                                                                              ret
f.c                                             → f_eq.c                       → f.s
```

## Accesso in memoria con base, indice e spiazzamento

$D(x) \rightarrow$  accede alla locazione con indirizzo  $x + D$ , in realtà la sintassi completa dell'operatore '()' è:

`imm(base, indice, scala)`

dove:

- *imm*  $\rightarrow$  rappresenta lo spiazzamento definito come valore costante
- *base*  $\rightarrow$  è l'indirizzo di memoria di base a cui deve accedere. **DEVE** essere un registro e **DEVE** essere valido
- *indice*  $\rightarrow$  indice dell'elemento a cui si vuole accedere a partire da BASE e **DEVE** essere un registro opzionale
- *scala*  $\rightarrow$  dimensione in byte di ogni elemento. **DEVE** essere un valore costante: 1, 2, 4 ed 8 (l'8 solo a 64 bit). Ed è opzionale

L'uso dell'operatore permette di accedere alla memoria con **indirizzo**  $\rightarrow$  Base + Indice  $\times$  Scala + Imm

Esempio:

```
int a[7] = 12;                                movl $7, %ecx
                                                movl $12, (%eax, %ecx, 4)
                                                # corrisponde a
                                                # movl $12, 28(%eax)
f.c                                             → f.s
```

Vediamo un esempio con un codice in C:

```

short get(short* v,
          unsigned n, unsigned i){
    if(i>=n) return -1;
    return v[i];
}
f.c

```

→

```

short get(short* v,
          unsigned n, unsigned i){
    short *c = v;
    unsigned d = i;
    unsigned a = n;
    short ax;
    if(d<a) goto E;
    ax = -1;
    return ax;
E:
    ax = c[d];
    return ax;
}
f_eq.c

```

→

```

.globl get
get:
    movl 4(%esp), %ecx
    movl 12(%esp), %edx
    movl 8(%esp), %eax
    cmpl %eax, %edx
    jnb E
    movlw $-1, %ax
    ret
E:
    movw (%ecx,%edx,2), %ax
    ret
f.s

```

Il tipo del puntatore definisce inoltre il valore da aggiungere all'indirizzo base. Inoltre opereremo su puntatori **sempre** con operatori a 4 byte. In C esiste un'aritmetica dei puntatori **implicita**, ovvero non dobbiamo preoccuparci di dove puntano i puntatori; mentre in Assembly l'aritmetica dei puntatori è **esplicita**.

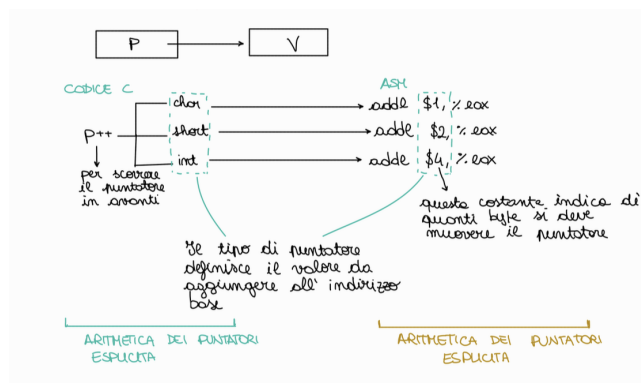


Figura 3.6: Puntatore

### 3.6 Chiamate a Funzione

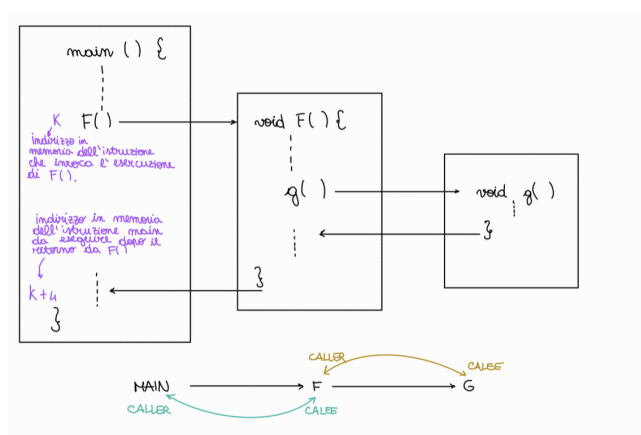


Figura 3.7: Chiamata a funzione

In questo esempio cominciamo definendo queste funzioni. Avremo che il main sarà il nostro **caller**, mentre f sarà il **callee**. Tuttavia, anche tra f e g esiste una relazione: in questo caso f sarà il **caller**, mentre g il **callee**. Quindi una funzione può assumere ruoli diversi a seconda del rapporto che ha con le altre funzioni. La stack è fondamentale per gestire le chiamate a funzione e per fornire spazio di memoria locale. Ogni chiamata a funzione crea uno **stack frame** (o record di attivazione), che contiene variabili locali, parametri e altre informazioni necessarie all'esecuzione della funzione. Gli stack frame sono organizzati come una sorta

di **lista collegata**: il registro ebp punta allo **stack frame** della funzione attualmente in esecuzione, e ogni frame contiene un riferimento a quello della funzione chiamante. Questo permette, ad esempio, a un debugger di ricostruire la sequenza di chiamate dal main fino alla funzione corrente. Tuttavia, il collegamento tra **stack frame** tramite ebp è **opzionale**.

**Importante:** per convenzione, nel momento in cui si effettua un'istruzione CALL, la base della stack puntata dal registro %esp deve essere sempre a un indirizzo multiplo di 16.

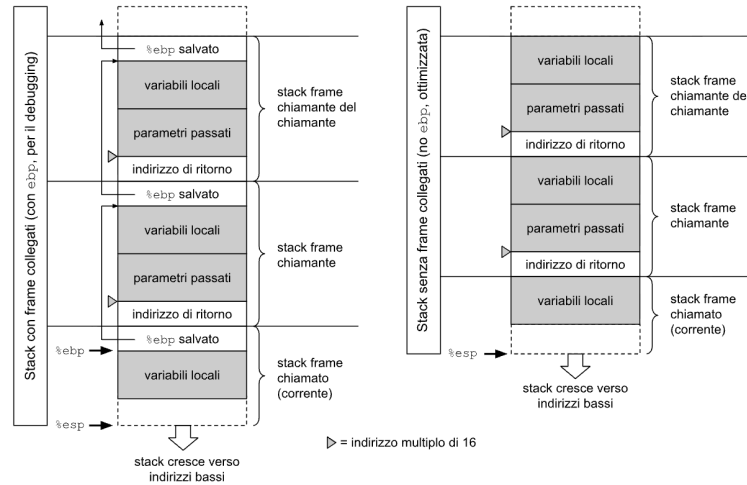


Figura 3.8: Stack Pointer specifico alle funzioni

Vediamo però cosa può contenere il frame/record di attivazione di una funzione:

- Variabili locali
- Copie dei registri → ebx, edi, esi, ebp
- Parametri per le funzioni chiamate
- Indirizzo di ritorno delle funzioni chiamate

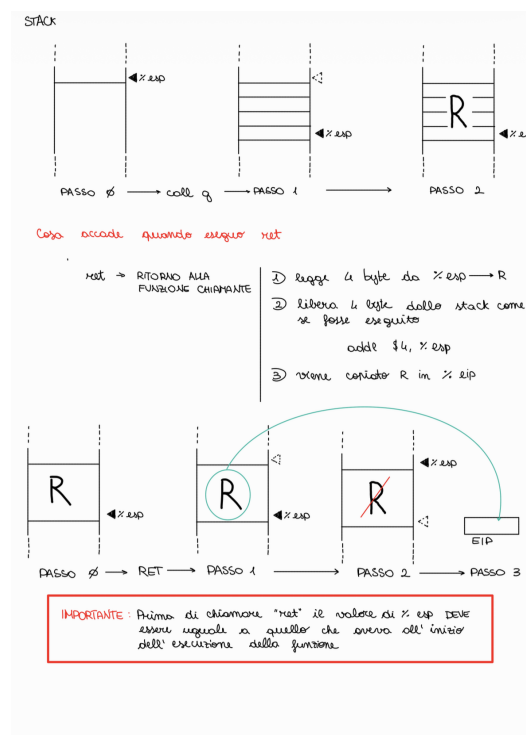


Figura 3.9: Cosa accade quando utilizziamo il ret?

**Importante:** prima di chiamare ret il valore %esp deve essere uguale a quello che aveva all'inizio dell'esecuzione

|                                                                                                                                                |                                                                                    |   |                                                                                                                                    |
|------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|---|------------------------------------------------------------------------------------------------------------------------------------|
| <pre>int f(){     int a = 1 + g();     //g è definita nel main     //che qui non scriviamo     //g () { return 10 }     return a; }  f.c</pre> | <pre>int f(){     int a;     a = g();     a = a + 1;     return a; }  f_eq.c</pre> | → | <pre>.globl f f:     #e -&gt; eax     call g     #il valore è già in eax     #essendo il ret di g     incl %eax     ret  f.s</pre> |
|------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|---|------------------------------------------------------------------------------------------------------------------------------------|

Vediamo invece cosa accade nello Stack Pointer quando abbiamo un passaggio di parametri

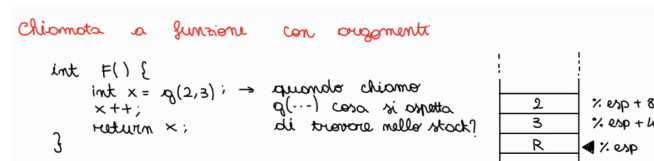


Figura 3.10: Passaggio parametri

Quindi avremo che lo SP:

1. Riserva spazio per i parametri come se fosse: `subl $8, %esp`, dove 8 è uguale al numero di parametri passati per 4 (spazio occupato in memoria)
2. Copia i valori attuali dei parametri nello spazio riservato in ordine inverso
3. Chiamata di funzione: `call g`

|                                                                                                                               |   |                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-------------------------------------------------------------------------------------------------------------------------------|---|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>int f(){     int x = g(2,3);     //g è definita nel main     //che qui non scriviamo     x++;     return x; }  f.c</pre> | → | <pre>.globl f f:     #8 perchè 4 * 2 parametri     subl \$8, %esp      #prologo     #scrivo i parametri attuali     #nello spazio che ho riservato     movl \$3, 4(%esp)     movl \$2, 8(%esp)     call g     #dato che se g restituisce qualcosa     #lo restituisce in eax avremo     #eax già modificato     incl %eax     #riportiamo %esp al valore "originale"     addl \$8, %esp      #epilogo     ret  f.s</pre> |
|-------------------------------------------------------------------------------------------------------------------------------|---|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

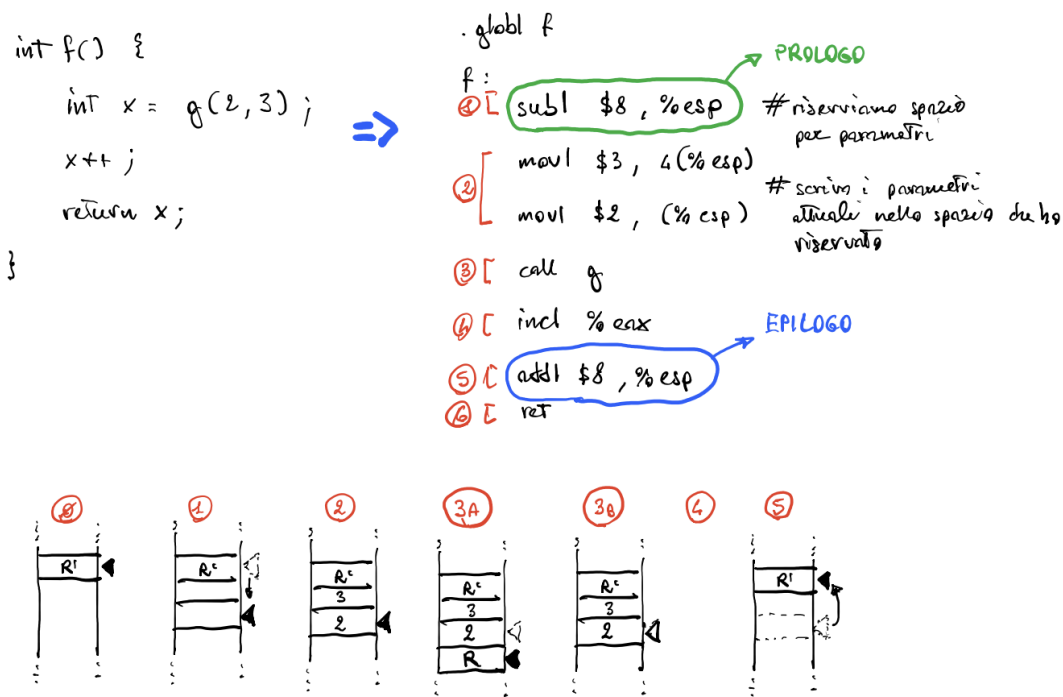


Figura 3.11: Struttura del codice

La lista di stack frame viene gestita mediante un codice di **prologo** all'inizio di una funzione e un codice di **epilogo** alla fine:

- *prologo* → salva nello stack il contenuto del registro **ebp**, che punta allo stack frame del chiamante, tramite l'istruzione `pushl %ebp`. Successivamente il registro **ebp** viene aggiornato per puntare alla posizione corrente dello stack, contenuta nel registro **esp**, mediante l'istruzione `movl %esp, %ebp`. In questo modo **ebp** viene a puntare allo stack frame della funzione corrente.
- *epilogo* → ripristina il valore del registro **ebp** presente prima dell'attivazione della funzione corrente eseguendo l'istruzione `popl %ebp`. In questo modo **ebp** tornerà a puntare allo stack frame della funzione chiamante.

In gcc, è possibile omettere il collegamento fra stack frame compilando con l'opzione `-fomit-frame-pointer`. In questo modo, non verranno generati prologo ed epilogo: la funzione sarà più veloce e compatta, ma il debugging potrebbe essere più difficoltoso.

L'esecuzione di una funzione può sovrascrivere i registri utilizzati dal chiamante. Se il chiamante vuole garantire che il contenuto di un registro non venga modificato durante l'invocazione di una funzione, è necessario salvarne il valore da qualche parte, generalmente nello stack. Esistono due possibili strategie:

1. **caller-save**: il registro viene salvato nello stack dal chiamante prima dell'invocazione della funzione e ripristinato subito dopo il ritorno. La funzione chiamante (caller) deve salvare il valore di questi registri prima di chiamare un'altra funzione. La funzione chiamata invece può modificare liberamente il valore di questi registri.
2. **callee-save**: il registro viene salvato nello stack dal chiamato prima di eseguire il corpo della funzione e ripristinato prima di ritornare al chiamante. La funzione chiamata (callee) deve preservare il valore che questi registri riservano nel momento in cui viene eseguita la call, e deve ripristinarli prima di eseguire il `ret` (il salvataggio avviene nel prologo e il ripristino nell'epilogo).

Per convenzione, alcuni registri vengono salvati dal chiamante, e altri dal chiamato:

- **Registri caller-save** → A, C, D.
- **Registri callee-save** → B, DI, SI, SP, BP.

Ma come si salva il valore di un registro?

1. Salviamo il valore di `%reg` (registro qualsiasi) nella stack
2. Eseguiamo le operazioni che "cambiano" il contenuto di `%reg`

3. Recuperiamo dalla stack il valore di %reg

Ci sono diversi modi per farlo, vediamo il *modo esplicito*:

- 1) `subl $4, %esp`  
`movl %reg, (%esp)`
- 2) `#%reg` viene sporcato
- 3) `movl (%esp), %reg`  
`addl $4, (%esp)`

## Metodo *Push + Pop*

- 1) `push %reg`
- 2) `# . . .`
- 3) `popl %reg`

In questi due metodi abbiamo che l'1) e il 3) sono equivalenti tra loro quindi possiamo usare entrambi a nostro piacimento.

|                                                     |   |                                                                                                                                               |   |                                                                                                                                                                                                     |
|-----------------------------------------------------|---|-----------------------------------------------------------------------------------------------------------------------------------------------|---|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>int f(){<br/>    return g() + h();<br/>}</pre> | → | <pre>int f(){<br/>    int a;<br/>    a = g();<br/>    int c;<br/>    c = a;<br/>    a = h();<br/>    a = a + c;<br/>    return a;<br/>}</pre> | → | <pre>.globl f<br/>f:<br/>    #a -&gt; EAX<br/>    #c -&gt; ECX<br/>    call g<br/>    movl %eax, %ecx<br/>    pushl %ecx<br/>    call h<br/>    popl %ecx<br/>    addl %ecx, %eax<br/>    ret</pre> |
| <code>f.c</code>                                    |   | <code>f_eq.c</code>                                                                                                                           |   | <code>f.s</code>                                                                                                                                                                                    |

Avremmo potuto usare anche il metodo esplicito ed avremmo avuto lo stesso identico risultato.

Vediamo però che registri usare a seconda delle situazioni:

- Se implemento una funzione che **NON** chiama altre funzioni:
  - uso liberamente i caller saved (A, C, D).
  - se non sono sufficienti allora uso i callee saved (B, DI, SI, BP) ma preservando il valore (usando prologo + epigolo).
- Se implemento una funzione che chiama altre funzioni:
  - Uso i callee saved (B, DI, SI, BP) preservandone il valore
  - Uso liberamente i caller saved (A, C, D) per valori temporanei **SOLO** in blocchi di istruzioni che **NON** chiamano altre funzioni,
  - Altrimenti uso i caller saved (A, C, D) ma ne preservo il valore (push + pop) prima di **QUALSIASI** chiamata a funzione.

```

int g(int x);

int f(){
    int y = 1, z = 2;
    return g(x+1) + y + z;
}

→                                     →

int f(){
    int c;
    int di; // y
    int si; // z
    c = x;
    di = 1;
    si = 2;
    c++;
    int a = g(c);
    a = a+di;
    a = a + si;
    return a;
}

                                     .globl f
f:
    #Prologo
    pushl %edi # -> y
    pushl %esi # -> z
    subl $4, (%esp)

    movl 16(%esp), %ecx
    movl $1, %edi
    movl $2, %esi
    incl %ecx
    movl %ecx, (%esp)
    call g
    addl %edi, %eax
    adll %esi, %eax
    #Epilogo
    addl $4, (%esp)
    popl %edi
    popl %esi
    ret

f.c                                     f_eq.c                                     f.s

```

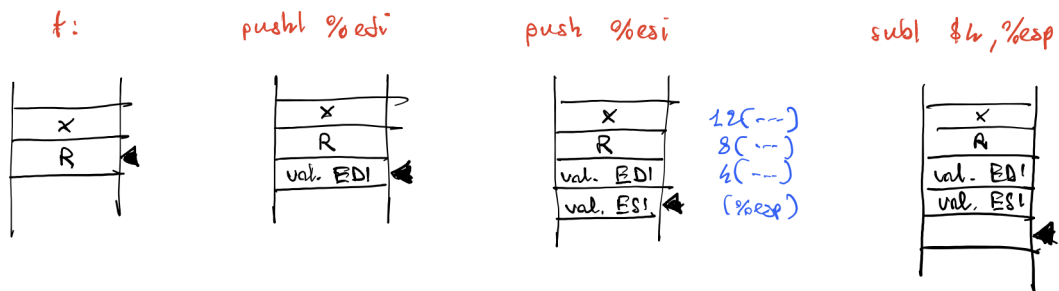


Figura 3.12: Graficamente

Esercizio riassuntivo su quello fatto fino ad ora:

```
int is_negative(int x);

int f(int *v, unsigned n){
    unsigned i, cnt = 0;
    for(i = 0; i < n; i++){
        cnt += is_negative(v[i]);
    }
    return cnt;
}

f.c
```

```
int f(int *v, unsigned n){
    int *di = v;
    unsigned si = n;
    unsigned b = 0;
    unsigned bp = 0;
L:
    if(b >= si) goto E;
    int c = di[b];
    unsigned a = is_negative(c);
    bp = bp + a;
    b++;
    goto L;
E:
    a = bp;
    return a;
}

f_eq.c
```

→

```
.globl f
f:
    pushl %edi
    pushl %esi
    pushl %ebx
    pushl %ebp
    subl $4, %esp

    movl 24(%esp), %edi
    movl 28(%esp), %esi
    xorl %ebx, %ebx
    xorl %ebp, %ebp

L:
    cmpl %esi, %ebx
    jae E
    movl (%edi,%ebx,4), %ecx
    #4 * ebx (=i) quindi muove di 4 * i %edi
    #per prendere tutti i valori del vettore
    movl %ecx, (%esp)
    call is_negative
    addl %eax, %ebp
    incl %ebx
    jmp L

E:
    movl %ebp, %eax
    ret
```

### 3.7 Conversione del tipo (cast)

Partiamo dal fatto che in Assembly il cambio di tipo, così come lo intendiamo nei linguaggi ad alto livello, non esiste. In questo contesto dobbiamo invece distinguere due casi principali:

1. **Operazioni di troncamento (downcast)** → ad esempio: `int c = 0x000CAFE` (32 bit) → `char a = c` (8 bit). In questo caso si mantengono solo i bit meno significativi, con conseguente perdita di informazione. (Esempio implementabile con istruzioni come `movl`).
2. **Operazioni di promozione (upcast)** → in questo caso il valore viene esteso a un tipo più grande. È importante distinguere tra estensione con segno e senza segno, a seconda che il dato originale sia signed oppure unsigned.

|       |  |                                   |        |
|-------|--|-----------------------------------|--------|
| con   |  | <code>char c='s';</code>          | 8 bit  |
| segno |  | <code>int a=c;</code>             | 32 bit |
| senza |  | <code>unsigned char c='s';</code> | 8 bit  |
| segno |  | <code>unsigned int a=c;</code>    | 32 bit |

Con questo tipo di operazione non si ha perdita di informazione, a differenza del downcast. Tuttavia, è necessario garantire esplicitamente che il segno venga preservato.

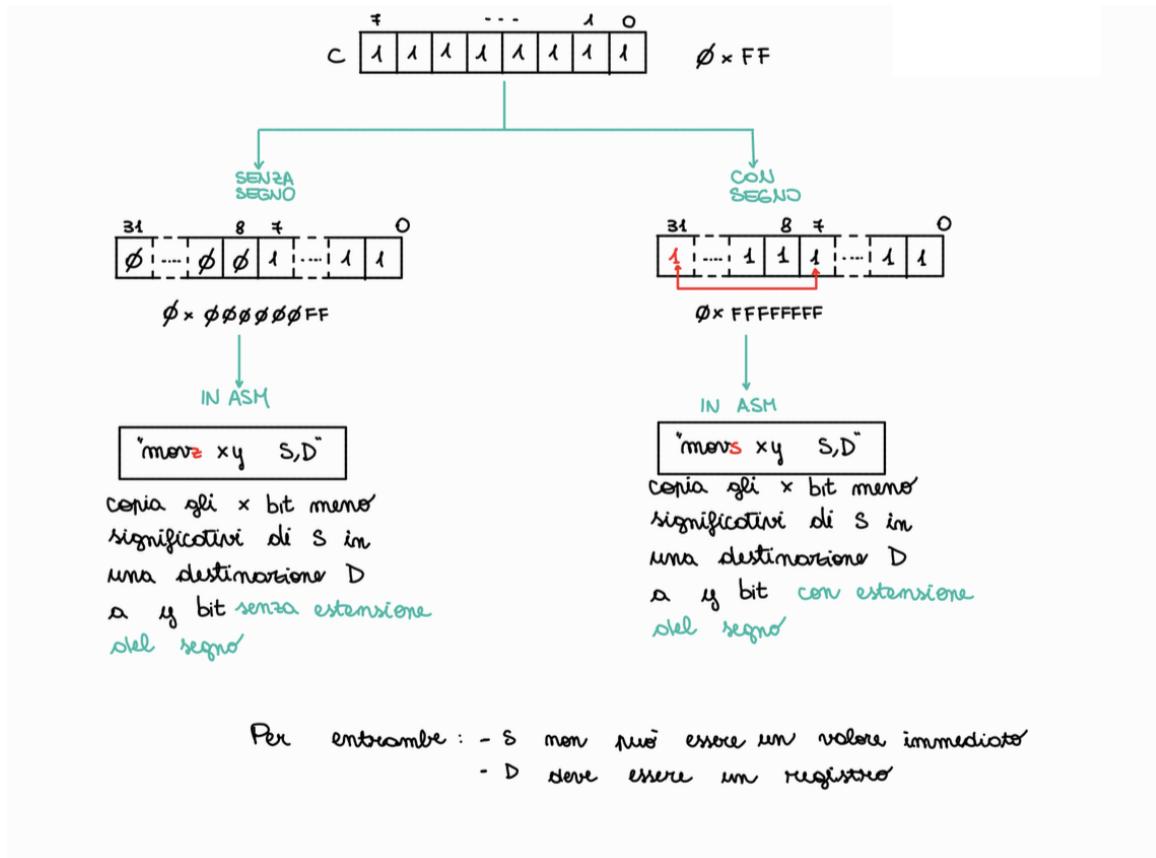


Figura 3.13: Estensione con segno da char a int

Nel caso di una promozione da `char` a `int`, per preservare il segno è necessario propagare il bit più significativo del `char` (bit di segno) su tutti i bit aggiuntivi dell'`int`.

Per questo motivo si distinguono due tipi di istruzioni:

- estensione con segno (*sign extension*);
- estensione senza segno (*zero extension*).

movs xy S, D -> copia gli x bit meno significativi di S in una destinazione D a y bit con estensione del segno  
 movz xy S, D -> copia gli x bit meno significativi di S in una destinazione D a y bit senza estensione del segno

Per entrambe vale che: S non può essere un valore immediato, e D deve essere un registro. Per le operazioni con segno utilizzeremo i seguenti comandi:

|         | char c           | short c          | int c           |
|---------|------------------|------------------|-----------------|
| char r  | movb %cl, %al    |                  |                 |
| short a | movsbw %cl, %ax  | movw %cx, %ax    |                 |
| int a   | movsbl %cl, %eax | movswl %cx, %eax | movl %ecx, %eax |

Per le operazioni senza segno:

|         | char c           | short c          | int c           |
|---------|------------------|------------------|-----------------|
| char r  | movb %cl, %al    |                  |                 |
| short a | movzbw %cl, %ax  | movw %cx, %ax    |                 |
| int a   | movzbl %cl, %eax | movzwl %cx, %eax | movl %ecx, %eax |

esempio:

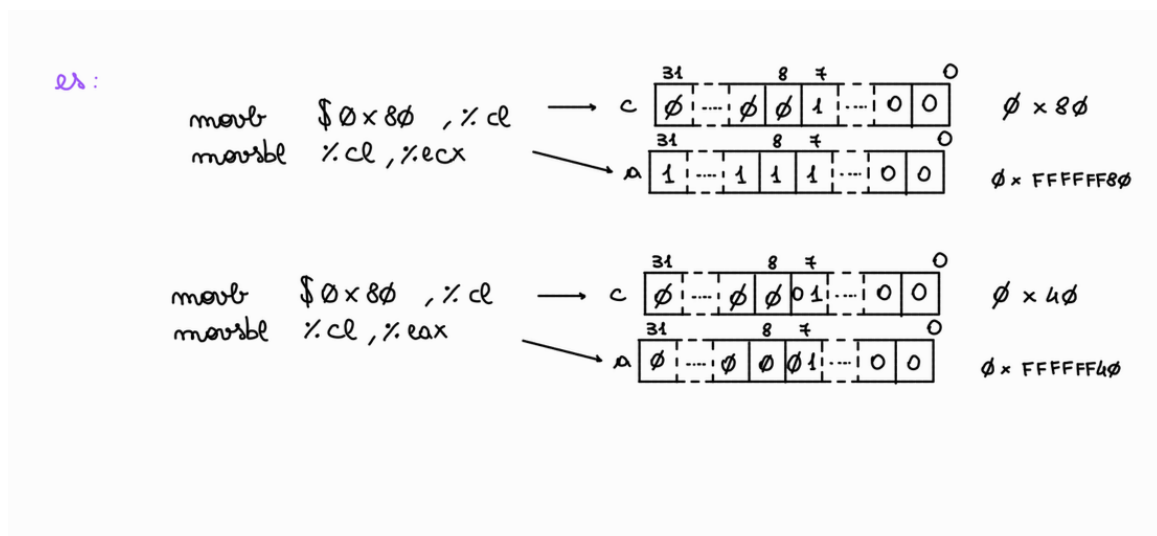


Figura 3.14: Esempio

Esempio:

```
int f(char *p, char *q){
    int x = *p + *q;
    return x
}
```

f.c

```
.globl f
f:
    movl 4(%esp), %eax # A <-> p
    movl 8(%esp), %ecx # C <-> q
    movsbl (%eax), %eax
    movsbl (%ecx), %ecx
    addl %ecx, %eax
    ret
f.s
```

Altro esempio:

|                                                                                                                                                                                    |                   |                                                                                                                                                                                                                                                                                  |                   |                                                                                                                                                                                                                                                                                                                                                                                             |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> int is_vowel(int c); // scritto nel mai e // restituisce 0/1 int num_vowels(char *s){     int cnt = 0;     while(*s)         cnt += is_vowel(*s++);     return cnt; } </pre> | $\longrightarrow$ | <pre> int is_vowel(int c); int num_vowels(char *s){     char *b = s;     int di = 0 // di &lt;-&gt; cnt L:     if(*b == 0) goto E;     int a = *b;     //ATTENZIONE: promozione     a = is_vowel(a);     di = di + a;     b++;     goto L; E:     a = di;     return a; } </pre> | $\longrightarrow$ | <pre> .globl num_vowels num_vowels:     #Prologo     pushl %ebx     pushl %edi     subl \$4(%esp)      movl 16(%esp), %ebx     xorl %edi, %edi L:     cmpb \$0, (%ebx)     je E     movsbl %ebx, %eax     movl %eax, (%esp)     call is_vowel     addl %eax, %edi     incl %ebx     jmp L E:     movl %edi, %eax     #Epilogo     addl \$4(%esp)     popl %edi     popl %ebx     ret </pre> |
| f.c                                                                                                                                                                                |                   | f_eq.c                                                                                                                                                                                                                                                                           |                   | f.s                                                                                                                                                                                                                                                                                                                                                                                         |

## Variabili Locali

Quando possibile, il compilatore utilizza i registri della CPU per memorizzare i valori delle variabili locali, poiché questa soluzione è la più semplice e garantisce le migliori prestazioni in termini di velocità di accesso. Tuttavia, non sempre ciò è possibile: in alcuni casi le variabili devono essere allocate nello *stack frame* della funzione, cioè in memoria. Questo accade nei seguenti casi:

1. **Registri insufficienti:** se il numero di variabili locali utilizzate dalla funzione supera quello dei registri disponibili, il compilatore è costretto a memorizzarne alcune nello stack.
2. **Uso dell'operatore &:** quando si applica l'operatore di indirizzo a una variabile locale, questa deve necessariamente risiedere in memoria, poiché deve avere un indirizzo ben definito.
3. **Variabili complesse:** se la variabile è un array o una struttura (**struct**), non può essere contenuta interamente in un registro e quindi viene allocata nello stack.

In sintesi, l'uso dei registri rappresenta un'ottimizzazione importante, ma è vincolato sia da limitazioni hardware (numero di registri) sia da requisiti semantici del programma (necessità di un indirizzo o dimensioni dei dati).

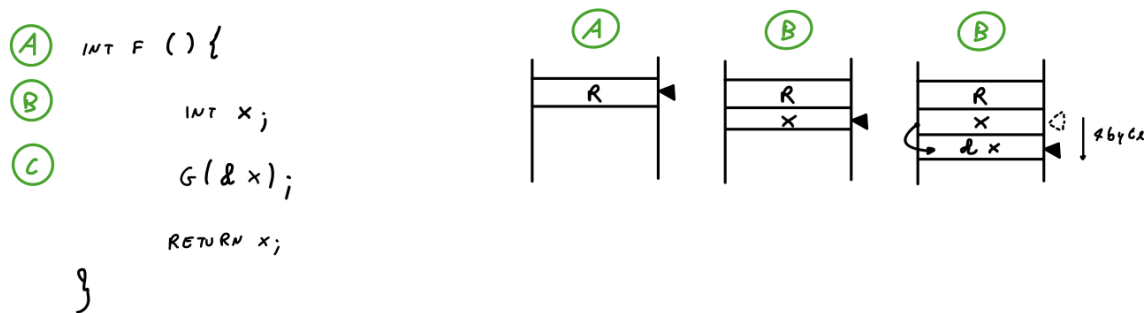


Figura 3.15: Caption

Questo in assembly:

```

.global f
f:
    #Prologo
    subl $8, %esp
    movl %esp, (%eax)    #lo salviamo in un registro
    addl $4, %eax        #poichè 'addl $4, (%esp)' non è corretto
    #quindi per modificare il valore dovremmo prima portarlo in un altro registro
    movl %eax, (%esp)
    call g
    movl 4(%esp), %eax
    #Epilogo
    addl $8, %esp
    ret

```

Nel dettaglio, la seguente sequenza di istruzioni:

```

movl %esp, (%eax)
addl $4, %eax

```

può essere riscritta in modo più compatto utilizzando l'istruzione `leal`:

```
leal 4(%esp), %eax
```

L'istruzione **LEA** (Load Effective Address) consente di calcolare un indirizzo di memoria senza accedere effettivamente al contenuto di quella locazione. Si limita quindi a eseguire un calcolo aritmetico e a salvare il risultato in un registro. La sua forma generale è:

**LEA**  $O(B, I, S)$ ,  $D$

dove l'indirizzo calcolato è:

$$B + I \cdot S + O$$

In altre parole, il registro di destinazione viene aggiornato come segue:

- inizialmente assume il valore base  $B$
- viene aggiunto l'offset  $O$
- viene aggiunto il contributo dell'indice  $I \cdot S$

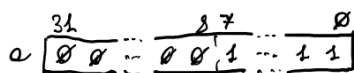
Per chiarire la differenza rispetto a mov, consideriamo il seguente confronto:

|                                                 |                                                                                          |
|-------------------------------------------------|------------------------------------------------------------------------------------------|
| leal 4(%esp), %eax                              | movl 4(%esp), %eax                                                                       |
| calcola ESP + 4 e<br>scrive il risultato in EAX | → calcola ESP + 4, accede alla memoria<br>in quell'indirizzo e copia il valore<br>in EAX |

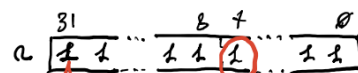
Quindi, mentre LEA esegue solo un calcolo di indirizzi, MOV comporta anche un accesso alla memoria.

**Importante:** Se usiamo LEA per calcoli aritmetici lo status register (EFLAGS) NON viene alterato. Quindi non è da usare per le jump condizionate.

|                                                 |                                       |
|-------------------------------------------------|---------------------------------------|
| void get_div_rem(int *d, int *r, int x, int y); | .globl f                              |
|                                                 | f:                                    |
|                                                 | #Prologo                              |
|                                                 | subl \$24, %esp                       |
|                                                 | movl 28(%esp), %eax #x                |
|                                                 | movl 32(%esp), %ecx #y                |
|                                                 | leal 16(%esp), %edx #&d               |
|                                                 | movl %edx, (%esp)                     |
|                                                 | #stessa cosa per il secondo parametro |
|                                                 | leal 20(%esp), %edx #&r               |
|                                                 | movl %edx, 4(%esp)                    |
|                                                 | movl %eax, 8(%esp)                    |
|                                                 | movl %ecx, 12(%esp)                   |
|                                                 | call get_div_rem                      |
|                                                 | movl 16(%esp), %eax                   |
|                                                 | #Epilogo                              |
|                                                 | addl \$24, %esp                       |
|                                                 | ret                                   |
|                                                 | f.s                                   |
| inf f(int x, int y){                            |                                       |
| int d, r;                                       |                                       |
| get_div_rem(&d, &r, x, y);                      |                                       |
| return d;                                       |                                       |
| }                                               |                                       |
| f.c                                             |                                       |



0x000000FF



0xFFFFFFFF

In ASM eseguiamo l'insert con due istruzioni diverse:

movsx y S, D

copia gli x bit meno significativi  
di S in una destinazione D a  
y bit con estensione del segno

movzx y S, D

copia gli x bit meno significativi  
di S in una destinazione D a  
y bit senza estensione del segno

Per entrambe: S non può essere un valore immediato  
D deve essere un registro

Figura 3.16: Graficamente

## Espressioni Booleane

L'istruzione `SETcc` (dove `cc` sta per *condition code*) consente di assegnare il valore 0 oppure 1 a un registro a 8 bit oppure a un byte in memoria, in base al risultato di una condizione valutata sui flag contenuti nel registro `EFLAGS`. La forma generale dell'istruzione è:

`SETcc D`

Il suo comportamento può essere descritto come:

$$D \leftarrow \text{condizione}$$

In particolare:

- se la condizione associata al suffisso `cc` è verificata, viene scritto il valore 1 in `D1`;
- altrimenti, viene scritto il valore 0.

Questa istruzione è spesso utilizzata per trasformare il risultato di un confronto in un valore booleano (0 o 1), utile ad esempio nelle espressioni condizionali.

|                                                             |                   |                                                                                                                            |                   |                                                                                                                                                                                          |
|-------------------------------------------------------------|-------------------|----------------------------------------------------------------------------------------------------------------------------|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>int f(int x,int y){<br/>    return x&lt;y;<br/>}</pre> | $\longrightarrow$ | <pre>int f(int x,int y){<br/>    int a = 1;<br/>    if(x&lt;y) goto E;<br/>    a = 0;<br/>E:<br/>    return a;<br/>}</pre> | $\longrightarrow$ | <pre>.globl f<br/>f:<br/>    movl 4(%esp), %eax #x<br/>    movl 8(%esp), %edx #y<br/>    cmpl %edx, %ecx<br/>    setl %al #restituisce un int<br/>    movzbl %al, %eax<br/>    ret</pre> |
|-------------------------------------------------------------|-------------------|----------------------------------------------------------------------------------------------------------------------------|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Altro esempio:

|                                                                                                    |                   |                                                                                                                                                                                                                                                                                                                                                                  |
|----------------------------------------------------------------------------------------------------|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>int between(short x,short y, short z){<br/>    return x&lt;=y &amp;&amp; y&lt;=z;<br/>}</pre> | $\longrightarrow$ | <pre>.globl between<br/>between:<br/>    #Prologo<br/>    pushl %ebx<br/><br/>    movw 8(%esp), %bx #x<br/>    movw 12(%esp), %cx #y<br/>    movw 16(%esp), %dx #z<br/><br/>    cmpw %cx, %bx<br/>    setle %al<br/>    cmpw %dx, %cx<br/>    stele %ah<br/><br/>    andb %ah, %al<br/>    movzbl %al, %eax<br/>    #Epilogo<br/>    popl %ebx<br/>    ret</pre> |
| <pre>f.c</pre>                                                                                     |                   | <pre>f.s</pre>                                                                                                                                                                                                                                                                                                                                                   |

### Regola del cortocircuito: differenza tra `&` e `&&` in C

In C esistono due operatori che possono sembrare simili ma hanno comportamenti diversi:

- `&&` (AND logico): applica la **valutazione con cortocircuito**. La seconda espressione viene valutata **solo se** la prima è vera (diversa da zero). Se la prima è falsa, il risultato è già determinato e la seconda non viene eseguita.
- `&` (AND bit a bit): **non utilizza il cortocircuito**. Entrambe le espressioni vengono sempre valutate, indipendentemente dal valore della prima.

In sintesi, il cortocircuito è una proprietà dell'operatore logico `&&`, mentre l'operatore `&` esegue sempre entrambe le valutazioni.

## 3.8 Allineamento nello Stack

Consideriamo il seguente esempio:

|                                              |                   |                                                                 |
|----------------------------------------------|-------------------|-----------------------------------------------------------------|
| <code>void g(char *c);</code>                |                   | <code>f:</code>                                                 |
|                                              |                   | <code>subl \$4, %esp # spazio per a (allineato a 4 byte)</code> |
| <code>void f(){</code>                       |                   | <code>subl \$4, %esp # spazio per il parametro di g</code>      |
| <code>...</code>                             | $\longrightarrow$ | <code>...</code>                                                |
| <code>char a; // 1 byte</code>               |                   | <code>leal ...</code>                                           |
| <code>g(&amp;a); // parametro: 4 byte</code> |                   | <code>movl ..., (%esp)</code>                                   |
| <code>...</code>                             |                   | <code>call g</code>                                             |
| <code>}</code>                               |                   | <code>...</code>                                                |

**Importante:** In ogni momento dell'esecuzione, lo **stack pointer** deve essere allineato a un multiplo di 4 byte. In pratica, i compilatori moderni allineano lo stack a **16 byte**.

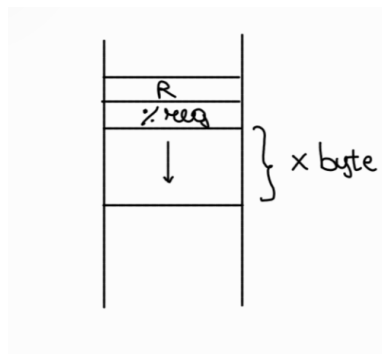


Figura 3.17: SP

Esempio con più variabili:

|                                         |                                                                          |
|-----------------------------------------|--------------------------------------------------------------------------|
|                                         | <code># Prologo</code>                                                   |
|                                         | <code>subl \$16, %esp # spazio totale</code>                             |
| <code>char a, b, c;</code>              |                                                                          |
| <code>g(&amp;a, &amp;b, &amp;c);</code> | $\longrightarrow$ <code># Spiegazione:</code>                            |
|                                         | <code># - a, b, c occupano 1 byte ciascuno</code>                        |
|                                         | <code># - per allineamento diventano 4 byte ciascuno</code>              |
|                                         | <code># - parametri di g: 3 * 4 byte = 12 byte</code>                    |
|                                         | <code># - totale: 3*4 + 12 = 24 → arrotondato a 16 (allineamento)</code> |

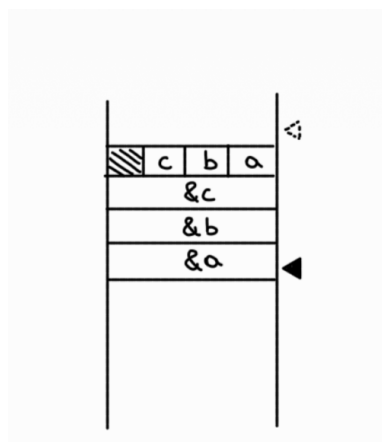


Figura 3.18: Ecco cosa succede nello SP

**Regola fondamentale:** Un dato di dimensione  $x$  viene allocato a indirizzi multipli di  $x$ .

Altro esempio:

|                                                        |   |                                                                                                                                                                                                                            |
|--------------------------------------------------------|---|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>char a[5]; short b; g(&amp;a, &amp;b);  f.c</pre> | → | <pre># Prologo subl \$16, %esp  # Spiegazione: # - a occupa 5 byte → allineato a 8 byte # - b occupa 2 byte → allineato a 4 byte # - parametri: 2 * 4 = 8 byte # - totale: 8 + 4 + 8 = 20 → arrotondato a 16/32  f.s</pre> |
|--------------------------------------------------------|---|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 3.9 Allineamento delle Struct

Per tradurre una **struct** in Assembly, bisogna rispondere a:

1. Dove inizia ogni campo in memoria?
2. Quanto spazio occupa complessivamente la struct?

**Regole di allineamento:**

1. Un dato di dimensione  $x$  deve essere allineato a indirizzi multipli di  $x$
2. L'allineamento della struct è determinato dal campo più grande
3. La dimensione totale della struct è un multiplo della dimensione del campo più grande

**Esempio (non ottimizzato)**

```
struct s{
    char x;    // +0 |x - - -|
    int y;     // +4 |x x x x|
    short z;   // +8 |x x - -|
                // sizeof(s) = 12 (5 byte sprecati)
};
```

**Esempio (ottimizzato):**

```
struct s{
    char x;    // +0 |x -
    short z;   // +2 |    x x|
    int y;     // +4 |x x x x|
                // sizeof(s) = 8 (1 byte sprecato)
};
```

**Nota:** Riordinare i campi riduce lo spazio sprecato e migliora l'efficienza.

**Esempio completo con Assembly:**

|                                                                                                                                      |   |                                                                                                                                                                                                  |
|--------------------------------------------------------------------------------------------------------------------------------------|---|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>struct s{     char x; // +0     int y;  // +4 };  void f(struct s *p, char a, int b){     p-&gt;x = a;     p-&gt;y = b; }</pre> | → | <pre>.globl f f:     movl 4(%esp), %eax    # p     movb 8(%esp), %cl     # a     movl 12(%esp), %edx   # b      movb %cl, (%eax)     # p-&gt;x     movl %edx, 4(%eax)   # p-&gt;y      ret</pre> |
|--------------------------------------------------------------------------------------------------------------------------------------|---|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 3.10 Istruzioni SHIFT

Le istruzioni di shift (spostamento) in Assembly consentono di spostare i bit di un operando a sinistra o a destra. Esistono due tipi principali di shift:

- Shift con segno

SAL S,D | D <- D << S    I nuovi bit meno significativi sono posti uguali a 0  
 SAR S,D | D <- D >> S    I nuovi bit più significativi sono posti uguali al  
 valore del bit più significativo

- Shift senza segno

SHL S,D | D <- D << S    I nuovi bit meno significativi sono posti pari a 0  
 SHR S,D | D <- D >> S    I nuovi bit meno significativi sono posti pari a 0

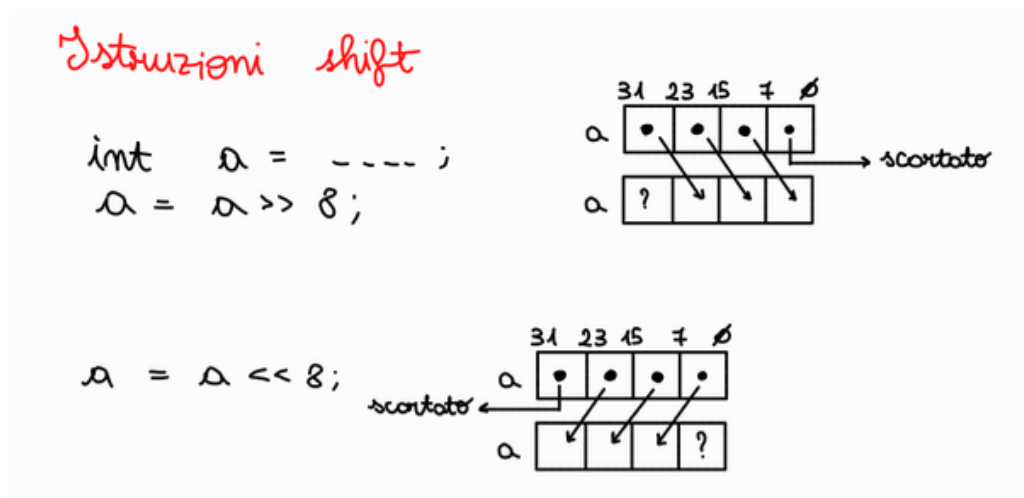


Figura 3.19: Shift dei bit

Esempio:

```
int c = 0xABADCAFE;
unsigned d = 0xABADCAFE;
c = c << 8;      // assembly: sall $8,%ecx      %ecx -> 0xADCAFE00
d = d << 8;      // assembly: shll $8,%ecx      %ecx -> 0xADCAFE00
c = c >> 8;      // assembly: sarl $8,%ecx      %ecx -> 0xFFADCAFE
d = d >> 8;      // assembly: shrll $8,%ecx     %ecx -> 0x00ADCAFE
```

Esempio:

```
int a = 8;      000...001000 -> 8
a = a >> 2;     000...000010 -> 2 = 8/(2^2) esponente di 2 è il numero di bit shift
```

$a \gg x \rightarrow a/2^x$

```
int a = 8;      000...001000 -> 8
a = a << 2;     000...100000 -> 32 = 8*(2^2) esponente di 2 è il numero di bit shift
```

$a \ll x \rightarrow a \cdot 2^x$

Vediamo ora attraverso un esercizio:

|                                                                                                                                                                     |                                                                                                                                                                                                                                      |                                                                                                                                                                                                           |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> unsigned count1(unsigned n){     unsigned a=0;     while(n&gt;0){         if(n &amp; 1)a++;         n = n &gt;&gt; 1;     }     return a; } </pre> <p>f.c</p> | <pre> unsigned count1(unsigned n){     unsigned eax=0;     unsigned ecx=n; L:     if(c &lt;= 0) goto E;     if ((n &amp; 1)==0) goto A;     eax++; A:     ecx = ecx &gt;&gt; 1;     goto L; E:     return eax } </pre> <p>f_eq.c</p> | <pre> .globl count1 count1:     xorl %eax,%eax     movl 4(%esp),%ecx L:     cmpl \$0,%ecx     jbe E     testl \$1,%ecx     jz A     incl %eax A:     shrl \$1,%ecx     jmp L E:     ret </pre> <p>f.s</p> |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### 3.11 Istruzione IDIV

L'istruzione IDIV esegue la divisione intera con segno tra il valore contenuto nel registro EAX (dividendo) e un operando specificato (divisore). Il risultato della divisione viene memorizzato in due registri:

- Il quoziente (risultato della divisione) viene memorizzato in EAX.
- Il resto (modulo) viene memorizzato in EDX.

```

idiv s | A <- D : A / S quoziente
       | D <- D : A % S resto

```

Esempio:

|                                                                |                                                                                                                                                  |                                                                                                                                              |                                                               |
|----------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| <pre> int f(int x, int y){     return x/y; } </pre> <p>f.c</p> | <pre> int f(int x, int y){     int a = x;     int c = y;     int d = a;     d=d &gt;&gt; 31;     a = a / c;     return a; } </pre> <p>f_eq.c</p> | <pre> .globl f f:     movl 4(%esp),%eax     movl 8(%esp),%ecx     movl %eax,%edx     sarl \$31,%edx     idivl %ecx     ret </pre> <p>f.s</p> | <p>questa coppia di<br/>istruzioni prepara<br/>per i idiv</p> |
|----------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|

Quando usiamo idiv, la divisione non avviene solo su EAX, ma su EDX:EAX (un numero a 64 bit). Quindi dobbiamo prima preparare EDX correttamente: deve contenere l'estensione del segno di EAX. Per fare ciò utilizziamo lo shift aritmetico a destra (SAR) di 31 bit, che riempie EDX con il bit più significativo di EAX (0 se EAX è positivo, 1 se è negativo). In questo modo, quando eseguiamo IDIV, otteniamo il risultato corretto anche per numeri negativi. Quindi:

```

movl %eax,%edx
sarl $31,%edx

```

serve proprio a questo: copia il bit di segno di EAX in tutto EDX

- EDX = 0 se il numero è positivo
- EDX = -1 se è negativo

### 3.12 Istruzione CMOV

L'istruzione CMOVcc (Conditional Move) consente di eseguire un'operazione di assegnamento condizionale, spostando il valore da una sorgente a una destinazione solo se una specifica condizione è soddisfatta. La forma generale dell'istruzione è:

```
CMOVcc S,D | D <- S se cc è vero
            | D <- D se cc è falso
```

S non può essere un valore immediato  
D deve essere un registro a 16/32 bit

Esempio

|                                                        |                                                                                                                                         |     |
|--------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|-----|
|                                                        | <pre>int max(int x, int y){     int eax = x;     int ecx = y;     if(ecx &gt; eax)         goto E;     eax=ecx;     return eax; }</pre> |     |
| int max(int x, int y){<br>return (x > y) ? x : y;<br>} | → E:                                                                                                                                    | →   |
| f.c                                                    | f_eq.c                                                                                                                                  | f.s |

### 3.13 Ottimizzazione dei programmi

L'ottimizzazione di un programma consiste nel modificarlo in modo da minimizzare l'uso delle risorse disponibili, cercando allo stesso tempo di massimizzare le prestazioni, senza compromettere la correttezza.

**Problemi principali:**

1. Come valutare l'impatto di una modifica sul programma?  
In altre parole: come misurare le prestazioni e l'uso delle risorse?
2. Come individuare le parti del programma da ottimizzare?  
Come identificare i *punti caldi* (hot spots) che meritano attenzione?

**Misure prestazionali:**

- **Latenza:** tempo necessario per eseguire il programma
- **Throughput:** numero di operazioni completate per unità di tempo
- **Spazio:** memoria utilizzata
- **Energia:** consumo energetico

#### Analisi prestazionale del programma

Ad esempio, quanto tempo richiede l'esecuzione dell'istruzione:

```
movl $10, (%eax) ?
```

Possiamo scomporre l'operazione in:

1. Lettura del registro **eax** → CPU + registri (operazione veloce)
2. Scrittura del valore 10 in memoria → RAM (operazione più costosa)

Il miglioramento prestazionale si misura tramite lo **speedup**:

$$\text{speedup} = \frac{T}{T'}$$

dove  $T$  è il tempo di esecuzione originale e  $T'$  quello ottimizzato.

## Legge di Amdahl

La legge di Amdahl permette di calcolare lo speedup complessivo di un programma quando si ottimizza solo una sua parte.

Sia:

$$T = \alpha T + (1 - \alpha)T$$

dove:

- $\alpha$  è la frazione di tempo spesa nella parte ottimizzabile
- $(1 - \alpha)$  è la parte non ottimizzabile

Supponiamo di ottenere uno speedup  $k_A$  sulla parte ottimizzabile. Allora:

$$T' = \frac{\alpha T}{k_A} + (1 - \alpha)T$$

Lo speedup complessivo diventa:

$$S = \frac{T}{T'} = \frac{1}{\frac{\alpha}{k_A} + (1 - \alpha)}$$

**Procedimento:**

$$S = \frac{T}{T'} \quad T' = T'_A + T'_B = \frac{T_A}{k_A} + T_B = \frac{\alpha T}{k_A} + (1 - \alpha)T = (\frac{\alpha}{k_A} + 1 - \alpha)T$$

$$S = \frac{T}{T'} = \frac{T}{(\frac{\alpha}{k_A} + 1 - \alpha)T} = \frac{1}{\frac{\alpha}{k_A} + 1 - \alpha}$$

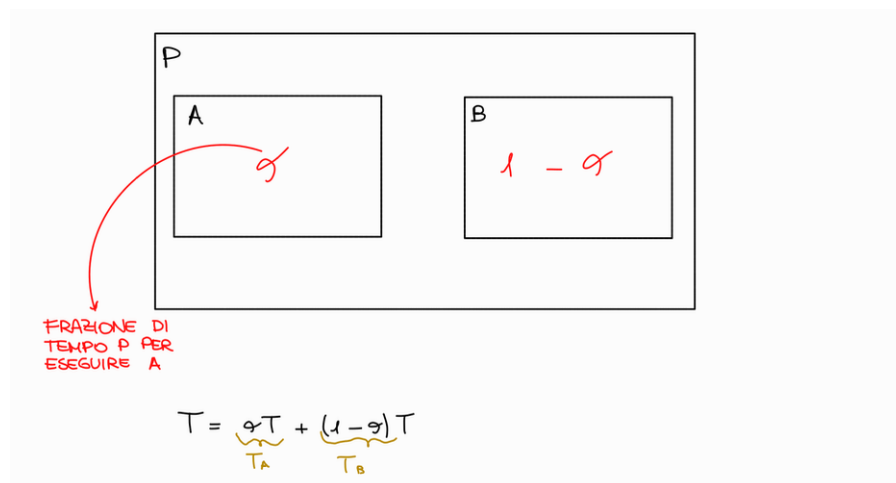


Figura 3.20: Legge di Amdahl

### Esempi

- $\alpha = 0.5, k_A = 2 \Rightarrow S \approx 1.33$
- $\alpha = 0.9, k_A = 6 \Rightarrow S \approx 4$

**Limite teorico** Qual è lo speedup massimo ottenibile?

$$\lim_{k_A \rightarrow \infty} \frac{1}{\frac{\alpha}{k_A} + (1 - \alpha)} = \frac{1}{1 - \alpha}$$

Questo mostra che lo speedup è limitato dalla parte non ottimizzabile del programma.

## 3.14 Gestione dinamica della memoria in Assembly con malloc

In assembly, la gestione della memoria dinamica non è automatica come in C: il programmatore deve gestire esplicitamente la chiamata alla funzione `malloc`, il controllo degli errori e l'utilizzo del puntatore restituito.

### Ruolo di malloc

La funzione `malloc` è una funzione di libreria che:

- Alloca un blocco di memoria nello heap
- Restituisce l'indirizzo del blocco allocato
- Restituisce NULL (0) in caso di errore

In assembly, non esiste alcun meccanismo automatico: `malloc` viene trattata come una normale funzione esterna.

## Gestione interna in assembly

Quando si usa `malloc` in assembly, bisogna gestire manualmente:

1. **Preparazione del parametro:** → la dimensione da allocare (in byte) deve essere calcolata dal programmatore. Ad esempio, per una struct:

```
sizeof(node_t) = 2 (short) + 4 (pointer) + 2 (padding)
               = 8 byte
```

Il padding sarebbe memoria vuota per garantire l'allineamento corretto dei campi.

2. **Preparazione dello Stack:** → viene riservato lo spazio nello stack per l'allocazione della grandezza (8 = 4 byte): (riprendendo l'esempio di sopra)

```
subl $4, %esp
```

3. **Chiamata della funzione**

```
call malloc
```

4. **Recupero del risultato** → il puntatore restituito è contenuto nel registro **EAX**:

```
eax = puntatore al blocco allocato
```

5. **Controllo dell'errore** → è necessario verificare se il puntatore è nullo:

```
cmpl $0, %eax
je errore
```

6. **Uso della memoria allocata** → il puntatore in **EAX** viene usato per accedere e inizializzare la struttura:

```
movl %val, (%eax)      #primo campo
movl %val, offset(%eax) #altri campi
```

7. **Pulizia dello stack:** → dopo la chiamata: (ripristiamo lo stack al suo stato originale)

```
addl $4, %esp
```

Abbiamo aggiunto 4 byte allo stack per rimuovere il parametro passato a `malloc` essendo quest'ultimo un parametro a 4 byte (l'intero 8).

### Osservazioni importanti

- In assembly non esiste il concetto di tipo: è il programmatore che gestisce gli offset.
- L'allineamento della memoria è importante (tipicamente a multipli di 4 byte).
- Il puntatore restituito deve essere salvato se serve dopo altre operazioni.

Ricordiamo che l'offset è una distanza in byte tra l'inizio della struttura e il campo specifico.

## Esempio:

```
int list_add_first(node_t **l, short elem) {
    node_t *p = *l;
    node_t *n = malloc(sizeof(node_t));
    if (n == NULL) return -1;
    n->elem = elem;
    n->next = p;
    *l = n;
    return 0;
}
```

f.c

```
int list_add_first(node_t **l, short elem){
    node_t **si = l;
    node_t *b = *si;
    node_t *a = malloc(sizeof(node_t));
    if (a != NULL) goto F;
    return -1;          // allocation error
F:
    short d = elem;
    (*a).elem = d;
    (*a).next = b;
    *si = a;
    return 0;
}
```

f\_eq.c

```
.globl list_add_first
```

```
# typedef struct node_t {    // base
#     short elem;           // offset: 0 |xx..| (base)
#     struct node_t *next;   // offset: 4 |xxxx| 4(base)
# } node_t;                 // sizeof: 8
```

```
list_add_first:
```

```
    pushl %ebx
    pushl %esi
    subl $4, %esp
```

```
    movl 16(%esp), %esi      # node_t **si = l;
    movl (%esi), %ebx        # node_t *b = *si;
    movl $8, (%esp)
    call malloc              # *a = malloc(sizeof(node_t));
    testl %eax, %eax         # if (a != NULL)
    jnz F                   # goto F;
    movl $1, %eax            # return -1;          // allocation error
    jmp E
```

```
F:
```

```
    movl 20(%esp), %edx      # short d = elem;
    movw %dx, (%eax)         # (*a).elem = d;
    movl %ebx, 4(%eax)       # (*a).next = b;
    movl %eax, (%esi)        # *si = a;
    movl $0, %eax
```

```
E:
```

```
    addl $4, %esp
    popl %esi
    popl %ebx
    ret                      # return 0;
```

f.s

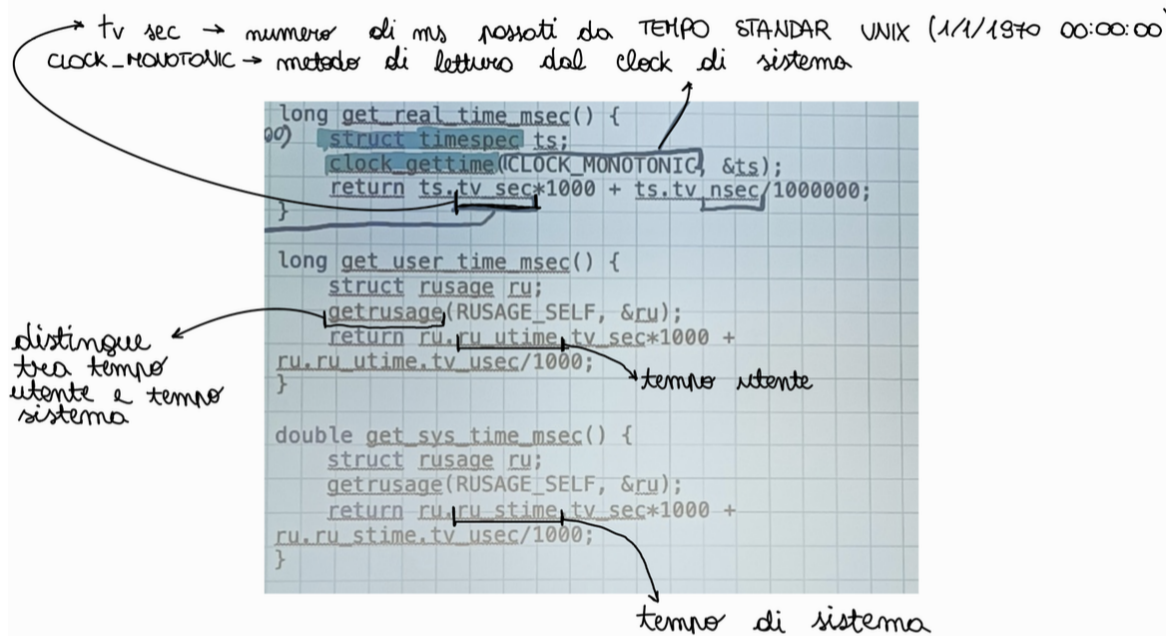
### 3.15 Profilazione delle prestazioni

L'obiettivo della profilazione è calcolare la latenza di esecuzione di ogni parte di un programma. Si divide in:

- **Manuale** → tramite strumentazione manuale del codice
- **Automatica** → l'strumentazione viene fatta da un tool

#### Profilazione Manuale

```
main(){  
    .  
    .  
    tempo -> x  
    -----  
    | target |  
    -----  
    tempo -> x'  
    calcolo x - x' // e sarà il tempo di esecuzione finale  
    .  
    .  
}
```



Quale "tempo" stiamo misurando?

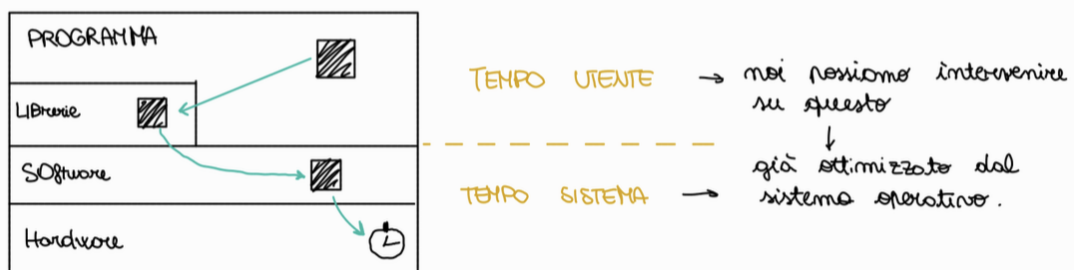


Figura 3.21: Profilazione manuale

Questo tipo di profilazione ha dei limiti:

1. **Latenza dello strumento** → tempo necessario per eseguire la misurazione
2. **Risoluzione dello strumento** → nel nostro caso risulta essere il minimo intervallo temporale misurabile

**Importante:** la misurazione di un tempo  $T$  con uno strumento  $A$  è significativa solo se la risoluzione di  $A$  è maggiore di  $T$ .

**Importante:** la latenza dello strumento utilizzato per la misura deve influire sui risultati in modo *trascurabile*.

## Profilazione Automatica

Per la profilazione automatica ci aiuteremo con il tool **"gprof"**, vediamo come funziona:

1. Bisogna innanzitutto compilare il codice target con **"-pg"**
2. Eseguo il programma compilato
3. Visualizzo i risultati con **"gprof <nome programma>"**

Vediamo un esempio dove c'è un codice che chiama 3 sottoprogrammi: main, A e B.

```
sc@scserver:~/shared/training/classes/5.1-profiling/E4-gprof$ gcc -m32 -pg -o e4 mioprogram.c
sc@scserver:~/shared/training/classes/5.1-profiling/E4-gprof$ ./e4
0
sc@scserver:~/shared/training/classes/5.1-profiling/E4-gprof$ l
total 60K
drwx----- 7 sc sc 224 Apr 14 09:12 .
drwx----- 6 sc sc 192 Feb 20 2024 ..
-rw----- 1 sc sc 18K Apr 17 2024 A_noopt.txt
-rwxrwxr-x 1 sc sc 16K Apr 14 09:12 e4
-rwxrwxr-x 1 sc sc 16K Apr 9 2025 e4unopt
-rw-rw-r- 1 sc sc 2.7K Apr 14 09:12 gmon.out
-rw----- 1 sc sc 470 Feb 20 2024 mioprogram.c
sc@scserver:~/shared/training/classes/5.1-profiling/E4-gprof$
```

Figura 3.22: Esempio del risultato della compilazione

**gmon.out** è il nostro profilo prestazionale dove all'interno troveremo appunto il tempo impiegato dal nostro programma.

```
sc@scserver:~/shared/training/classes/5.1-profiling/E4-gprof$ gprof e4
Flat profile:

Each sample counts as 0.01 seconds.
 %   cumulative   self           calls   self   total    name
time  seconds    seconds             us/call  us/call  us/call  name
50.44      6.81      6.81          100000      68.10    68.10    A
49.56     13.50      6.69          100000      66.90    66.90    B
 0.00      13.50      0.00              1         0.00     0.00    init
```

Figura 3.23: gmon.out

C'è inoltre un altro tipo di lista ovvero il **Call graph**, ovvero un grafo che rappresenta le chiamate:

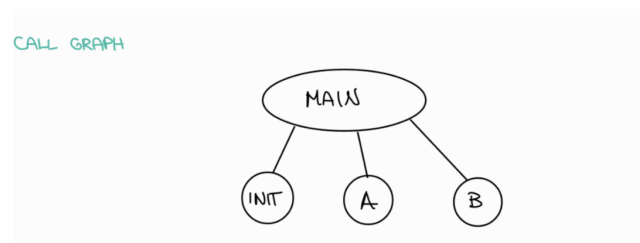


Figura 3.24: Call Graph

## Assetti chiave della ottimizzazione

L'obiettivo è ridurre l'uso delle risorse massimizzando le prestazioni (nel nostro caso la latenza) senza impattare la correttezza del programma. In linea generale possiamo intervenire su due livelli:

1. **Algoritmo** → scegliere l'algoritmo con costo computazionale minore che risolve il problema. Tipicamente questa scelta è affidata al progettista, ma in alcuni casi può impattare anche il programmatore. Anche questo ha i suoi pregi e difetti:
  - + La scelta dell'algoritmo ha un impatto molto significativo sulle prestazioni.
  - - Non è sempre possibile
  - - Richiede una implementazione ad-hoc
2. **Implementazione** → ottimizzare il codice che implementa l'algoritmo scelto. Questa parte è affidata tipicamente al programmatore o il compilatore. Anche qui ci sono vantaggi e svantaggi:
  - + Soluzione (parzialmente) automatica
  - + Soluzione "economica" da adottare
  - - Alcune implementazioni possono essere "platform-dependent" (potrebbero funzionare solo su alcune piattaforme)

## Tecniche di ottimizzazione del codice

Per l'ottimizzazione del codice abbiamo due approcci principali:

1. **Riduzione del "work"** → ridurre il numero di istruzioni da eseguire.
2. **Riduzione del "costo"** → scelgo tra più alternative quella che impegna la CPU per meno tempo possibile.

Il compilatore applica questi approcci usando numerose tecniche di ottimizzazione attraverso queste opzioni:

- "-O0" → default con cui non applica *NESSUNA* ottimizzazione
- "-O1" → ottimizza
- "-O2" → ottimizza di più
- "-O3" → ottimizza ancora di più
- "-Og" → ottimizza il codice cercando di preservare i simboli di debug
- "-Os" → ottimizza riducendo lo spazio

Abbiamo diverse tecniche di ottimizzazione:

- **Constant Folding** → precalcola i valori costanti presenti nel codice e viene applicata anche ad "-O0":

```
int f(){
    return 8+(14/2)* 3; ---> return 29;
}
```

- **Constant Propagation** → i valori costanti vengono propagati nel codice dove possibile:

```
int f(){
    int x = 8;           (passo 1)      (passo 2)
    return x - 2; ---> return 8-2; ---> return 6;
}
```

- **Common SubExpression Elimination** → sostituisce occorrenze multiple della stessa sottoespressione (un'espressione all'interno del codice che esegue un certo numero di attività):

```
int f(int x, int y){          int z = x+y;    //aggiunge una variabile per semplificare
    return (x+y) * (x+y); ---> return z * z;
}
```

- **Loop Invariant Code Motion** → sposta fuori dal ciclo le espressioni che non dipendono dal ciclo stesso:

```

int f(int x, int n){
    int s = 1;
    for( ; n > 0; n--)
        s += 7+x;
        // ^ sottoespressione che non →
        // | dipende dal ciclo
    return s;
}

int f(int x, int n){
    int s = 1;
    int z = 7+x;
    for( ; n > 0; n--)
        s += z;
    return s;
}

```

**ATTENZIONE:** non è sempre applicabile. Vediamo un esempio:

```

int g(int k){
    printf("Pippo");
    return k+1;
}

int f(int x, int n){
    int s = 1;
    for( ; n > 0; n--)
        s += g(x); // espressioni come questa o l'uso di variabili globali poichè potrebbero
    return s;      // essere modificate durante il programma a nostra insaputa
}

```

- **Loop Unrolling** → dato un ciclo con un numero di iterazioni noto a tempo di compilazione, riduce o elimina le iterazioni.

```

int f(){
    int a = 0;
    for(int i = 0; i < 10; i++)
        a += 1;
    return a;
}

int f(){
    int a = 0;
    .
    . // ripetuto 10 volte
    .
    a++;
    return a;
    // ottimizzato con i vari metodi
    // avremo semplicemente: return 10;
}

```

Come facciamo se il numero di iterazioni è noto solo in esecuzione?

```

int sum(int *v, int n){
    int i,s = 0;
    for(i = 0; i<n; i++)
        s += v[i];
    // n però è noto solo
    // a tempo di esecuzione
    return s;
}

int sum(int *v, int n){
    int i,s = 0;
    // 3 è il valore k, 2 è k-1
    for(i = 0; i + 2 < n; i+=3){
        s += v[i];
        s += v[i+1]; // eseguo k operazioni
        s += v[i+2]; // per iterazione
    }
}

```

Esegue blocchi di dimensione 1 per massimo k - 1 volte

Questa esegue blocchi di istruzione di dimensione K per n/k volte (parte intera inferiore della divisione).

K è un valore arbitrario scelto da noi. Se *n* è *multiplo di k* non avremo problemi, se non è multiplo rischiamo di non eseguire alcune operazioni.

- **Register Allocation** → massimizza l'uso dei registri.

**ATTENZIONE:** è una ottimizzazione platform dependent

- **Operato Strenght Reduction** → sostituisce una istruzione con un'altra equivalente ma meno costosa.

```

x / 8  ---> x>>3 (shift a destra di 3 bit)
x * 64 ---> x<<6
x * 2  ---> x + x

```

$x * 15 \rightarrow (x \ll 4) - x$

```

void f(int *v, int n, int k){
    int i;
    for(i = 0; i < n; i++){
        v[i] = i * k;
    }
}

void f(int *v, int n, int k){
    int i, c = 0;
    for(i = 0; i < n; i++){
        v[i] = c;
        c += k;
    }
}

```

- **Dead Code Elimination** → elimina il codice non raggiungibile.
- **Riduzione Spazio di Allocazione** → riordinamento dei campi di una struct (dal più grande al più piccolo) per minimizzare il padding (spreco di memoria). Questa va applicata dal programmatore.
- **Allineamento in Memoria** → ogni dato di dimensione S byte viene salvato in memoria ad *indirizzo X* multiplo di S.

Le ottimizzazioni possono essere sia sul codice C (sorgente) che sul codice Assembly, questo varia a seconda del tipo di compilatore che stiamo utilizzando. In generale, le ottimizzazioni a livello di codice sorgente sono più portabili, mentre quelle a livello di Assembly possono essere più specifiche per una piattaforma.

### 3.16 Come vengono eseguiti i programmi

Quando un programma viene eseguito, diventa un **processo**. In altre parole, un processo è semplicemente un programma in esecuzione. La gestione dei processi è uno dei compiti principali di un sistema operativo. Uno stesso programma può essere avviato più volte, generando così più processi che possono esistere contemporaneamente nel sistema. Ogni processo è identificato in modo univoco da un numero chiamato **PID** (Process ID). Il PID è un valore progressivo che aumenta di uno ogni volta che viene creato un nuovo processo.

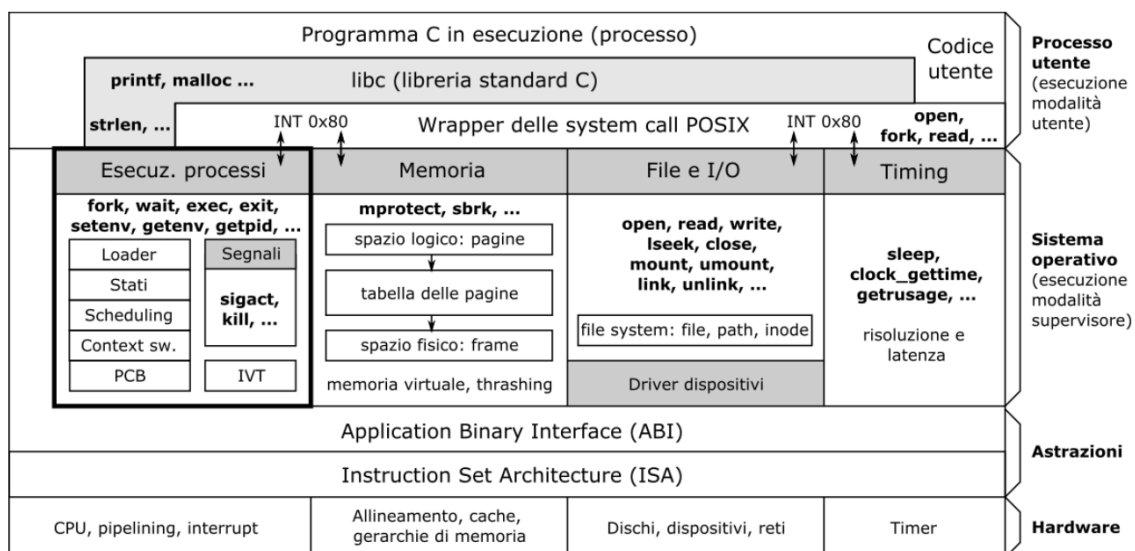


Figura 3.25: Panoramica di un sistema di calcolo Linux/x86

Per la documentazione di posix/libc possiamo utilizzare il comando **"man"**, oppure cercarlo sul web sul sito dell'"open group". "man" prevede diverse sezioni tra cui:

1. *Comandi del terminale (shell)*

Esempio: `man ls`

## 2. *System call*

Esempio: `man fork`

## 3. *Funzioni Libc*

Esempio: `man strlen`

Per accedere a una sezione specifica:

Esempio: `man 3 printf`

## Terminazione dei programmi

Ogni programma termina restituendo un **codice di terminazione**, detto anche return code, rappresentato da un valore intero. Normalmente per convenzione, il return code è utilizzato per segnalare la corretta terminazione o meno del programma.

- $\emptyset \rightarrow$  "nessun errore"
- Qualsiasi altro valore  $\rightarrow$  rappresenta un errore (error code)

Il return code può essere generato in diversi luoghi:

- Se siamo nel `main()`
  - dopo il termine dell'ultima istruzione
  - "return"
- Dopo l'esecuzione di:
  - "exit"
  - "abort"

## Gestire la terminazione di un programma

È possibile configurare nel programma una o più di gestione della terminazione, che vengono eseguite quando il programma termina e prendono il nome di **EXIT Handler**.

```
#include <stdlib.h>
int atexit(void (*handler)(void));
```

L'Handler è un gestore di tipo void, quindi un puntatore a funzione che non prende parametri e non restituisce nulla. Esempio:

```
void f(){
    .
    .
    .
}
main(){
    .
    .
    atexit(f);
    .
    .
}
```

## Tipologie di Errore

1. **Errori non recuperabili** → causano la terminazione anomala del programma (es. divisione per zero, accesso a memoria non valida, ecc.)
2. **Errori recuperabili** → possono essere gestiti all'interno del programma (es. file non trovato, input non valido, ecc.). Questi errori pongono il programma in uno stato che potrebbe omeno richiedere la sua terminazione (la scelta è del programmatore).

Per restituire il codice di errore spesso le librerie scrivono il codice di errore nella variabile globale **ERRNO**. Ci sono presenti dei **Pattern di Gestione degli Errori**

Figura 3.26: Codice

## Capitolo 4

# Approfondimento C

### 4.1 Funzioni di Libreria

#### Assert

Cominciamo dicendo che `assert` non è una funzione di libreria, anche se viene utilizzata nello stesso modo, ma è un **macro**. Essa prende in ingresso un'espressione e, se questa risulta falsa, interrompe l'esecuzione del programma. Per utilizzarla:

```
#include <assert.h>
assert(exp); // espressione da valutare
```

L'`assert` serve tipicamente per il **debug** e per verificare condizioni interne del programma. In caso di errore, il programma viene terminato (chiamando `abort()`). Se è definito `NDEBUG`, gli `assert` vengono disabilitati e non hanno effetto.

#### Funzioni per la gestione di Stringhe

Dato che C non offre delle funzioni per la gestione delle stringhe è molto utile utilizzare delle funzioni esterne in base alle nostre esigenze, come:

```
#include <string.h>
size_t strlen(const char* s); // calcola la lunghezza della stringa
char* strcpy(char *dst, const char* src); // copia la stringa in un'altra stringa
char* strncpy(char *dst, const char* src, size_t len); // copia la stringa fino al len
// carattere da src in dst. Se src
// è più piccolo di dst i verranno riempiti
// con il carattere '/0'. Mentre in caso
// di dst è più piccolo di src i
// caratteri andranno persi.
char* strcat(char* dst, const char* src); // concatena le due stringhe
```

**ATTENZIONE:** `dst` deve contenere abbastanza spazio libero per contenere `src`

```
char* strtok(char* src, const char* delim); // divide la stringa in token
```

Utilizzo:

```
char* token = strtok(str, delim);
while(token != null){
    .... codice che fa qualcosa con token
    token = strtok(NULL, delim); // per andare al prossimo token
}
```

Vediamo però come funziona ogni parametro:

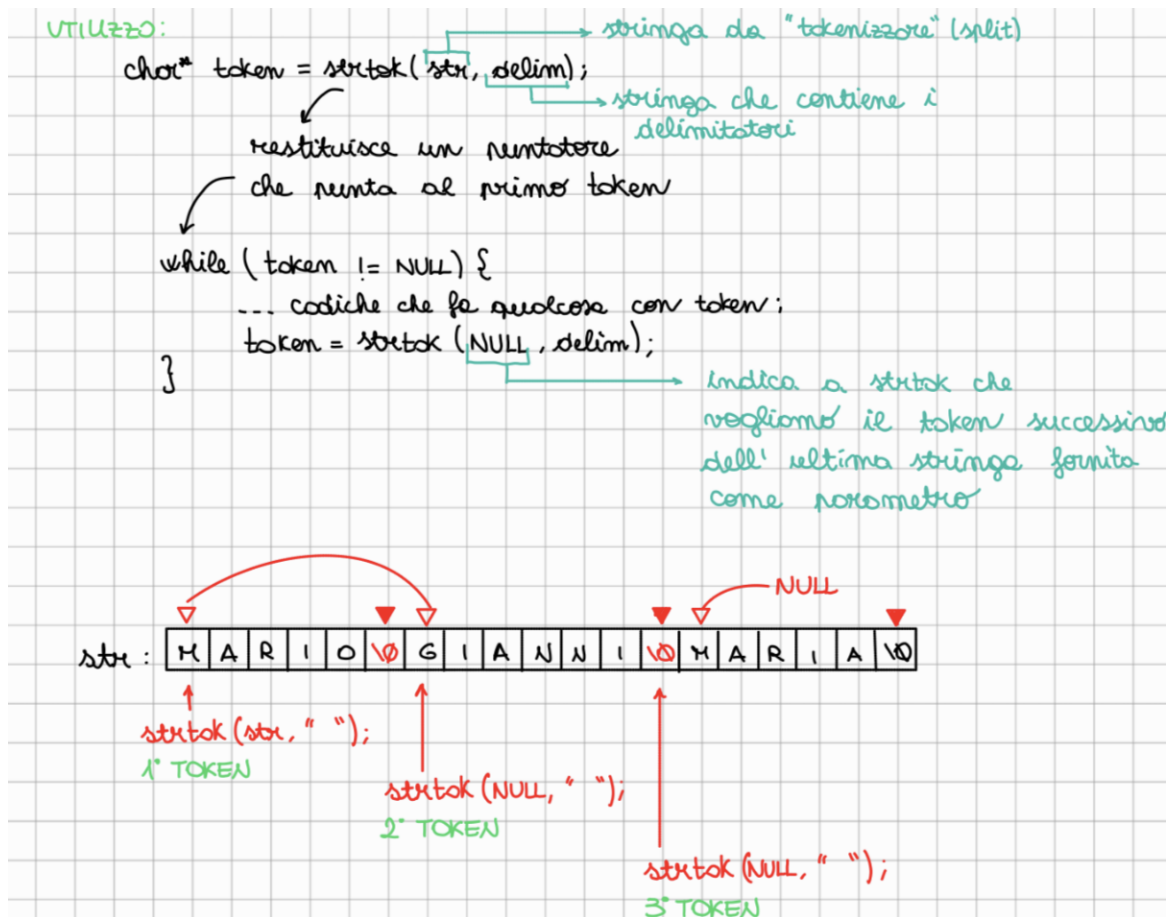


Figura 4.1: Funzionamento token

Vediamo ora un esempio di utilizzo:

```

#define DELIM = " , "

char* f(const char* s){
    char* temp = NULL;
    char* res = NULL;

    size_t len = strlen(s);
    char* tempo = malloc(sizeof(char)*(len+1));
    if(temp == NULL) goto cleanup;
    strncpy(temp, s, len);

    char* res = malloc(sizeof(char)*(len+1));
    if(res == NULL) goto cleanup;

    char* token = strtok(temp, DELIM);
    while(token){
        strncat(res, token, strlen(token));
        token = strtok(NULL, DELIM);
    }

    free(temp);
    return res;

    cleanup:
    if(temp != NULL) free(temp);
    if(res != NULL) free(res);
    return NULL;
}

```

Utilizziamo il `"#DEFINE DELIM"` per definire i delimitatori, in modo da non doverli riscrivere e poterli confondere.

Procediamo col vedere altre funzioni presenti nella libreria **string.h**:

```
int strcmp(const char* s1, const char* s2);
```

Questa funzione confronta `s1` ed `s2` e restituisce:

$$\begin{cases} <0 & \text{se } s1 \text{ precede } s2 \\ 0 & \text{se } s1 == s2 \\ >0 & \text{altrimenti} \end{cases}$$

Passiamo alla libreria **stdlib.h**:

```
#include <stdlib.h>
```

```
int atoi(const char* s); // converte s in un valore intero se possibile, altrimenti restituisce 0
```

Esempio:

```
atoi("10");    --> 10
atoi("10abc");  --> 10
atoi("abc10");  --> 0 perchè non inizia con un numero
atoi("-10");    --> -10
```

Esempio basilare di utilizzo:

```
void f(char* in[], int out[], int n){
    for(int i=0; i<n; i++)
        out[i] = atoi(in[i]);
}
```

Vediamo ora degli esercizi dove otteniamo il minimo della lunghezza di due stringhe e un esercizio che verifica se un array di stringhe è ordinato o meno, utilizzando le funzioni viste precedentemente:

```
int min_len(const char* a, const char* b){
    if(strlen(a)>=strlen(b))
        return strlen(a);
    else return strlen(b);
}
```

```
int is_sorted(char *v[], int n){
    for(int i=1; i<n; i++)
        if(strncmp(v[i-1], v[i], min_len(v[i-1], v[i])) > 0) return -1;
    return 0;
}
```

Passiamo ora alla libreria **stdio.h**:

```
#include <stdio.h>
```

```
int sscanf(const char* s, const char* format, ...);
```

Questa funzione è simile a `scanf`, ma invece di leggere da standard input, legge da una stringa. Il primo parametro è la stringa da cui leggere, il secondo è il formato da utilizzare per la lettura, e i successivi sono i puntatori alle variabili dove salvare i valori letti.

Abbiamo poi la funzione `sprintf` che è simile a `printf`, ma invece di scrivere su standard output, scrive su una stringa. Che fa sempre parte della libreria **stdio.h**:

```
int sprintf(char *d, const char* format, ...);
```

Questa funzione scrive in `d` la stringa format formattata con i valori passati come argomenti, quindi: il primo parametro è la stringa dove scrivere, il secondo è il formato da utilizzare per la scrittura, e i successivi sono i valori da scrivere.

Vediamo un esempio di utilizzo:

```
// converte una data da formato (input) "gg/mm/aaaa" a "mm/gg/aaaa"
int convert_date(const char *input, const char *output){
    int a,b,c;
    int res = sscanf(input, "%d/%d/%d", &a, &b, &c);
    if (res < 3) return -1;
    sprintf(output, "%d/%d/%d", b, a, c);
}
```

Vediamo ora `getchar()`:

```
int getchar(); // legge un carattere da standard input (da tastiera)
               // e lo restituisce come intero
int scanf(const char* format, ...); // legge da tastiera e salva i valori letti nelle
                                   // variabili passate come argomenti
```

## Gestione della Memoria

Vediamo ora come gestire la **memoria** in C.

- `void *malloc(size_t size);` → alloca `size` byte di memoria e restituisce un puntatore a tale memoria. Se l'allocazione fallisce, restituisce `NULL`.
- `void free(void *ptr);` → libera la memoria precedentemente allocata con `malloc`.
- `void *calloc(size_t count, size_t size);` → dove `count` indica il numero di elementi da allocare e `size` la dimensione di ciascun elemento. Restituisce un puntatore alla memoria allocata, inizializzata a zero.
- `void *memset(void *buf, int c, size_t len);` → imposta i primi `len` byte di `buf` al valore del byte meno significativo di `c`. Restituisce un puntatore a `buf`.
- `void *memcpy(void *dst, const void *src, size_t len);` → copia `len` byte da `src` a `dst`. Restituisce un puntatore a `dst`.

**ATTENZIONE:** non usare su buffer che si sovrappongono.

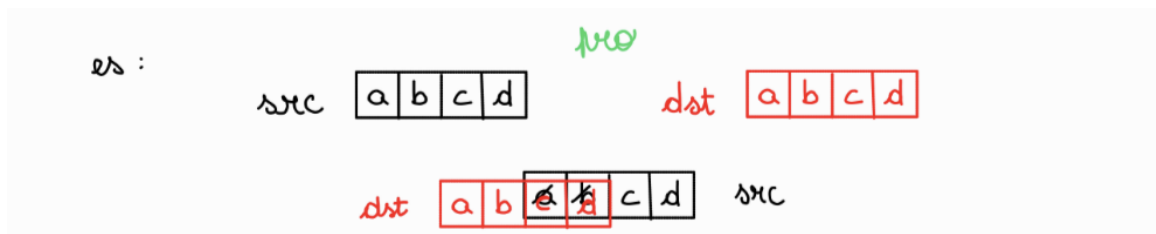


Figura 4.2: Esempio di buffer che si sovrappongono

- `void *memmove(void *dst, const void *src, size_t len);` → copia `len` byte da `src` a `dst`. Restituisce un puntatore a `dst`.
- ATTENZIONE:** può essere usato su buffer che si sovrappongono. Poichè `src` non viene mai sovrascritto.

Vediamo un esempio di utilizzo tramite una funzione `bubblesort` che ordina un array di stringhe di stessa lunghezza:

```
// buf buffer con all'interno stringhe, nel numero di stringhe presenti, size grandezza delle stringhe
void bubblesort(void *buf, size_t nel, size_t size){
    char *tmp = buf;
    for(;;){
        for(int i = 1, i<nel; i++){
            int swap = 0; // variabile aggiuntiva per uscire dal ciclo
            strncmp(tmp+(i-1)*size, tmp+i*size, size) > 0{
                // scambio elementi
                swap_mem(tmp+(i-1)*size, tmp+i*size, size);
                swap = 1;
            }
        }
    }
}
```

```

    }
    // condizione di uscita poichè for infinito
    if(!swap) break;
}
}

void swap_mem(void *s1, void *s2, size_t l){
    char *buf = malloc(l);
    if(buf == NULL) return;
    memmove(buf, s1, l); // copiamo s1 in buf
    memmove(s1, s2, l); // copiamo s2 in s1
    memmove(s2, buf, l); // copiamo buf in s2
    free(buf);           // liberiamo la memoria allocata per buf
}

```

Vediamo ora un ulteriore esempio di utilizzo, tramite una funzione che indica il carattere più frequente in un array buf di dimensione nel:

```

int most_freq_char(void *buf, size_t nel){
    int *c = malloc(sizeof(int)*256); // array per contare le occorrenze di ogni carattere
    assert(c != NULL);                // equivalente a if(c == NULL) return -1;
    memset(c, 0, sizeof(int)*256);    // inizializziamo a 0 l'array delle occorrenze
    // potevamo semplicemente fare int *c = calloc(sizeof(int)*256) che inizializza a 0 l'array,
    // ma abbiamo visto che è più efficiente usare malloc e memset
    for(int i=0; i<nel; i++)
        c[buf[i]]++; // incrementiamo il contatore del carattere
    int max = 0; // variabile per tenere traccia del carattere più frequente
    for(int i=1; i<256; i++)
        if(c[i] > c[max])
            max = i; // aggiorniamo il carattere più frequente
    free(c);
    return max;
}

```

## Funzioni di Ordinamento e Ricerca

Vediamo ora le funzioni di ordinamento e ricerca presenti nella libreria **stdlib.h**:

```
void qsort(void *base, size_t numelem, size_t sizeelem, int (*compar)(const void *, const void *));
```

Dove:

- **base** → è un puntatore al primo elemento dell'array da ordinare
- **numelem** → è il numero di elementi in base
- **sizeelem** → è la dimensione di ogni elemento dell'array
- **compar** → è un puntatore a una funzione di confronto che prende in ingresso due puntatori a elementi dell'array e restituisce un intero che indica l'ordine degli elementi:

$$\begin{cases} < 0 & \text{se il primo elemento precede il secondo} \\ = 0 & \text{se i due elementi sono uguali} \\ > 0 & \text{se il primo elemento segue il secondo} \end{cases}$$

**ATTENZIONE:** la funzione di confronto deve essere definita dall'utente in base al tipo di dati dell'array da ordinare

Vediamo un esempio di utilizzo:

```

int comparatore(const void *a, const void *b){
    int x = *(int *)a;
    int y = *(int *)b;
    return x - y; // confronto tra due interi
}

void sort_ints(int ints[], size_t n){
    qsort(ints, n, sizeof(int), comparatore);
}

```

Vediamo una versione più articolata di questa tipologia di esercizio, dove ordiniamo un array di stringhe in ordine alfabetico:

```
int comparatore_stringhe(const void *a, const void *b){
    const char *x = *(char **)a; // cast a char** per ottenere il puntatore alla stringa
    const char *y = *(char **)b; // cast a char** per ottenere il puntatore alla stringa
    // ad a e b utilizziamo il doppio asterisco perchè a e b sono puntatori a puntatori
    return strcmp(x, y); // confronto tra due stringhe
}

void sort_strings(char *strings[], size_t n){
    qsort(strings, n, sizeof(char *), comparatore_stringhe);
}
```

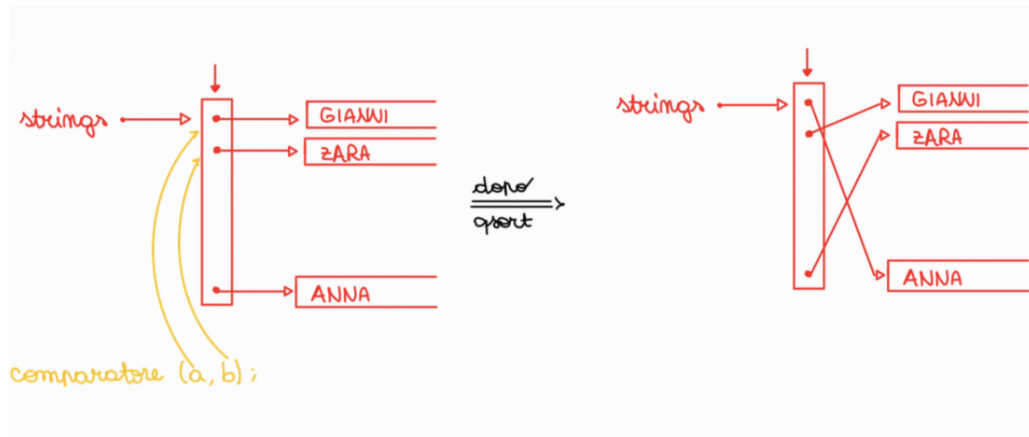


Figura 4.3: Qsort

Ricordiamo che:

- $p \rightarrow$  indirizzo
- $*p \rightarrow$  valore
- $\&x \rightarrow$  indirizzo di  $x$
- $*(&x) \rightarrow$  valore di  $x$
- $(\text{char } **)a \rightarrow$  puntatore a elemento ( $\text{char } *$ )
- $*(\text{char } **)a \rightarrow$  valore di elemento ( $\text{char } *$ )

Altra funzione:

```
void bsearch (const void* key, const void *base, size_t numelem,
              size_t sizeelem, int (*compar)(const void *, const void *));
```

Cerca l'elemento **key** nell'array **ordinato** base di **numelem** elementi, ciascuno di dimensione **sizeelem**, utilizzando la funzione di confronto **compar**. Restituisce un puntatore all'elemento trovato o NULL se l'elemento non è presente nell'array.

**ATTENZIONE:** l'array base deve essere ordinato poichè è una ricerca binaria.

Esempio di utilizzo:

```
int comparatore(const void *a, const void *b){
    int x = *(int *)a;
    int y = *(int *)b;
    return x - y; // confronto tra due interi
}

int search_int(int v[], size_t n, int x){
    void * risultato = bsearch(&x, v, n, sizeof(int), comparatore);
    // &x poichè devo passare un puntatore
    return risultato != NULL;
}
```

## Funzioni Matematiche

Vediamo ora le funzioni matematiche presenti nella libreria **math.h**, hanno tutte la stessa sintassi:

```
double sqrt(double x);           // restituisce la radice quadrata di x
double pow(double x, double y);  // restituisce x elevato alla potenza y
double sin(double x);            // restituisce il seno di x (x in radianti)
double cos(double x);            // restituisce il coseno di x (x in radianti)
double tan(double x);            // restituisce la tangente di x (x in radianti)
double log(double x);            // restituisce il logaritmo naturale di x
double exp(double x);            // restituisce e elevato alla potenza x
```

**ATTENZIONE:** per compilare bisogna aggiungere lo switch `-lm` per linkare la libreria matematica, altrimenti si ottiene un errore di linker.

## Funzioni per la Gestione degli Stream

Vediamo ora le funzioni per la gestione degli stream presenti nella libreria **stdio.h**. Gli stream sono un'astrazione che rappresenta un flusso di byte, che può essere utilizzato per leggere o scrivere dati. Gli stream possono essere associati a file, dispositivi di input/output, o anche a stringhe in memoria.

Quando eseguiamo un programma in C, il sistema operativo assegna al corrispondente processo automaticamente tre stream standard:

- **stdin** → stream standard di input, associato alla tastiera. Utilizzato per leggere l'input del programma.
- **stdout** → stream standard di output, associato al terminale (schermo), utilizzato per scrivere l'output del programma.
- **stderr** → stream standard di errori, associato al terminale (schermo), utilizzato per scrivere messaggi di errore.

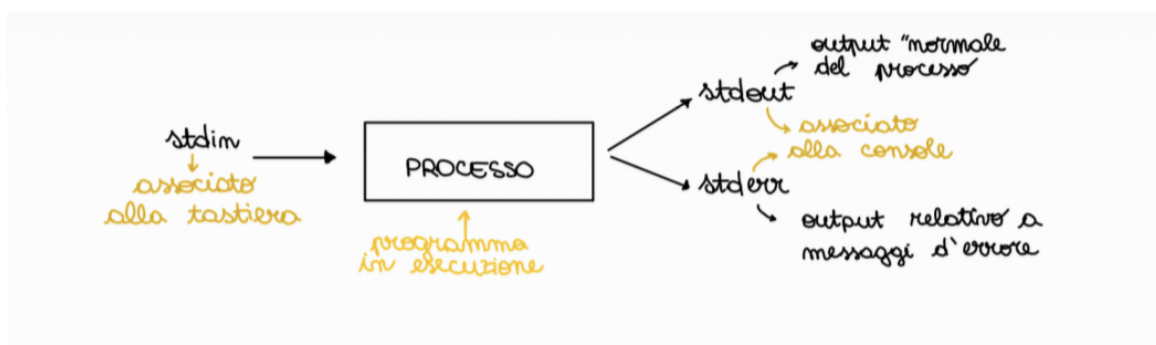


Figura 4.4: Stream standard

**Redirezione stdout:** è possibile redirigere l'output di un programma verso un file invece che verso il terminale, utilizzando l'operatore `>` in bash:

```
$ ./programma > output.txt
```

In questo modo, l'output del programma verrà scritto nel file `output.txt` invece che sul terminale.

**Redirezione stderr:** è possibile redirigere anche l'output di errori verso un file, utilizzando l'operatore `2>` in bash:

```
$ ./programma 2> error.txt
```

In questo modo, i messaggi di errore del programma verranno scritti nel file `error.txt` invece che sul terminale.

**Redirezione stdin:** è possibile redirigere l'input di un programma da un file invece che dalla tastiera, utilizzando l'operatore `<` in bash:

```
$ ./programma < input.txt
```

In questo modo, l'input del programma verrà letto dal file `input.txt` invece che dalla tastiera.

**Redirezione combinata:** è possibile redirigere sia l'output che l'input di un programma, utilizzando entrambi gli operatori:

```
$ ./programma < input.txt > output.txt
```

In questo modo, l'input del programma verrà letto dal file `input.txt` e l'output verrà scritto nel file `output.txt`. Per gestire gli stream da codice ho due opzioni:

- utilizzo le libc
- utilizzo le syscall POSIX

Le libc sono più semplici da utilizzare, ma le syscall POSIX sono più efficienti. Vediamo ora le funzioni per la gestione degli stream presenti nella libreria **stdio.h**:

```
FILE *fopen(const char *path, const char *mode); // apre un file e restituisce un puntatore
|                                               | di tipo FILE
|                                               |
|                                               | ____ mode è una stringa che rappresenta le operazioni
|                                               | da svolgere sul file:
|                                               | r -> leggi, w -> scrivi, a -> aggiungi
|
| ____ percorso sul file system (disco) attraverso cui raggiungere il file
| Esempio: "/home/studente/a.txt"

int ferror(FILE *f); // restituisce un valore diverso da 0
| se si è verificato un errore durante l'operazione di I/O su f

void clearerr(FILE *f) // pone pari a 0 il flag di errore

int fclose(FILE *stream); // chiude lo stream associato al file

int fprintf(FILE *stream, const char *format, ...); // scrive su stream una stringa formattata

int fscanf(FILE *stream, const char *format, ...); // legge da stream una stringa formattata

char *fgets(char *str, int n, FILE *stream); // legge da stream una stringa di al massimo n-1 caratteri
| e la salva in str. Restituisce str in caso di successo,
| e NULL in caso di errore o fine file

int feof(FILE *f); // restituisce un valore diverso da 0 se si è raggiunta la fine del file

size_t fread(void *ptr, size_t size, size_t numitems, FILE *f); // legge da 'f' 'numitems' blocchi di
| elementi da leggere di
| dimensione 'size' e
| li salva in 'ptr'.
| Restituisce il numero di elementi
| letti con successo

size_t fwrite(const void *ptr, size_t size, size_t numitems, FILE *f); // scrive su 'f' 'numitems'
| blocchi di
| elementi da scrivere di
| dimensione 'size' e
| li legge da 'ptr'.
| Restituisce il numero di elementi
| scritti con successo.

int fseek(FILE *f, long offset, int whence); // sposta il puntatore al prossimo byte da
|
| | leggere/scrivere in 'f'
|
| |
| |
| | ____ SEEK_SET -> inizio del file
| | SEEK_CUR -> posizione attuale
| | SEEK_END -> fine del file
```

```

|
|_____ spostamento da applicare al puntatore espresso in byte
          rispetto alla posizione indicata da 'whence'

```

Esempi per capire meglio `fseek`:

```

fseek(f,10,SEEK_CUR); // sposta il puntatore di 10 byte in avanti rispetto alla posizione attuale
fseek(f,5,SEEK_SET); // sposta il puntatore al 5 byte dall'inizio del file
fseek(f,-2,SEEK_END); // sposta il puntatore a 2 byte prima della fine del file

```

Continuiamo con l'ultima funzione:

```

long ftell(FILE *f); // restituisce la posizione attuale del puntatore
                     in byte rispetto all'inizio del file

```

**ATTENZIONE:** bisogna sempre ricordarsi di effettuare i controlli di errore su tutte queste funzioni, poichè possono fallire per vari motivi (file non esistente, permessi insufficienti, ecc...).

Vediamo qualche esempio di utilizzo:

```

// La funzione read_file deve allocare dinamicamente un buffer
// e caricarci dentro il contenuto di un file
int read_file(char *filename, void **buffer, size_t *size){
    FILE *fd = NULL;
    int res;
    long len;
    *buffer = NULL;

    fd = fopen(filename, "r");
    if(fd == NULL) goto cleanup;

    res = fseek(fd, 0, SEEK_END); // sposto il puntatore alla fine del file
    if(res == -1) goto cleanup;

    len = ftell(fd); // ottengo la lunghezza del file
    if(len == -1) goto cleanup;
    *size = (size_t)len; // salvo la lunghezza del file in size

    *buffer = malloc(len); // alloco un buffer di dimensione len
    if(*buffer == NULL) goto cleanup;

    res = fseek(fd, 0, SEEK_SET); // sposto il puntatore all'inizio del file
    if(res == -1) goto cleanup;

    res = fread(*buffer, 1, *size, fd); // leggo il contenuto del file nel buffer
    if(res != *size) goto cleanup; // controllo che abbia letto tutto il file

    res = fclose(fd); // chiudo il file
    if(res == -1) goto cleanup;
    return 0; // successo

cleanup:
    int old_errno = errno; // salvo il valore di errno prima di effettuare le operazioni di pulizia
    if(fd != NULL) fclose(fd);
    if(*buffer != NULL) free(*buffer);
    errno = old_errno; // ripristino il valore di errno prima di restituire l'errore
    return -1;
}

```

Passiamo ora alle syscall POSIX per la gestione degli stream, che sono più efficienti ma anche più complesse da utilizzare:

```

#include <fcntl.h>
int open(const char *pathname, int flags, mode_t mode); // apre un file e restituisce
                                                         un file descriptor

```

```

|           |           |____ mode è un intero che rappresenta i permessi
|           |           del file (solo se si usa O_CREAT)
|           |_____ flags è un intero che rappresenta la modalità di
|           apertura del file:
|           O_RDONLY -> leggi, O_TRUNC -> tronca il file a 0 byte
|           O_CREAT -> crea il file se non esiste
|
|
|_____ percorso sul file system (disco) attraverso cui raggiungere il file
Esempio: "/home/studente/a.txt"

```

**ATTENZIONE:** la funzione open restituisce un file descriptor, che è un intero che rappresenta il file aperto. Il file descriptor 0 è riservato per stdin, 1 per stdout e 2 per stderr. I file descriptor restituiti da open sono sempre maggiori o uguali a 3.  
Esempio di utilizzo:

```

int fd = open("file.txt",O_CREAT|O_RDONLY|O_TRUNC, 0644); // apre il file file.txt in modalità lettura
                                                         e scrittura, se il file non esiste
                                                         lo crea, se esiste lo tronca a 0 byte

```

Abbiamo inoltre una funzione apposita per la scrittura e lettura:

```

ssize_t write(int fd, const void *buf, size_t count); // scrive su 'fd' 'count' byte da buf.
                                                         Restituisce il numero di byte scritti
                                                         con successo

```

```

ssize_t read(int fd, void *buf, size_t count); // legge da 'fd' 'count' byte

```

```

off_t lseek(int fd, off_t offset, int whence); // Invece di ftell si può usare
                                                         lseek(fd, 0, SEEK_CUR) per ottenere
                                                         la posizione attuale del file descriptor

```

## 4.2 Esecuzione dei programmi

### Processi

Un processo è un'istanza di un programma in esecuzione. Ogni processo ha un proprio spazio di indirizzamento, che include il codice del programma, i dati, lo stack e l'heap. Ogni processo ha anche un proprio stato, che può essere:

- **Running** → il processo è in esecuzione
- **Ready** → il processo è pronto per essere eseguito, ma non è ancora stato assegnato a una CPU
- **Blocked** → il processo è in attesa di un evento (ad esempio, l'input dell'utente o la fine di un'operazione di I/O)
- **Terminated** → il processo ha terminato la sua esecuzione

Più istanze dello stesso programma in **esecuzione concorrente**, ovvero che nel mentre è già in esecuzione un processo, ne viene avviato un altro che esegue lo stesso programma; sono rappresentate da processi distinti, con spazi di indirizzamento separati, e possono essere eseguiti in parallelo su più CPU o in modo interleaved su una singola CPU.

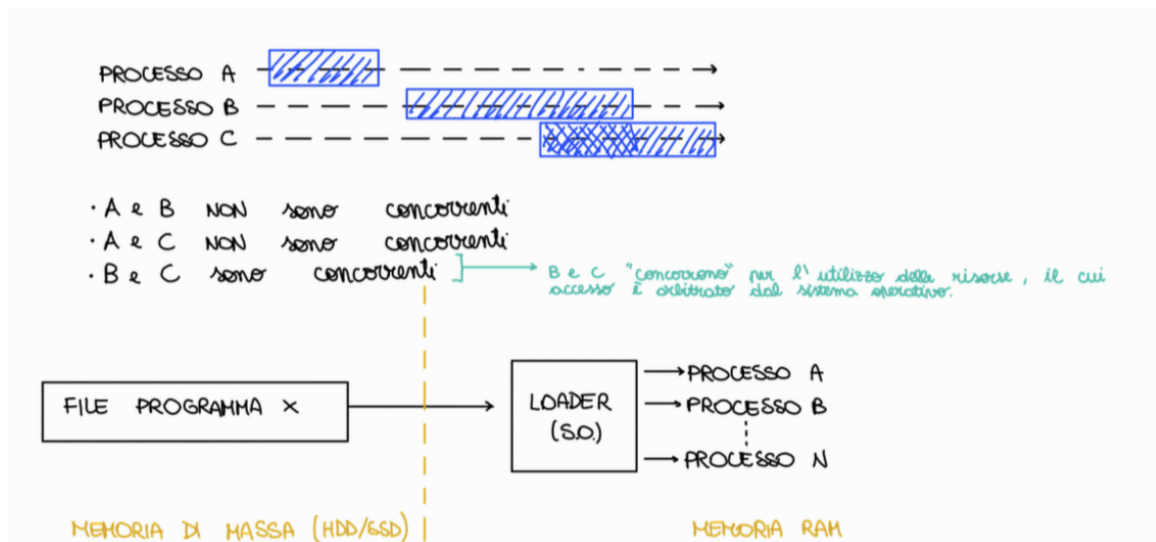


Figura 4.5: Programmi in esecuzione concorrente

In questo esempio B e C sono due processi concorrenti, ovvero "concorrono" per l'utilizzo delle risorse, il cui accesso è gestito dal sistema operativo.

Nei processi abbiamo la presenza del **Loader**, che prepara la memoria al fine di eseguire il processo, caricando il codice del programma, i dati, lo stack e l'heap. Il loader è responsabile anche di risolvere le dipendenze dinamiche (ad esempio, le librerie condivise) e di inizializzare il processo prima di avviarlo.

Un processo è costituito dai seguenti elementi informativi:

- **Immagine di Memoria** → contiene il codice del programma, i dati definiti dal programmatore, lo stack, l'heap, ecc...
- **Stato della CPU** → indica se il processo è in esecuzione, pronto, bloccato o terminato. Contiene quindi registri, ecc...
- **Risorse in uso** → indica le risorse che il processo sta utilizzando, come file aperti, socket, ecc...
- **Metadati** → contiene informazioni sul processo, come il PID (Process ID), il PPID (Parent Process ID), la priorità, user, ora di avvio, linea di comando, ecc... Questi metadati si possono trovare nel *file system* all'interno di una directory speciale chiamata */proc*, che contiene un file per ogni processo in esecuzione, con il nome del file corrispondente al PID del processo. Ad esempio, il file */proc/1234* contiene i metadati del processo con PID 1234.

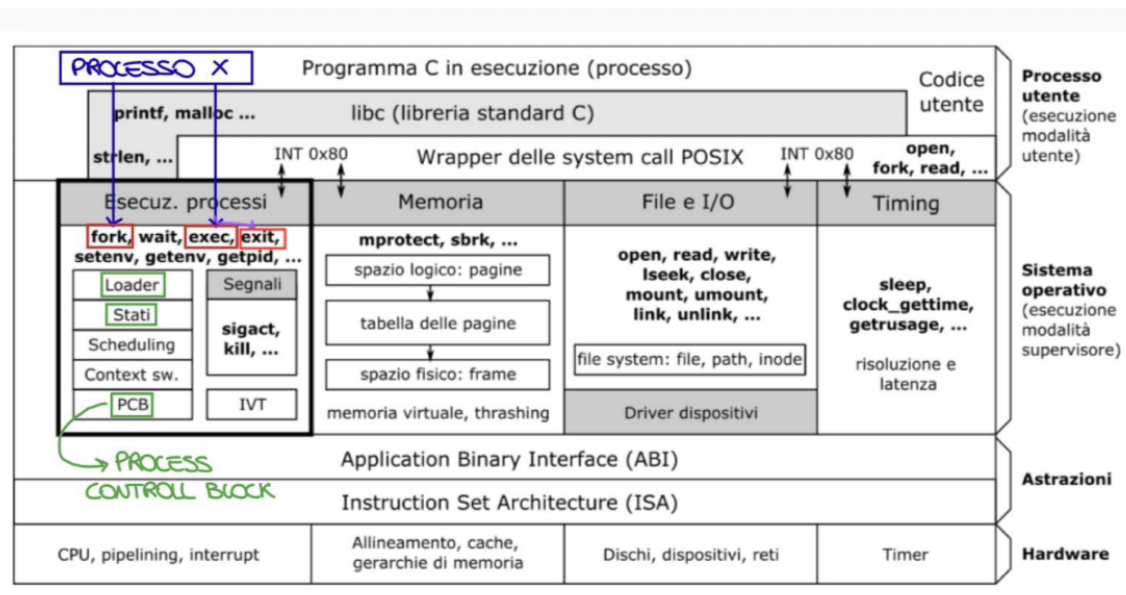


Figura 4.6: Processi Funzionamento

I processi per lavorare nella loro immagine di memoria non accedono direttamente alla memoria fisica (RAM), ma utilizzano un meccanismo chiamato **memoria virtuale**, gestita dal sistema operativo; che permette di mappare la memoria virtuale del processo alla memoria fisica del sistema.

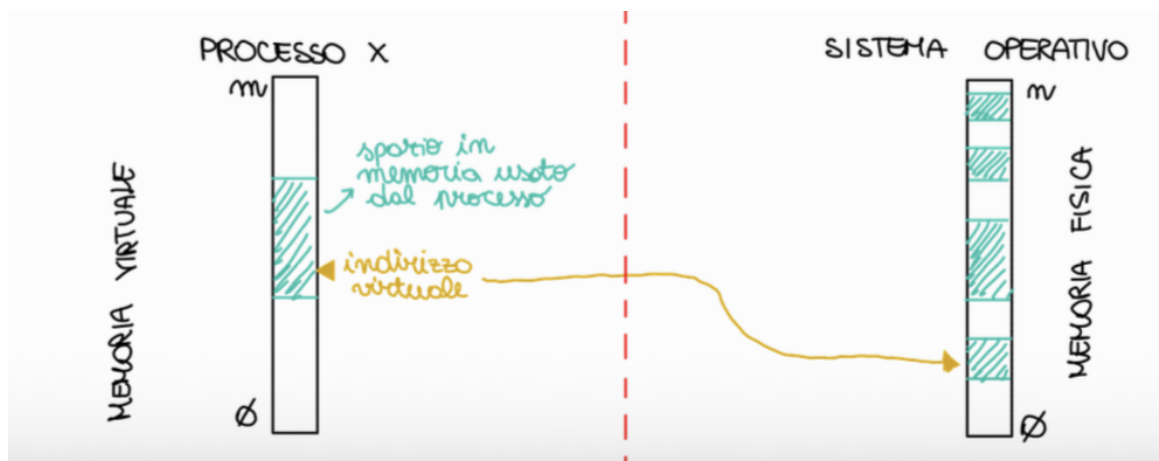


Figura 4.7: Memoria Virtuale

Consideriamo come è organizzata l'immagine di memoria di un processo dopo il caricamento di un file eseguibile (ad esempio in formato ELF). Ogni processo vede il proprio spazio di memoria come un array lineare di byte, i cui indirizzi partono da 0 e arrivano fino a  $2^b - 1$ , dove  $b$  rappresenta il numero di bit utilizzati per i puntatori (tipicamente 32 o 64). I sistemi operativi garantiscono l'isolamento tra i processi attraverso il meccanismo della **memoria virtuale**. Questo impedisce a un processo di accedere allo spazio di memoria di un altro processo, sia in modo malevolo sia a causa di errori di programmazione (bug).

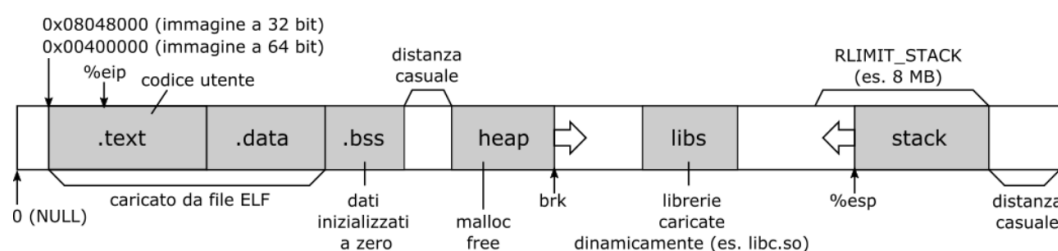


Figura 4.8: Immagine di un Processo

Immaginiamo di avere un codice del tipo:

```
int i[10000], j = 10;
int main(int argc, char *argv[]){
    static char *k = "hello";
    char *p = malloc(sizeof(int) * 10000); // alloca un array di 10000 interi nell'heap
    free(p);
    printf(...);
}
```

Immaginando di eseguire questo programma, l'immagine di memoria del processo sarà organizzata in questo modo:

- **i** → si trova in `.bss` poiché è un array globale non inizializzato
- **j** → si trova in `.data` poiché è una variabile globale inizializzata
- **k** → si trova in `.data` poiché è una variabile statica inizializzata
- **"hello"** → si trova in `.rodata` poiché è una stringa letterale (read-only)
- **p** → la variabile si trova nello stack, mentre la memoria allocata con `malloc` si trova nell'heap
- **main** → si trova in `.text` poiché è codice eseguibile
- **argc** e **argv** → si trovano nello stack poiché sono parametri della funzione `main`

## 4.3 Multiprogrammazione

La **multiprogrammazione** è una tecnica che permette ai programmi di avere più processi residenti in memoria contemporaneamente, condividendo le risorse del sistema (CPU, memoria, I/O).

Il sistema operativo per gestire le multiple richieste di accesso alla CPU da parte dei processi concorrenti utilizza un sistema di **Time-Sharing**. Nel **Time-Sharing** tutti i processi sono "sospesi" fino a quando non gli viene assegnato dal Sistema Operativo l'uso della CPU per un intervallo di tempo (**time slice**) durante il quale possono eseguire. Al termine del time slice, il processo viene nuovamente sospeso e viene assegnato un nuovo time slice a un altro processo, e così via in modo ciclico.

Gli stati principali di un processo in un sistema multiprogrammato sono in genere tre:

- **ready** → il processo è pronto per l'esecuzione, ma è tenuto fermo poiché tutti i core della CPU disponibili sono impegnati nell'esecuzione di altri processi;
- **running** → il processo è attualmente in esecuzione su un core della CPU;
- **waiting/sleeping** → il processo è in attesa di un evento, come il completamento di un'operazione di I/O. Ad esempio, un processo entra nello stato waiting quando esegue una chiamata a sistema che richiede la lettura di un blocco di dati da disco. Quando un processo entra in stato waiting, libera il core su cui era in esecuzione in modo che diventi disponibile per un altro processo ready.

Un processo entra tipicamente nello stato di *attesa* (*waiting* o *sleeping*) in modo volontario, eseguendo una *system call* che può richiedere tempi di completamento non trascurabili, come ad esempio operazioni di accesso alla rete o al disco. Al termine di tali operazioni, il processo viene risvegliato e passa nello stato di *ready*, ovvero pronto per essere eseguito. A questo punto interviene lo *scheduling*, ovvero il meccanismo con cui il sistema operativo seleziona uno dei processi nello stato *ready* per assegnargli la CPU. Nei sistemi operativi moderni è presente il meccanismo di **preemption**, che consente di interrompere l'esecuzione di un processo in stato *running* per assegnare la CPU a un altro processo. Questo meccanismo è fondamentale per garantire equità nell'uso delle risorse. In particolare, la *preemption* aiuta a prevenire il fenomeno della **starvation**, in cui un processo rischia di rimanere indefinitamente in attesa senza poter progredire, a causa di altri processi che monopolizzano la CPU. La *preemption* è una caratteristica tipica dei sistemi operativi *time-sharing*, come *Linux*, *macOS* e *Windows*. Tali sistemi permettono di eseguire un numero di processi superiore rispetto ai core disponibili, creando l'illusione di un'esecuzione simultanea grazie alla rapida alternanza dei processi sulla CPU.

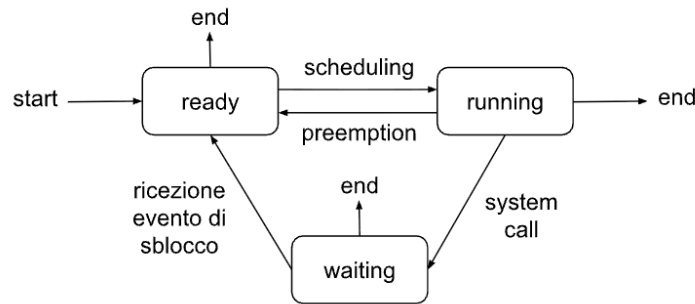


Figura 4.9: Stati dei processi e CPU

Per evitare la **starvation**, possiamo adoperare:

1. **Preemption**
2. **Scheduler** con algoritmo di tipo *fair time sharing* (esempio round-robin).

Per consentire a tutti i processi ready di progredire uniformemente nell'esecuzione, un popolare algoritmo di schedulazione è il **round-robin**, che assegna un core a rotazione a ciascun processo ready per la durata di una time slice.

Il sistema operativo può anche implementare un sistema di **priorità** per i processi, in modo che alcuni processi vengano eseguiti più frequentemente di altri. In pratica è spesso possibile assegnare una priorità a un processo in modo che venga schedulato per l'esecuzione più frequentemente degli altri. Il funzionamento dell'algoritmo di schedulazione round-robin è illustrato nel seguente diagramma a tre processi con la stessa priorità:

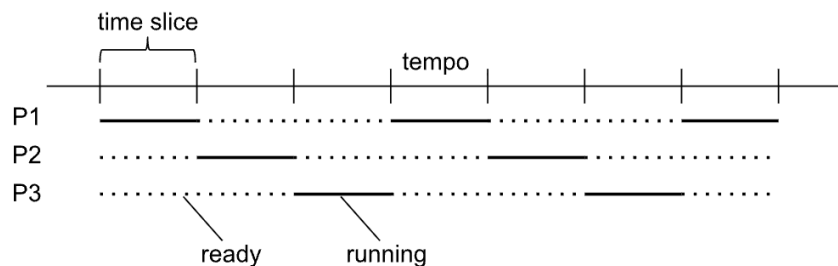


Figura 4.10: Round-Robin

Come si vede, l'algoritmo round-robin non genera starvation nei processi, che hanno tutti l'opportunità di progredire nella computazione. Il sistema operativo tiene tipicamente i PCB dei processi organizzati in liste separate per ciascuno stato.

## Funzioni per la gestione dei processi

Vediamo ora le funzioni per la gestione dei processi presenti nella libreria **unistd.h**:

**fork ()** -> Genera un nuovo processo (chiamato processo figlio) che è una copia del processo chiamante (chiamato processo padre). Il processo figlio ha un proprio PID (Process ID) e PPID (Parent Process ID), ma condivide con il processo padre lo stesso codice, dati, stack e heap (inizialmente). La funzione fork restituisce 0 al processo figlio e il PID del processo figlio. Una volta generata la copia del processo, vengono messi in stato di READY entrambi i processi (padre e figlio) e il sistema operativo decide quale dei due eseguire per primo.

**ATTENZIONE:** in questo momento i due processi possono essere distinti solo perchè hanno PID diversi, ma condividono lo stesso spazio di memoria, quindi se uno dei due processi modifica una variabile, anche l'altro processo vedrà la modifica.

```
pid_t fork();
```

Restituisce:  $\begin{cases} -1 & \text{in caso di errore} \\ 0 & \text{restituisce il PID del nuovo processo creato (figlio)} \\ >0 & \text{restituito al processo che ha invocato fork() (processo padre) e rappresenta il PID del processo creato.} \end{cases}$

Per distinguere il codice che va eseguito nel processo padre/figlio si usa la seguente struttura:

```
#include <stdlib.h>
#include <unistd.h>
int main(){
    pid_t pid = fork();
    if(pid == -1){
        // gestione errore
        exit(-1);
    } else if(pid == 0){
        // codice eseguito dal processo figlio
        printf("sono il processo figlio\n");
    } else {
        // codice eseguito dal processo padre
        printf("sono il processo padre\n");
    }
    return 0;
}
```

Vediamo ora **Fork Multiple**:

```
int main(){
    pid_t p1 = fork();
    pid_t p2 = fork();
    pid_t p3 = fork();
    // In totale si ottengono 8 processi (2^3)
    if(p3 == 0){
        // eseguito dai processi figli dell'ultima fork()
        printf("*");
        // 4 processi stampano "*"
    } else if(p2 == 0){
        // eseguito dai processi che sono figli della seconda fork()
        // ma NON figli della terza
        printf("+");
        // 2 processi stampano "+"
    } else if(p1 == 0){
        // eseguito dai processi che sono figli della prima fork()
        // ma NON figli della seconda e terza
        printf("-");
        // 1 processo stampa "-"
    } else {
        // processo padre originale (non figlio di nessuna fork)
        printf("0");
        // 1 processo stampa "0"
    }
}
```

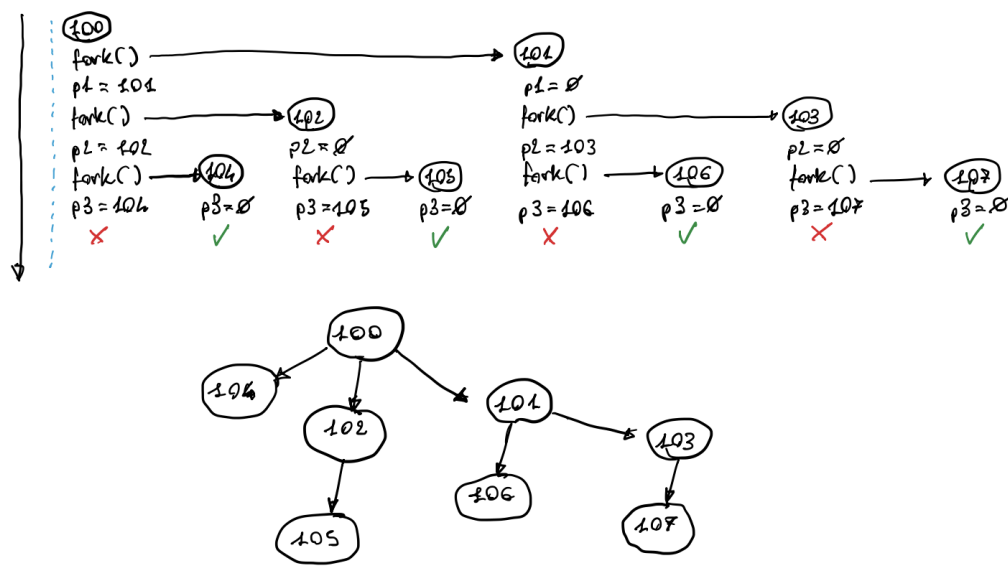


Figura 4.11: Fork Multipli

Quale codice genera il seguente albero di processi?

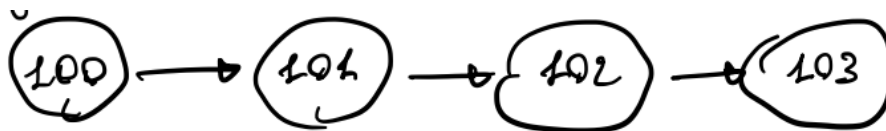


Figura 4.12: Fork Multipli

```
int main(){
    pid_t p1 = fork();
    if(p1 == 0) pid_t p2 = fork();
    if(p2 == 0) pid_t p3 = fork();
    if(p3 == 0) printf("*");
    // abbiamo 1 solo processo che stampa * dato che solo 1 processo
    // avrà come padre p3 e quindi eseguirà il codice che stampa "*"
}
```

FARE ESERCIZIO != "£="£=")£"=£"

Altri comandi utili:

|                                 |                                                                                            |
|---------------------------------|--------------------------------------------------------------------------------------------|
| <code>pid_t getpid();</code>    | -> restituisce il PID del processo chiamante (PID processo figlio)                         |
| <code>pid_t getppid();</code>   | -> restituisce il PPID del processo chiamante (PPID processo padre)                        |
| <code>_exit(int status);</code> | -> funziona come <code>exit()</code> ma non esegue l'eventuale gestione della terminazione |

## Sincronizzazione dei processi

La sincronizzazione dei processi è un aspetto fondamentale nei sistemi operativi, poiché permette di coordinare l'accesso alle risorse condivise tra più processi concorrenti. In un sistema multiprogrammato, più processi possono accedere contemporaneamente a risorse condivise, come ad esempio file, memoria, dispositivi di I/O, ecc. . . Senza un meccanismo di sincronizzazione, si potrebbero verificare situazioni di **race condition**, in cui il risultato di un'operazione dipende dall'ordine di esecuzione dei processi, portando a comportamenti imprevedibili e potenzialmente dannosi. Il processo padre può quindi voler sapere quando i suoi processi figli terminano la loro esecuzione.

**IMPORTANTE:** non possiamo mai far alcuna assunzione sui tempi e sull'ordine con cui verranno eseguite le istruzioni nei processi concorrenti, poiché dipende dallo scheduler del sistema operativo, che può decidere di eseguire i processi in modo interleaved o in parallelo su più CPU.

```
#include <sys/types.h>
```

```
#include <sys/wait.h>
pid_t wait(int *status);    -> restituisce il PID del processo figlio che è terminato, e salva lo stato
                             di terminazione del processo figlio in status.
                             Se non ci sono processi figli, restituisce -1. Wait è bloccante
                             fino alla terminazione di un processo figlio.
```

Quando invoco wait sono possibili due casi:

1. **Il processo figlio è già terminato** → in questo caso wait restituisce immediatamente il PID del processo figlio terminato e lo stato di terminazione.
2. **Il processo figlio non è ancora terminato** → in questo caso wait rimane bloccato fino a quando il processo figlio non termina, momento in cui restituisce il PID del processo figlio terminato e lo stato di terminazione.

**ATTENZIONE:** se il processo padre termina prima del processo figlio, il processo figlio diventa un processo *orfano* e viene adottato dal processo *init* (PID 1), che si occupa di gestire i processi orfani.

Per leggere il valore di uscita del processo figlio, possiamo utilizzare le seguenti macro:

```
wifEXITED(status_ptr) -> restituisce il valore 1 se il processo figlio è terminato normalmente
                           (ad esempio, chiamando exit() o return dalla funzione main)
                           altrimenti restituisce 0
wEXITSTATUS(status_ptr) -> restituisce il codice di terminazione ottenuto dal figlio
```

## Caricamento di programmi

Il caricamento di un programma è il processo mediante il quale un file eseguibile viene caricato in memoria e preparato per l'esecuzione.

```
#include <unistd.h>
int execvp(const char *filename, char *argv[]);
|
|                                     |_____ array di stringhe terminato con NULL che contiene gli
|                                     argomenti da passare al main del programma,
|                                     dove argv[0] deve essere il nome del programma eseguibile
|
|_____ percorso su disco del programma
|       da eseguire (esempio: "/bin/ls")
|
|_____ restituisce - 1 in caso di errore
```

execvp sostituisce integralmente l'immagine di memoria del processo che la invoca con quella del programma da eseguire:

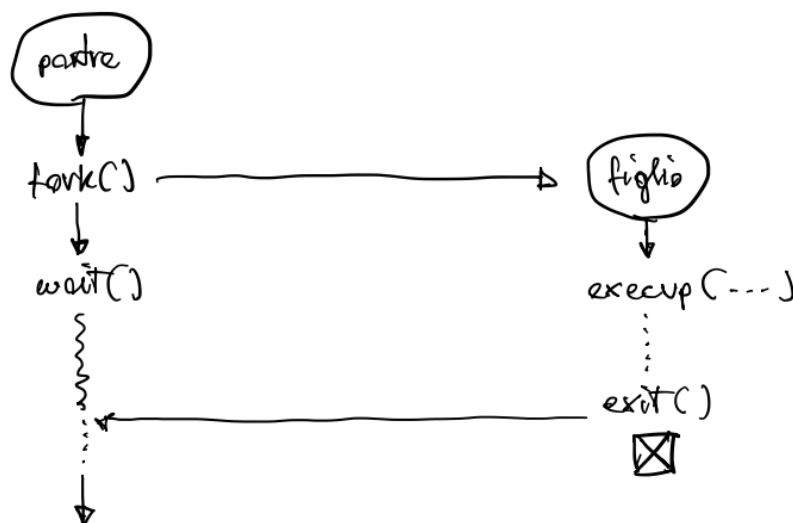


Figura 4.13: Esecuzione di un programma con `execvp`

```
int main(){
    if(pid == -1){
        perror("fork");
        return EXIT_FAILURE;
    }
    if(pid == 0){
        char* argv[] = {"ls", "-l", NULL};
        execvp("ls", argv);
        perror("execvp");
        _exit(EXIT_FAILURE);
    }
    return EXIT_SUCCESS;
}
```

## 4.4 Thread

I thread nascono per ridurre il costo computazionale (*overhead*) dovuto alla multiprogrammazione. Un processo può essere composto da una o più unità di esecuzione chiamate **thread**.

Un **thread** è un'unità di esecuzione più leggera rispetto a un processo. I thread appartenenti allo stesso processo condividono:

- lo stesso codice del programma;
- la gran parte dei dati in memoria (costanti, variabili globali, variabili statiche, heap, ecc...);
- file aperti;
- segnali e le loro routine di gestione;
- directory di lavoro corrente;
- diritti dell'utente proprietario del processo;
- lo stesso spazio di memoria;
- le stesse risorse del processo padre;

Ovvero la **sezione dati**, **TUTTI** i thread associati ad uno stesso processo ne condividono l'immagine di memoria.

Grazie a questa condivisione, i thread possono collaborare ed eseguire più attività contemporaneamente all'interno dello stesso processo, migliorando efficienza e reattività del programma.

In pratica, un thread può essere visto come una sequenza di istruzioni che viene eseguita in modo indipendente rispetto agli altri thread dello stesso programma.

Ciascun thread gestisce in modo indipendente le seguenti informazioni:

1. Un identificatore univoco (**Thread ID**);
2. **Stack**
3. **Stato** della CPU (registri, IP, ecc...)
4. alcune variabili **globali** (ad esempio, `errno`)
5. **Metadati** (proprietà specifiche di ogni thread, come priorità, stato, ecc...)

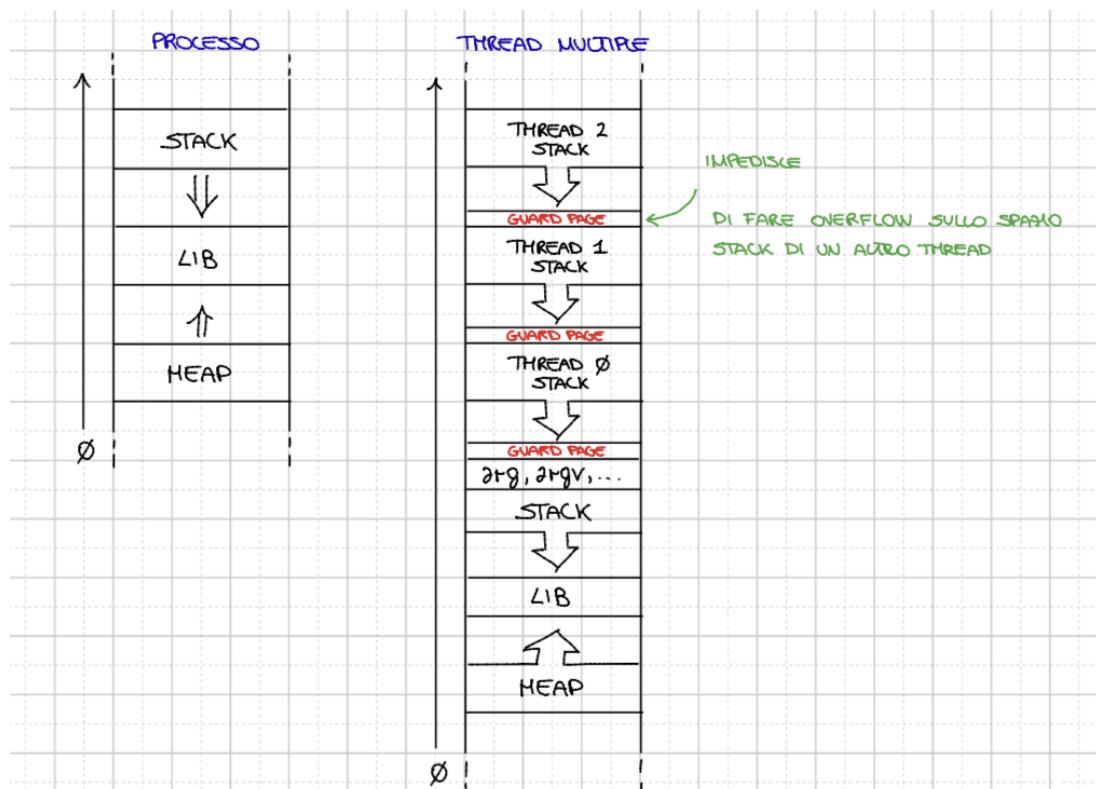


Figura 4.14: Thread

Come possiamo vedere nei thread multipli in ogni stack il sistema operativo inserisce il **guard page** ovvero uno spazio di memoria non accessibile che serve a rilevare eventuali overflow dello stack, evitando così che un thread possa sovrascrivere lo stack di un altro thread.

**ATTENZIONE:** se un thread va in **crash** a causa di un errore di segmentazione (ad esempio, accedendo a una posizione di memoria non valida), tutti i thread dello stesso processo vanno in crash, poiché condividono lo stesso spazio di memoria.

Riassumendo:

1. Un thread esiste all'interno di un processo e ne condivide le risorse;
2. Ogni thread ha un controllo del flusso di esecuzione del codice indipendentemente dagli altri;
3. Un thread duplica le risorse minime necessarie per il suo funzionamento;
4. Un thread condivide le risorse del processo con gli altri thread dello stesso processo;

## Posix Thread

I Posix Thread (Pthread) sono una specifica standard per la gestione dei thread nei sistemi operativi Unix-like.

**IMPORTANTE:** per usare i posix thread bisogna compilare con switch `"-lpthread"` per linkare la libreria pthread.

Vediamo una prima funzione di `pthread.h`:

```
#include <pthread.h>
int pthread_create(pthread_t *threadID, const pthread_attr_t *attr,
                  void *(*start_routine)(void *), void *arg);
```

Questa funzione restituisce 0 in caso di successo, altrimenti restituisce un codice di errore, mentre per quanto riguarda i parametri:

- **pthread\_t \*threadID** → puntatore a una variabile di tipo pthread che in caso di successo contiene l'identificatore del thread creato
- **const pthread\_attr\_t \*attr** → struct con cui specificare attributi del thread
- **void \*(\*start\_routine)(void \*)** → puntatore alla funzione che il thread dovrà eseguire
- **void \*arg** → eventuali parametri per la funzione eseguita nel thread

Nei thread la funzione exit non è la funzione giusta da utilizzare, ma dovremmo usare:

```
void pthread_exit(void *retval)
```

L'istruzione termina il thread corrente restituendo `retval`, ovvero il parametro che abbiamo passato. Vediamo un esempio:

```
void *PrintHello(void *threadid){
    long tid;
    tid = (long) threadid;
    printf("Hello Word! Sono il thread #%ld!\n", tid);
    pthread_exit(NULL);
}

int main(int argc, char *argv[]){
    pthread_t threads[NUM_THREADS];
    int rc;
    long t;
    for(t = 0; t < NUM_THREADS; t++){
        printf("Funzione main: creo il thread %ld\n", t);
        rc = pthread_create(&threads[t], NULL, PrintHello, (void*) t);
        pthread_detach(threads[t]);
        if(rc){
            printf("ERRORE: codice %d\n", rc);
            exit(-1);
        }
    }
    sleep(2);
}
```

Abbiamo poi:

```
int pthread_join(pthread_t threadID, void ** retval);
|               |
|               |____ puntatore a variabile in cui sarà memorizzato
|               |       il valore di ritorno
|               |
|_____ ID del thread di cui si vuole attendere
|       la terminazione
```

La funzione `pthread_join` permette a un thread di aspettare la terminazione di un altro thread esclusivamente se "joinable" al thread che li ha creati.

Un thread può essere:

- **joinable** → il thread che lo ha creato può aspettarne la fine usando `pthread_join`;
- **detached** → termina in modo indipendente e nessun altro thread può aspettarlo con `pthread_join`.

Quindi, quando un thread è **joinable**, il thread creatore può:

- fermarsi in attesa della sua terminazione;
- recuperare il valore restituito dal thread;
- liberare correttamente le risorse utilizzate.

Per rendere un thread detached, si può usare la funzione:

```
int pthread_detach(pthread_t threadID);
```

**ATTENZIONE:** se un thread è detached, non è possibile aspettarne la terminazione con `pthread_join`. Si dice che "distacca" il threadID dal thread che lo ha creato, rendendolo indipendente.

## Come la CPU esegue le istruzioni

Fino ad ora abbiamo assunto che ogni istruzione Assembly venga eseguita in modo "atomico". In realtà, all'interno della CPU, l'esecuzione di un'istruzione richiede diversi passaggi intermedi.

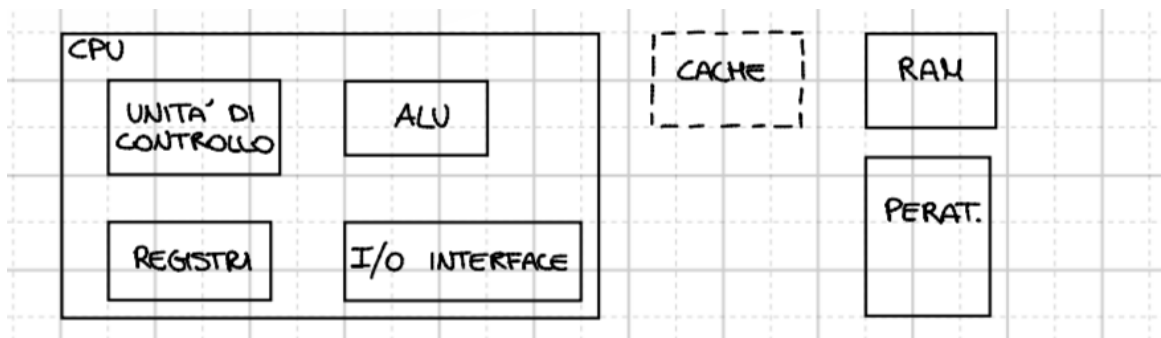


Figura 4.15: CPU

Ogni istruzione viene eseguita dalla CPU attivando, in una precisa sequenza temporale, diversi componenti elettronici interni. Ciascun componente deve essere attivato nel momento corretto affinché venga rispettata la semantica dell'istruzione.

L'attività dei vari componenti è coordinata dall'unità di controllo (*Control Unit*) e deve essere accuratamente temporizzata. L'esecuzione avviene quindi per stadi (*stages*) di durata prefissata.

Supponendo di considerare una CPU RISC semplificata, l'esecuzione di un'istruzione può essere suddivisa nelle seguenti fasi:

1. **Fetch (F)** → la CPU preleva dalla memoria la prossima istruzione da eseguire. Successivamente viene aggiornato il *Program Counter* (PC), in modo che punti all'istruzione successiva.
2. **Decode (D)** → l'istruzione appena letta viene interpretata analizzandone l'opcode. In questa fase vengono inoltre letti gli eventuali operandi immediati e i valori contenuti nei registri coinvolti.

3. **Execute (E)** → se previsto dall'istruzione, l'ALU (*Arithmetic Logic Unit*) esegue l'operazione richiesta e calcola il risultato.
4. **Memory (M)** → se necessario, viene effettuato un accesso alla memoria in lettura o scrittura.
5. **Write-Back (W)** → il risultato finale viene scritto nei registri di destinazione, se richiesto dall'istruzione.

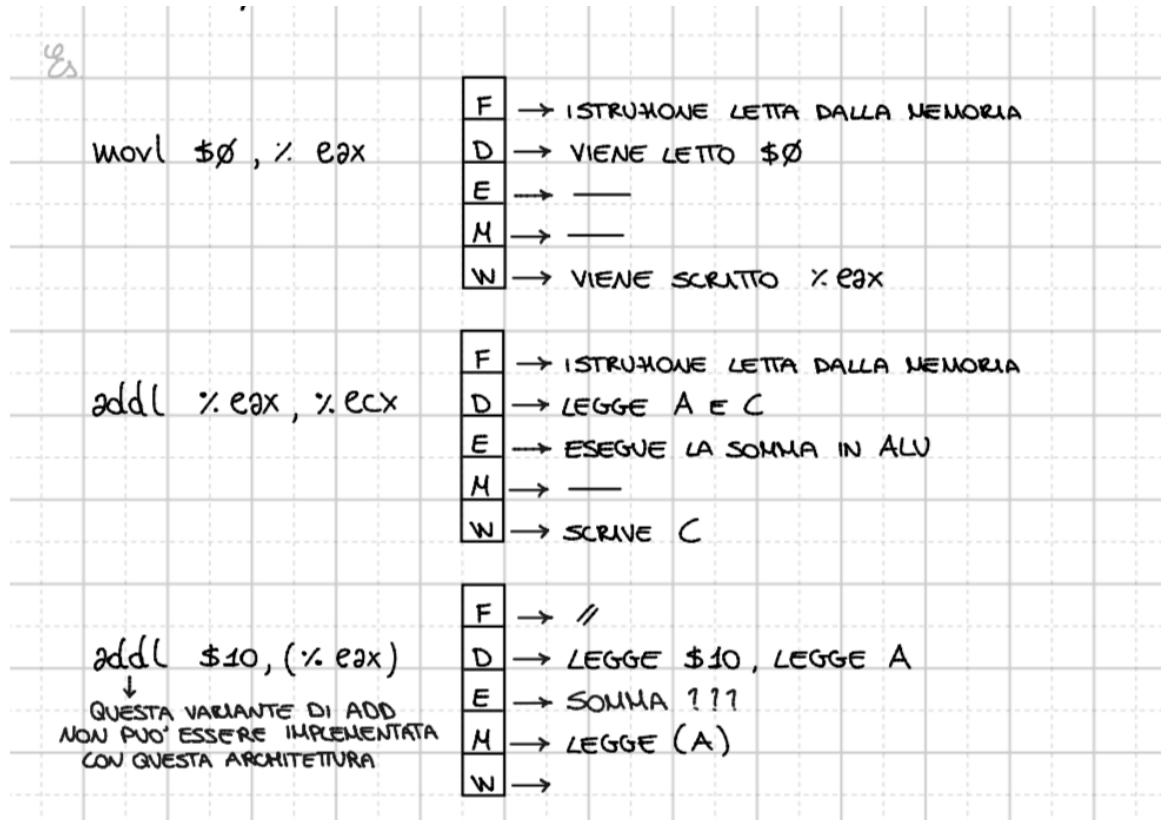


Figura 4.16: Esempio di esecuzione a stadi

Le architetture **RISC** (*Reduced Instruction Set Computer*) semplificano l'esecuzione delle istruzioni mantenendo un numero limitato di operazioni semplici.

- numero elevato di registri e istruzioni semplici;
- esecuzione più rapida delle istruzioni;
- pipeline più semplice ed efficiente;
- programmi generalmente più lunghi.

Le architetture **CISC** (*Complex Instruction Set Computer*), invece, utilizzano istruzioni più complesse e potenti.

- numero ridotto di istruzioni, ma più sofisticate;
- istruzioni spesso più lente da eseguire;
- hardware di controllo più complesso;
- programmi generalmente più compatti.

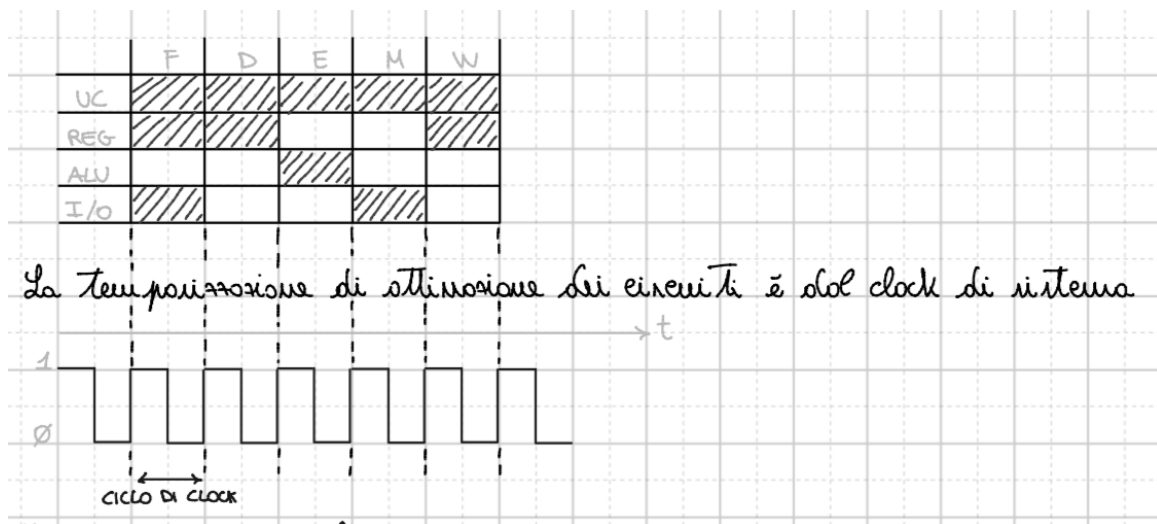


Figura 4.17: Graficamente

La temporizzazione di attivazione dei circuiti è dal clock di sistema.

### Esecuzione con Pipelining

Supponiamo di dover eseguire tre istruzioni consecutive.

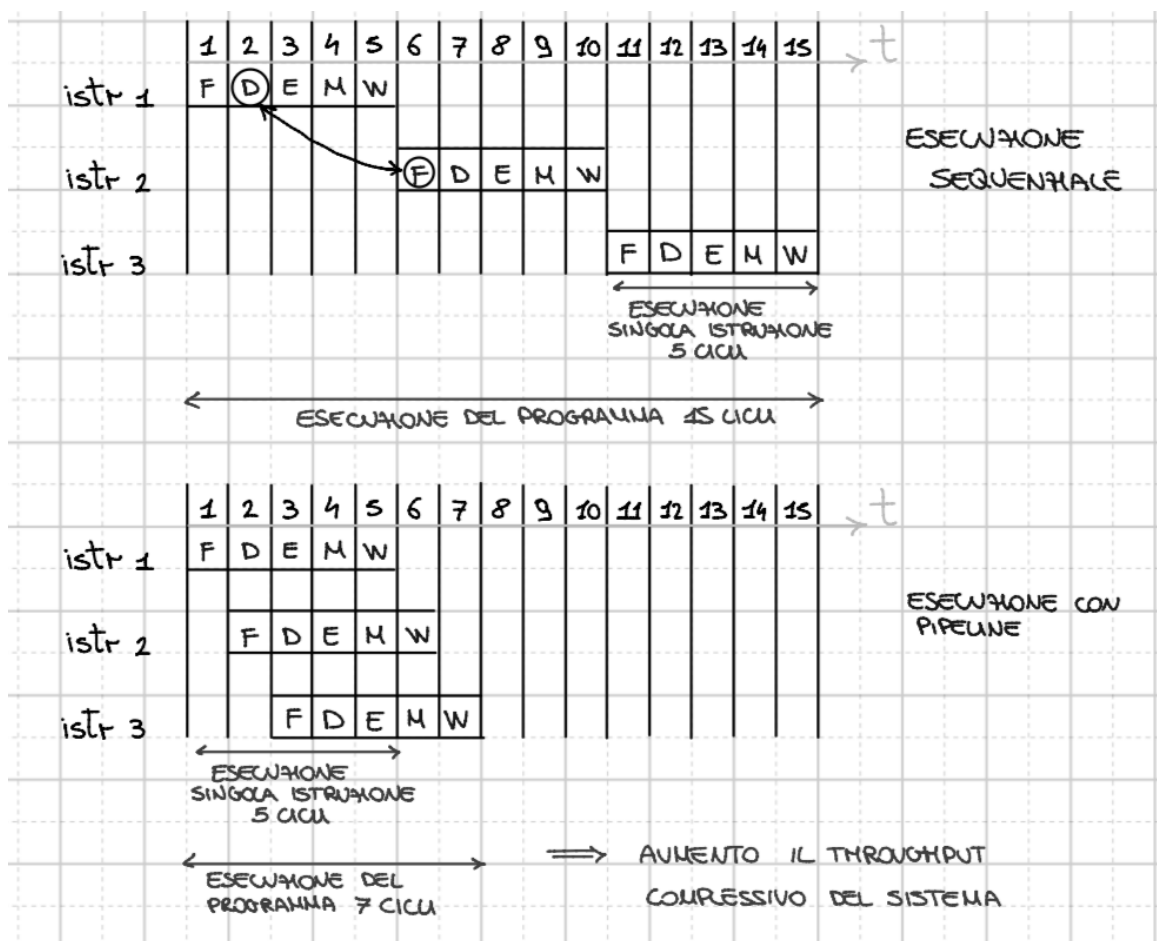


Figura 4.18: Esempio di pipeline

Nel **pipelining**, l'esecuzione delle istruzioni viene suddivisa in più stadi, che possono lavorare in parallelo su istruzioni differenti. In questo modo, mentre un'istruzione si trova in una certa fase, un'altra può trovarsi in una fase successiva o precedente.

**Attenzione:** mantenere la pipeline sempre “piena” consente di massimizzare il *throughput*, cioè il numero di istruzioni completate nell’unità di tempo.

Tuttavia, non è sempre possibile occupare tutti gli stadi della pipeline a causa della presenza degli **hazard**, ovvero situazioni che impediscono la corretta esecuzione parallela delle istruzioni.

### Hazard strutturali

Gli **hazard strutturali** si verificano quando due o più istruzioni richiedono contemporaneamente la stessa risorsa hardware della CPU.

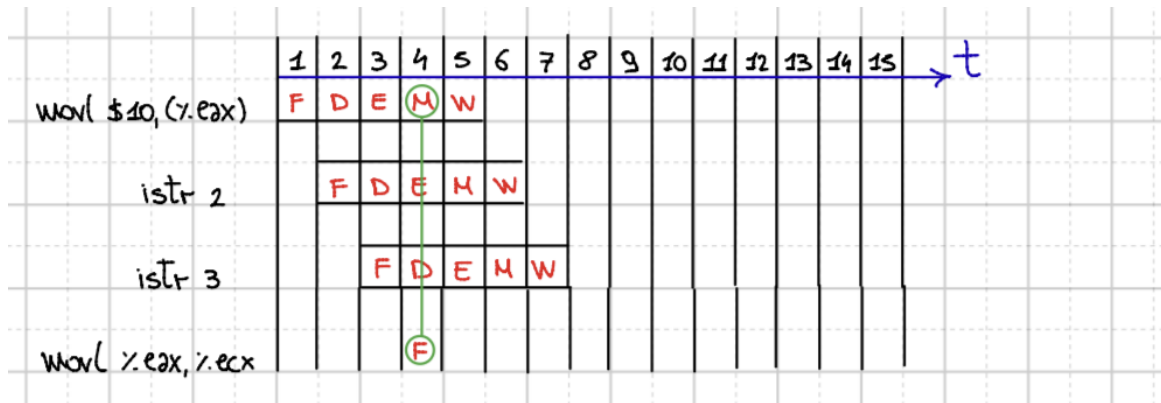


Figura 4.19: Esempio di hazard strutturale

Possibili soluzioni:

- utilizzare cache separate per dati e istruzioni;
- duplicare le risorse hardware coinvolte;
- introdurre uno **stallo** (*stall*) della pipeline.

**Tecnica dello stallo:** la CPU blocca temporaneamente la pipeline, impedendo l’ingresso di nuove istruzioni, per il numero minimo di cicli di clock necessario a risolvere l’hazard.

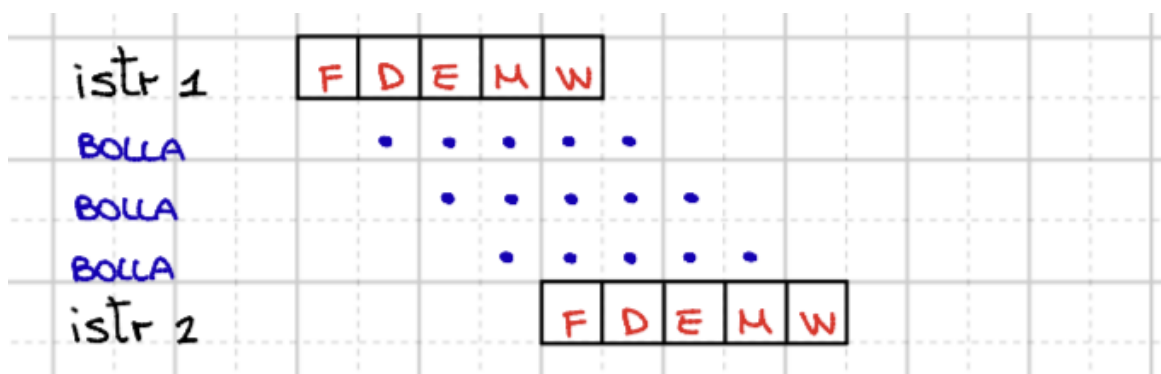


Figura 4.20: Pipeline in stallo

L’introduzione di “bolle” (*bubbles*) nella pipeline consente di risolvere qualsiasi hazard, ma comporta una riduzione del *throughput*.

### Hazard sui dati

Gli **hazard sui dati** si verificano quando esistono dipendenze tra i dati utilizzati da istruzioni differenti.

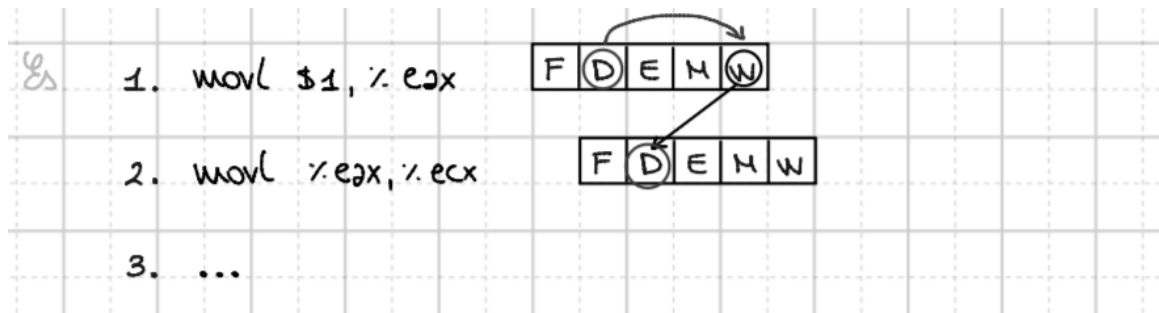


Figura 4.21: Esempio di hazard sui dati

Una possibile soluzione consiste nel **riordinamento del codice**, cioè modificare l'ordine delle istruzioni senza alterare la semantica del programma, in modo da evitare conflitti tra i dati.

|                              |               |                              |
|------------------------------|---------------|------------------------------|
| <code>movl \$4, %eax</code>  | $\Rightarrow$ | <code>movl \$4, %eax</code>  |
| <code>movl %eax, %ecx</code> |               | <code>movl \$2, %ebx</code>  |
| <code>movl \$2, %ebx</code>  |               | <code>movl \$3, %edi</code>  |
| <code>movl \$3, %edi</code>  |               | <code>movl \$4, %esi</code>  |
| <code>movl \$4, %esi</code>  |               | <code>movl %eax, %ecx</code> |

Nell'esempio precedente, l'istruzione

```
movl %eax, %ecx
```

viene spostata alla fine poiché non influenza le altre istruzioni e può quindi essere eseguita successivamente senza modificare il comportamento del programma.

### Hazard sul controllo di flusso

Gli **hazard sul controllo di flusso** si verificano quando l'esecuzione di un'istruzione dipende dal risultato di una precedente istruzione di salto condizionato.

```
testl %eax, %eax
je L1
movl %eax, %edx
L1:
.
.
.
```

In questo caso, la CPU non può sapere immediatamente quale sarà la prossima istruzione da eseguire finché la condizione del salto non viene valutata.

**Branch Prediction:** la CPU tenta di predire quale sarà il prossimo ramo di esecuzione, così da continuare a riempire la pipeline senza attendere il risultato effettivo del salto.

**Uso di codice alternativo:** in alcuni casi è possibile sostituire salti condizionati con istruzioni che non alterano il flusso di controllo, riducendo così gli hazard.

```
testl %eax, %eax
cmovl %eax, %edx
```

L'istruzione `cmovl` (*conditional move*) esegue il trasferimento solo se la condizione specificata risulta verificata, evitando un salto esplicito.

### Interruzioni

Un **interrupt** è una segnalazione generata dall'hardware oppure tramite specifiche istruzioni software, che richiede l'attenzione immediata del sistema.

La notifica di un interrupt rappresenta un evento asincrono rispetto al normale flusso di esecuzione del programma. Quando si verifica un interrupt, l'esecuzione del processo nello stato **RUNNING** viene sospesa e il processo passa tipicamente nello stato **READY**. Successivamente, il sistema operativo esegue una particolare routine di gestione dell'evento detta **interrupt handler**.

Eventi differenti vengono gestiti da handler differenti. La corrispondenza tra interrupt e relativo handler è mantenuta dal sistema operativo attraverso una struttura dati chiamata **Interrupt Vector Table (IVT)**.

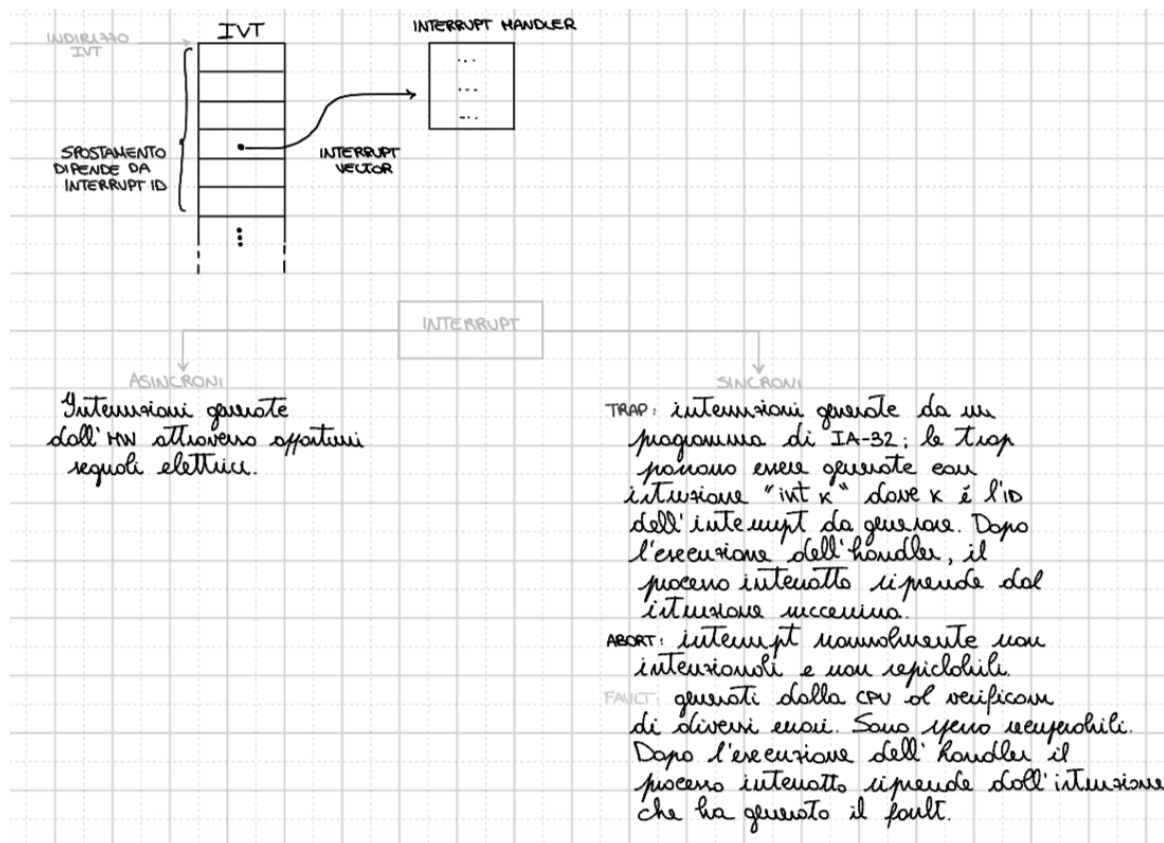


Figura 4.22: Interrupt Vector Table

### Tipologie di interrupt

- **Abort** → interrupt generalmente non intenzionale e non recuperabile. Solitamente provoca la terminazione del processo corrente.
- **Fault** → interrupt generato dalla CPU al verificarsi di errori durante l'esecuzione di un'istruzione. I fault sono spesso recuperabili.

Dopo l'esecuzione dell'handler, il processo interrotto può riprendere dall'istruzione che ha generato il fault.

## 4.5 Segnali

Un **segnale** è una notifica asincrona inviata a un processo per indicare il verificarsi di un evento specifico. I segnali vengono utilizzati dal Sistema Operativo per gestire situazioni come errori, interruzioni, eccezioni o richieste di terminazione. Gli **interrupt**, invece, sono il meccanismo con cui l'hardware comunica al Sistema Operativo la necessità di gestire una particolare condizione. Dal punto di vista concettuale, interrupt e segnali svolgono un ruolo simile: notificare il verificarsi di un evento asincrono.

| Meccanismo | Destinatario | Mittente                                | Gestore                    |
|------------|--------------|-----------------------------------------|----------------------------|
| Interrupt  | Kernel       | Periferiche, CPU, programmi             | Interrupt Handler (kernel) |
| Segnale    | Processo     | Kernel, altri processi, processo stesso | Signal Handler (processo)  |

Tabella 4.1: Confronto tra interrupt e segnali

Gli interrupt sono sempre indirizzati al kernel, unico componente responsabile della gestione dell'hardware. I segnali, invece, possono essere inviati dal Sistema Operativo a processi differenti, permettendo una gestione asincrona degli eventi a livello di processo. Ogni segnale è caratterizzato da:

- **Numero identificativo** → identifica il tipo di segnale. In Linux esistono numerosi segnali standard, generalmente numerati da 1 a 31 (con alcune eccezioni e segnali aggiuntivi real-time).
- **PID destinatario** → identifica il processo a cui il segnale è destinato.

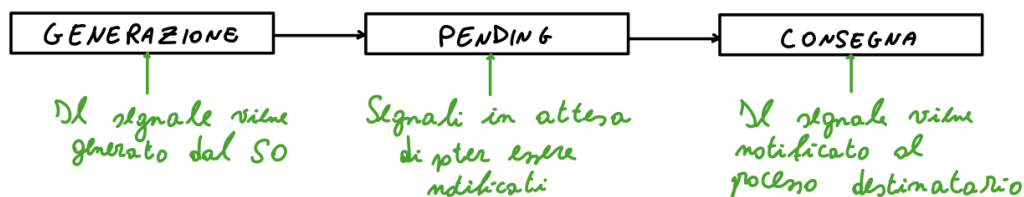


Figura 4.23: Ciclo di vita di un segnale

Il passaggio dallo stato **Pending** alla consegna del segnale avviene tipicamente:

- al ritorno da una *system call*;
- al termine della gestione di un interrupt.

Quando il kernel notifica un segnale a un processo, quest'ultimo può reagire in tre modi differenti:

1. **Term/Core** → il processo viene terminato oppure viene generato un *core dump*, cioè una copia dello stato della memoria e della CPU salvata su disco. Il *core dump* è utile per attività di debug e analisi post-mortem.
2. **Ignora** → il processo ignora il segnale e continua normalmente la propria esecuzione.
3. **Catch** → il processo intercetta il segnale ed esegue una specifica routine di gestione (*signal handler*). È possibile definire un handler diverso per ciascun segnale oppure utilizzare lo stesso handler per più segnali. Se non viene definito alcun handler personalizzato, viene utilizzata l'azione di default prevista dal sistema.

| Categoria    | Segnale | N° | Descrizione                                                                                                                                               | Ignorabile | Gestibile | Default |
|--------------|---------|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------|------------|-----------|---------|
| Terminazione | SIGTERM | 15 | Terminazione "morbida" in cui il processo ha la facoltà di liberare le proprie risorse. Inviato dal comando <code>kill</code> di default (si veda sotto). | Sì         | Sì        | Term    |
|              | SIGINT  | 2  | Come SIGTERM, ma attivato anche tramite la combinazione di tasti CTRL+C                                                                                   | Sì         | Sì        | Term    |
|              | SIGQUIT | 3  | Come SIGTERM, ma attivato anche tramite la combinazione di tasti CTRL+\                                                                                   | Sì         | Sì        | Core    |
|              | SIGKILL | 9  | Terminazione forzata a cui il processo non può opporsi e liberare le risorse acquisite.                                                                   | No         | No        | Term    |
|              | SIGCHLD | 17 | Evento terminazione di un figlio del processo ricevente                                                                                                   | Sì         | Sì        | Ign     |
| Mem.         | SIGSEGV | 11 | Accesso errato a memoria da parte del processo stesso                                                                                                     | Sì         | Sì        | Term    |
| Expr.        | SIGFPE  | 8  | Errore aritmetico (es. divisione per zero)                                                                                                                | Sì         | Sì        | Core    |

Figura 4.24: Principali segnali Linux

**Attenzione:** per ogni coppia *processo-segnale* può esistere al massimo un solo segnale nello stato di **Pending**. Se lo stesso segnale viene generato più volte prima della consegna, le notifiche aggiuntive vengono perse.

Spesso la generazione di un interrupt comporta successivamente l'invio di un segnale al processo interessato. Ad esempio:

UTENTE PREME:

CTRL-C -> interrupt tastiera  
-> interrupt handler  
-> invio SIGINT al processo attivo  
-> gestione del processo

CTRL-Z -> interrupt tastiera  
-> interrupt handler  
-> invio SIGTSTP al processo attivo  
-> gestione del processo

Timer scaduto  
-> interrupt timer  
-> interrupt handler  
-> invio SIGALRM al processo interessato  
-> gestione del processo

## Gestione dei segnali

In linguaggio C esistono diverse funzioni per inviare e gestire i segnali. Una delle principali funzioni utilizzate per inviare un segnale a un processo è `kill()`:

```
#include <signal.h>
int kill(pid_t pid,    // PID del processo destinatario
        int sig);     // ID del segnale da inviare
```

La funzione invia il segnale identificato da `sig` al processo avente PID uguale a `pid`.

Un processo può inviare segnali soltanto:

- ai propri processi figli;
- ai processi appartenenti allo stesso utente;

- oppure, nel caso del superutente (**root**), a qualsiasi processo.

L'invio di segnali può essere effettuato anche direttamente da terminale tramite il comando:

```
kill -9 12345
```

dove:

- 12345 rappresenta il PID del processo destinatario;
- -9 indica il segnale **SIGKILL**.

In generale, il numero associato al comando **kill** identifica il tipo di segnale da inviare.

Per gestire un segnale, invece, si utilizza tipicamente la funzione **sigaction()**:

```
#include <signal.h>

int sigaction(int sign,                // ID del segnale da gestire
               const struct sigaction *act, // Nuova configurazione (indica come gestire il segnale)
               struct sigaction *oldact); // Vecchia configurazione (se diverso da NULL,
                                         // permette di salvarla)
```

La funzione restituisce 0 in caso di successo, altrimenti restituisce -1 e imposta **errno**.

I parametri della funzione hanno il seguente significato:

- **sign** → identificativo del segnale da configurare;
- **act** → struttura contenente la nuova modalità di gestione del segnale;
- **oldact** → se diverso da **NULL**, permette di salvare la precedente configurazione del segnale.

La struttura **sigaction** è definita nel seguente modo:

```
struct sigaction {
    void (*sa_handler)(int);
    void (*sa_sigaction)(int, siginfo_t *, void *);
    sigset_t sa_mask;
    int sa_flags;
    void (*sa_restorer)(void);
};
```

I principali campi della struttura sono:

- **sa\_handler** → definisce il comportamento associato al segnale:
  - puntatore a una funzione *signal handler*;
  - **SIG\_IGN** per ignorare il segnale;
  - **SIG\_DFL** per utilizzare l'azione di default del sistema.
- **sa\_sigaction** → versione avanzata dell'handler che permette di ottenere informazioni aggiuntive sul segnale ricevuto.
- **sa\_mask** → insieme di segnali temporaneamente bloccati durante l'esecuzione del signal handler.
- **sa\_flags** → insieme di flag che modificano il comportamento dell'handler. Ad esempio, il flag **SA\_RESTART** permette di riavviare automaticamente alcune system call interrotte da un segnale.
- **sa\_restorer** → campo riservato all'uso interno del kernel e normalmente non utilizzato nei programmi applicativi (non utilizzato).

## 4.6 Gestione della memoria

Ogni processo vede e utilizza la memoria come se disponesse di uno spazio di indirizzamento lineare e dedicato esclusivamente a sé stesso. In realtà, la memoria fisica del sistema è condivisa tra tutti i processi attivi e deve quindi essere gestita dal Sistema Operativo.

Uno dei compiti fondamentali del Sistema Operativo consiste proprio nella gestione della memoria fisica disponibile, garantendo che più processi possano accedervi in modo concorrente, efficiente e sicuro.

In particolare, il Sistema Operativo deve:

1. tenere traccia delle locazioni di memoria fisica attualmente utilizzate;
2. gestire le nuove richieste di memoria, allocando opportunamente gli spazi liberi disponibili.

La gestione della memoria introduce numerosi problemi e aspetti progettuali complessi. Per semplicità, ci concentreremo principalmente su questi due aspetti fondamentali. Immaginiamo che nel Sistema Operativo sia presente un componente software chiamato **Allocatore di Memoria** rappresentato da una struttura dati che in ogni istante registra quali locazioni della memoria fisica sono libere/occupate.

I programmi interagiscono con l'allocatore di memoria attraverso, delle primitive di base che per semplificare possiamo immaginare siano tre:

1.  $p = \text{alloca}(n) \rightarrow$  restituisce l'indirizzo iniziale  $p$  di un blocco di memoria contiguo di dimensione pari ad almeno  $n$  byte, oppure NULL se non è possibile soddisfare la richiesta. In C questa funzione è rappresentata da `malloc(n)`.
2.  $p' = \text{ridimensiona}(p, n) \rightarrow$  ridimensiona il blocco di memoria precedentemente allocato a partire da  $p$  purché abbia una nuova dimensione di almeno  $n$  byte.  
**ATTENZIONE:** il nuovo blocco allocato potrebbe essere raggiungibile ad un nuovo indirizzo  $p'$ .
3.  $\text{dealloca}(p) \rightarrow$  libera lo spazio di memoria che era stato precedentemente allocato. In C questa funzione è rappresentata da `free(p)`;

### Frammentazione della memoria

La **frammentazione della memoria** è il fenomeno per cui la memoria disponibile risulta suddivisa in più blocchi sparsi e non contigui. Di conseguenza, può accadere che una richiesta di allocazione non venga soddisfatta anche se la quantità totale di memoria libera sarebbe sufficiente.

In altre parole, nessuno spazio libero contiguo è abbastanza grande da contenere la nuova allocazione richiesta. Esistono due principali tipi di frammentazione:

- **Frammentazione interna**  $\rightarrow$  si verifica quando all'interno di un blocco di memoria allocato rimangono porzioni inutilizzate. Questo accade tipicamente quando l'allocazione avviene in blocchi di dimensione fissa.
- **Frammentazione esterna**  $\rightarrow$  si verifica quando una richiesta di allocazione di dimensione  $n$  fallisce perché tutti i blocchi liberi disponibili hanno dimensione inferiore a  $n$ , anche se la somma totale della memoria libera sarebbe sufficiente.

**Attenzione:** la frammentazione esterna può verificarsi soltanto se l'allocatore utilizza blocchi di dimensione variabile.

Figura 4.25: Esempio di frammentazione esterna

L'allocazione della memoria è tipicamente organizzata su due livelli:

1. un primo allocatore, operante in *kernel space*, che gestisce la memoria fisica del sistema e la rende disponibile ai processi;
2. un secondo allocatore, operante in *user space*, che gestisce la memoria del singolo processo e risponde alle richieste di allocazione dinamica (ad esempio tramite `malloc()`).

## Memoria virtuale

La **memoria virtuale** è un'astrazione fornita dal Sistema Operativo che consente a ogni processo di utilizzare uno spazio di memoria logico dedicato, indipendente dalla reale disposizione della memoria fisica.

Ogni processo opera utilizzando **indirizzi logici** (o **virtuali**), come se avesse a disposizione un'intera memoria privata e continua. La traduzione tra indirizzi logici e indirizzi fisici viene effettuata automaticamente dal sistema tramite appositi meccanismi hardware e software.

Indirizzo logico  $\longleftrightarrow$  Indirizzo fisico

Dal punto di vista del processo, l'utilizzo della memoria avviene in modo trasparente: il programma non deve preoccuparsi della reale posizione dei dati nella RAM.

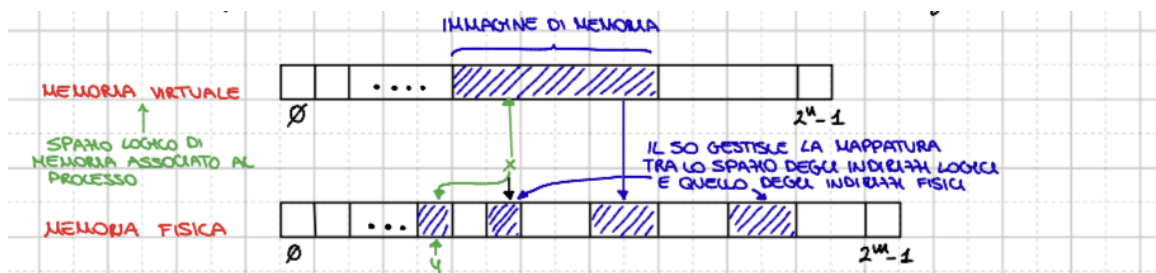


Figura 4.26: Traduzione tra memoria virtuale e memoria fisica

In sintesi, il meccanismo della memoria virtuale permette ai processi di accedere alla memoria fisica attraverso una traduzione automatica degli indirizzi virtuali in indirizzi fisici.

Questo approccio offre numerosi vantaggi:

- isolamento tra i processi;
- maggiore sicurezza;
- utilizzo più efficiente della memoria fisica;
- possibilità di eseguire processi più grandi della RAM disponibile;
- semplificazione della gestione della memoria da parte dei programmi.

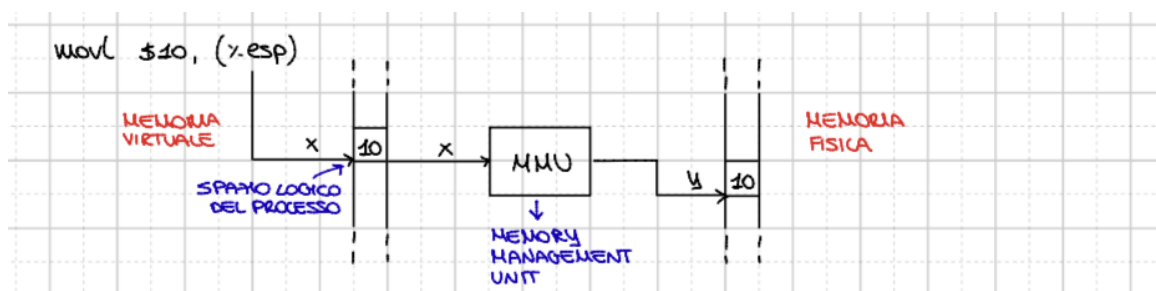


Figura 4.27: Vista logica della memoria di un processo

## Paginazione della memoria

La **paginazione** è una tecnica di gestione della memoria utilizzata nei moderni sistemi operativi per implementare la memoria virtuale.

Con la paginazione, sia lo spazio di indirizzamento logico sia la memoria fisica vengono suddivisi in blocchi di dimensione fissa:

- **Pagine** (*pages*)  $\rightarrow$  blocchi della memoria virtuale;

- **Frame** (*page frames*) → blocchi della memoria fisica.

Ogni pagina della memoria virtuale può essere caricata in un qualsiasi frame libero della memoria fisica.

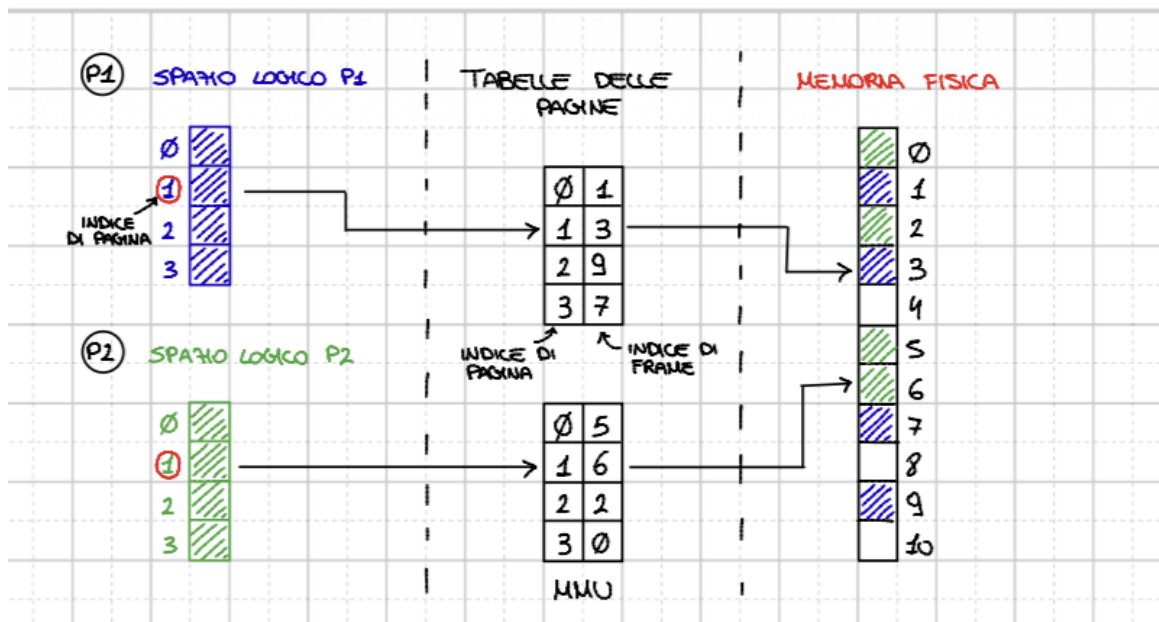


Figura 4.28: Paginazione della memoria

La traduzione degli indirizzi virtuali in indirizzi fisici viene effettuata dalla **MMU** (*Memory Management Unit*).

Supponiamo di avere:

- una memoria virtuale di dimensione  $2^v$  byte (ad esempio  $v = 32 \Rightarrow 4\text{ GB}$ );
- una memoria fisica di dimensione  $2^r$  byte (ad esempio  $r = 33 \Rightarrow 8\text{ GB}$ );
- pagine di dimensione  $2^d$  byte (ad esempio  $d = 12 \Rightarrow 4\text{ KB}$ ).

Un indirizzo virtuale di  $v$  bit viene suddiviso in due parti:

- **Numero di pagina** → identifica la pagina virtuale;
- **Offset** → identifica la posizione del dato all'interno della pagina.

Anche l'indirizzo fisico è composto da:

- **Numero di frame** → identifica il frame fisico;
- **Offset** → stessa posizione relativa all'interno del frame.

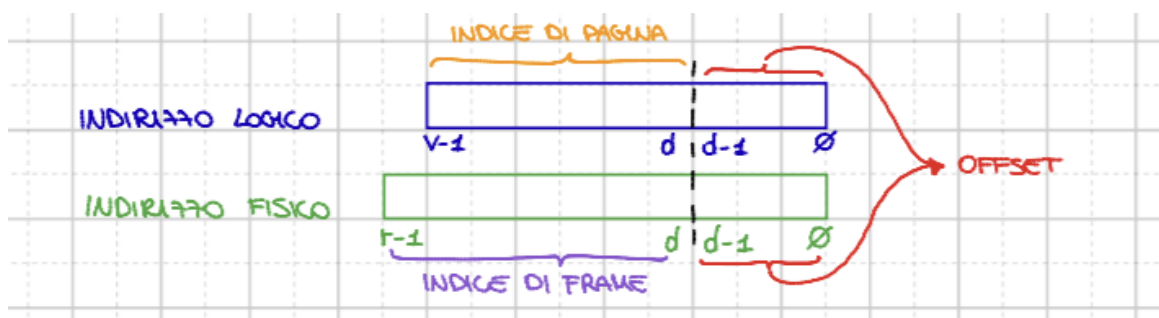


Figura 4.29: Traduzione degli indirizzi tramite MMU

La traduzione avviene nel seguente modo:

1. si estrae dall'indirizzo virtuale il numero di pagina;
2. tramite la **tabella delle pagine** si individua il frame fisico corrispondente;
3. si concatena il numero di frame con lo stesso offset presente nell'indirizzo virtuale.

L'offset rimane invariato perché pagina e frame hanno la stessa dimensione.

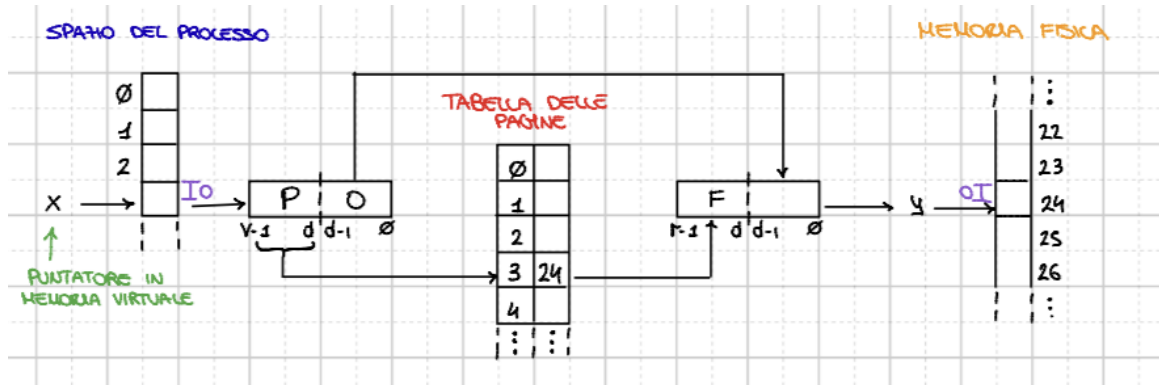


Figura 4.30: Schema di traduzione da indirizzo virtuale a indirizzo fisico

### Paginazione con bit di validità

Ad ogni riga della tabella delle pagine aggiungo un bit di validità con seguente significato:

- 0 pagina inutilizzata (non valida)
- 1 pagina in uso (valida)

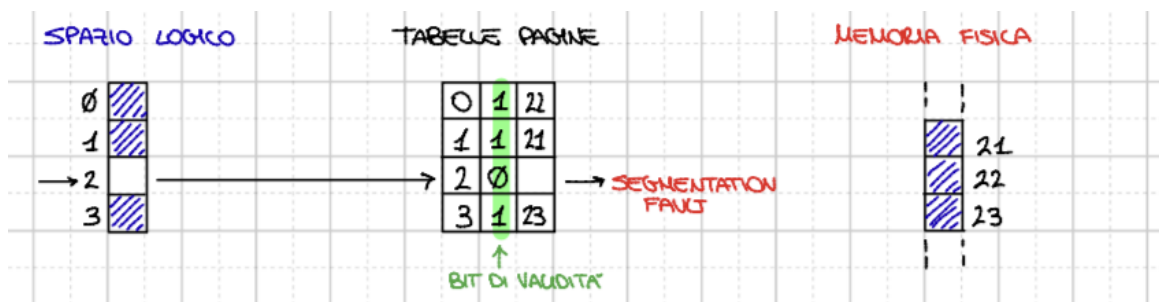


Figura 4.31: Paginazione bit validità

### Memory swapping

Il **memory swapping** è una tecnica utilizzata dal Sistema Operativo per gestire situazioni in cui la memoria fisica non è sufficiente a contenere tutte le pagine attive dei processi.

Spesso accade che alcune pagine di memoria, pur essendo valide, rimangano inutilizzate per lunghi periodi di tempo. Quando la memoria fisica è satura, il sistema può trasferire alcune di queste pagine su disco per liberare spazio: questa operazione prende il nome di **swap-out**.

Quando un processo tenta di accedere a una pagina che è stata precedentemente spostata su disco, si verifica un **page fault**. In questo caso, il Sistema Operativo deve recuperare la pagina dalla memoria secondaria e riportarla in RAM tramite un'operazione di **swap-in**, permettendo al processo di proseguire l'esecuzione.

Il funzionamento può essere riassunto nei seguenti passi:

1. la CPU genera un interrupt di tipo *page fault*;
2. il kernel seleziona un frame "vittima" nella memoria fisica;
3. il contenuto del frame vittima viene salvato su disco (*swap-out*);

4. la pagina richiesta viene caricata nel frame liberato (*swap-in*);
5. le tabelle delle pagine vengono aggiornate per riflettere la nuova mappatura;
6. il processo viene riattivato e ripete l'istruzione che aveva causato il page fault.

Questo meccanismo viene spesso indicato come **paginazione con swap e bit di validità**.

I principali vantaggi e svantaggi sono:

#### VANTAGGI:

- + Tutti i processi condividono uno spazio di indirizzamento virtuale
- + Migliore utilizzo della memoria fisica (solo le pagine attive restano in RAM)
- + Possibilità di utilizzare più memoria di quella fisicamente disponibile

#### SVANTAGGI:

- Fenomeno di thrashing (continui swap che degradano le prestazioni)
- Maggiore complessità nella gestione della memoria
- Dipendenza dalla presenza di memoria secondaria veloce (disco/SSD)

## Allocazione della memoria

Quando viene invocata la funzione `malloc(x)`, la memoria richiesta viene allocata all'interno dell'**heap** del processo.

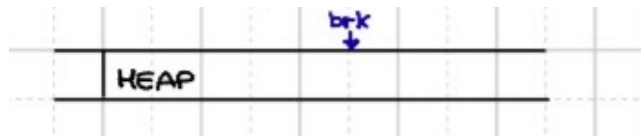


Figura 4.32: Struttura dell'heap

Il puntatore **brk** definisce il limite superiore dell'heap, ovvero la massima area attualmente allocata. Questo limite può essere modificato tramite la system call `sbrk()`.

La gestione dell'heap suddivide lo spazio disponibile in tre tipologie di aree:

1. memoria allocata e attualmente utilizzata dal processo;
2. memoria libera, disponibile per nuove allocazioni;
3. memoria occupata da strutture di gestione (metadati).

Lo spazio dell'heap è organizzato in **blocchi**, ciascuno dei quali può appartenere a una delle categorie precedenti. I blocchi sono disgiunti tra loro e contengono sempre un'intestazione (*header*) con informazioni di gestione.

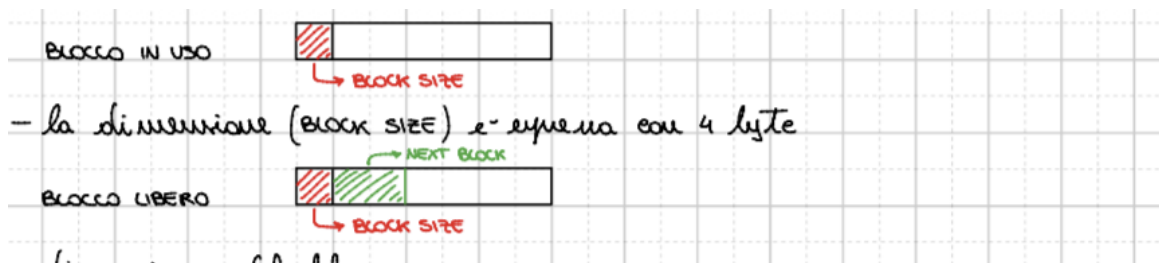


Figura 4.33: Struttura dei blocchi di memoria

I blocchi liberi sono organizzati in una struttura dati chiamata `free_list`, tipicamente implementata come lista concatenata.

```
p = malloc(n)
-> cerca nella free_list un blocco libero di dimensione S >= n + overhead
-> dimensione minima tipica di un blocco: 12 byte
```

Se viene trovato un blocco adatto, questo viene rimosso dalla `free_list`. La scelta del blocco può seguire diverse strategie:

- **First-Fit** → seleziona il primo blocco sufficientemente grande;
- **Best-Fit** → seleziona il blocco più piccolo tra quelli disponibili che soddisfano la richiesta, riducendo la frammentazione interna.

Se non è disponibile alcun blocco adeguato, l'heap viene esteso aumentando il valore di `brk` e creando un nuovo blocco di dimensione adeguata.

```
free(p)
-> inserisce il blocco puntato da (p - header) nella free_list
```

## Altri miglioramenti

- Fusione di blocchi liberi adiacenti

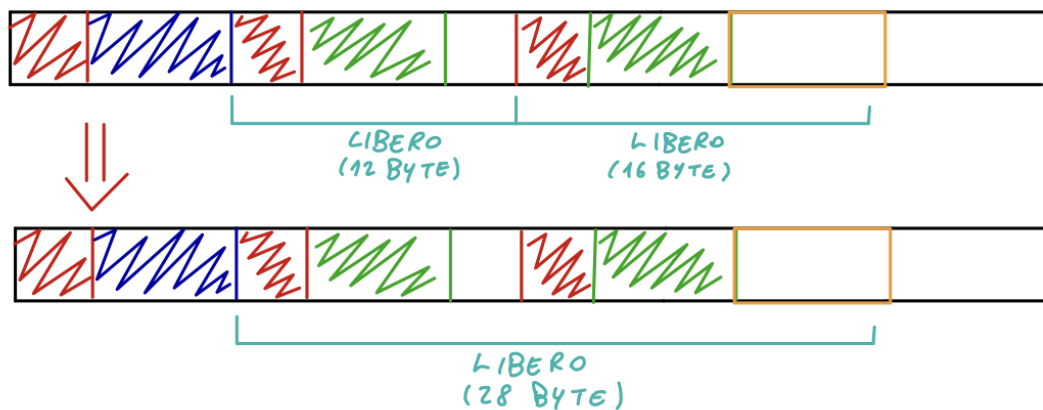


Figura 4.34: Fusioni di blocchi liberi adiacenti

- Partizionamento dei blocchi liberi

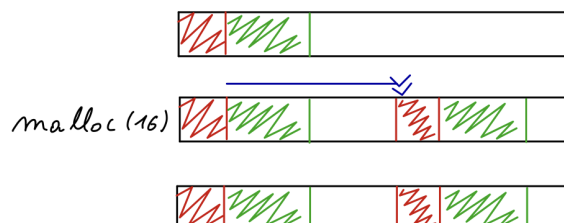


Figura 4.35: Partizionamento dei blocchi liberi

- Liste free separate per blocchi di dimensioni diverse
- Allineamento

## Cache e uso ottimizzato della memoria

Consideriamo due operazioni a livello di assembly:

```
movl $1, %eax    -> 1 ns
movl $1, (%eax)   -> 100 ns
```

Si nota come l'accesso alla memoria sia significativamente più lento rispetto alle operazioni sui registri.

La **cache** è una memoria ad alta velocità che conserva temporaneamente i dati utilizzati più frequentemente, riducendo il tempo medio di accesso alla memoria principale.

Essa è organizzata in **linee (cache lines)** di dimensione fissa (ad esempio 64 byte). Ogni linea contiene una copia di una porzione della memoria principale.

Ogni indirizzo di memoria  $X$  è associato a un blocco di cache identificato come:

$$\left\lfloor \frac{X}{L} \right\rfloor$$

dove  $L$  rappresenta la dimensione della cache line.

Quando si accede a un indirizzo di memoria, possono verificarsi due casi:

- **Cache hit** → il dato è già presente in cache e può essere letto rapidamente;
- **Cache miss** → il dato non è presente in cache e deve essere caricato dalla memoria principale.

Se la cache è piena, è necessario sostituire una linea esistente secondo una politica di rimpiazzo.

La progettazione della cache si basa sui principi di località:

- **Località temporale** → se un dato viene acceduto, è probabile che venga riutilizzato nel breve periodo;
- **Località spaziale** → se un indirizzo viene acceduto, è probabile che vengano acceduti anche indirizzi vicini.

### Esempio:

```
int sum(int **m, int n){
    int i, j, s = 0;

    for(i = 0; i < n; i++)
        for(j = 0; j < n; j++)
            s = s + m[i][j];

    return s;
}
```

Questa versione sfrutta la località spaziale (accesso per righe) ed è quindi più efficiente.

Se invece si accede agli elementi in ordine trasposto:

```
m[j][i]
```

si perde la località spaziale, causando molti più cache miss e un aumento significativo del tempo di esecuzione (ad esempio da 0.3s a 1.5s).

## 4.7 Cache e associatività

La **cache** è una memoria ad accesso rapido organizzata in blocchi di dimensione fissa, chiamati **linee di cache** (*cache lines*). Ogni linea contiene una copia di una porzione della memoria principale, permettendo di ridurre significativamente i tempi di accesso ai dati più utilizzati.

La cache non è gestita direttamente dai programmi applicativi, ma dal sistema hardware e dal Sistema Operativo. Ogni linea di cache è associata a un **tag**, utilizzato per identificare quale blocco di memoria principale è attualmente memorizzato nella linea.

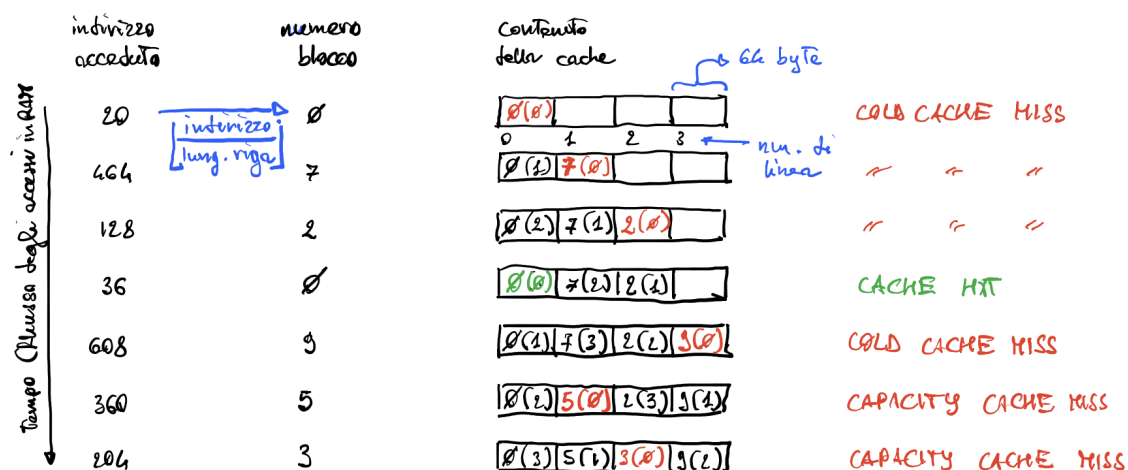


Figura 4.36: Esempio di organizzazione della cache

### Tipologie di cache

Esistono tre principali modalità di organizzazione della cache.

- **Cache completamente associativa** (*fully associative*) → ogni blocco di memoria può essere memorizzato in qualsiasi linea della cache.

#### VANTAGGI

- + utilizzo molto efficiente dello spazio in cache
- + riduzione dei conflict miss

#### SVANTAGGI

- implementazione complessa
- costo hardware elevato

- **Cache associativa a k-vie** (*k-way set associative*) → ogni blocco di memoria può essere memorizzato solo in un insieme (*set*) specifico di *k* linee, determinato tramite una funzione di mapping.

Questo approccio rappresenta un compromesso tra:

- efficienza della cache completamente associativa;
- semplicità della cache a mappatura diretta.

- **Cache a mappatura diretta** (*direct mapped*) → ogni blocco di memoria può essere memorizzato in una sola linea specifica della cache.

#### VANTAGGI

- + implementazione semplice
- + costo ridotto
- + accesso molto veloce

#### SVANTAGGI

- maggiore probabilità di cache miss
- utilizzo meno efficiente della cache

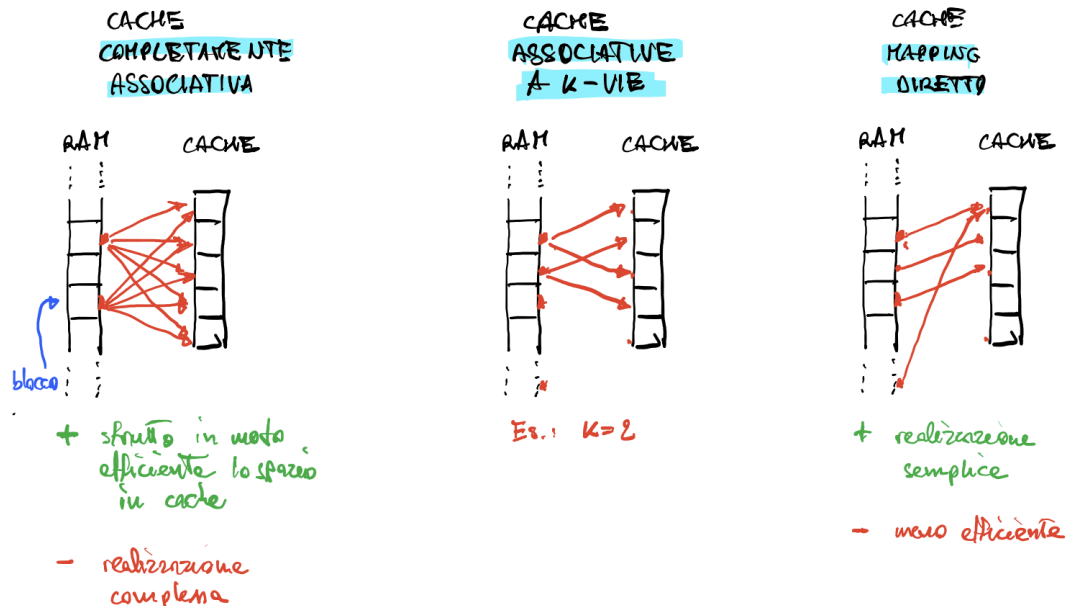


Figura 4.37: Confronto tra i diversi tipi di cache

### Tipologie di cache miss

A seconda dell'organizzazione della cache e dello stato corrente della memoria, durante un accesso possono verificarsi differenti situazioni.

- **Cache hit** → il dato richiesto è già presente nella cache e può essere letto rapidamente.
- **Cache miss** → il dato richiesto non è presente nella cache e deve essere caricato dalla memoria principale.
- **Capacity miss** → la cache è piena e il caricamento del nuovo dato richiede la sostituzione di una linea esistente (linea vittima).
- **Conflict miss** → il dato non può essere inserito perché la linea di cache assegnata è già occupata da un altro blocco incompatibile.

Questo fenomeno si verifica principalmente nelle cache a mappatura diretta o nelle cache associative con pochi modi.

## 4.8 File System

Il **File System** è l'insieme delle strutture organizzative e delle funzionalità attraverso cui il Sistema Operativo gestisce i dati memorizzati nei dispositivi di archiviazione. In altre parole, rappresenta il livello software che consente al Sistema Operativo di interfacciarsi con le periferiche di memoria (dischi, SSD, chiavette USB, ecc.) organizzando i dati in modo strutturato ed efficiente.

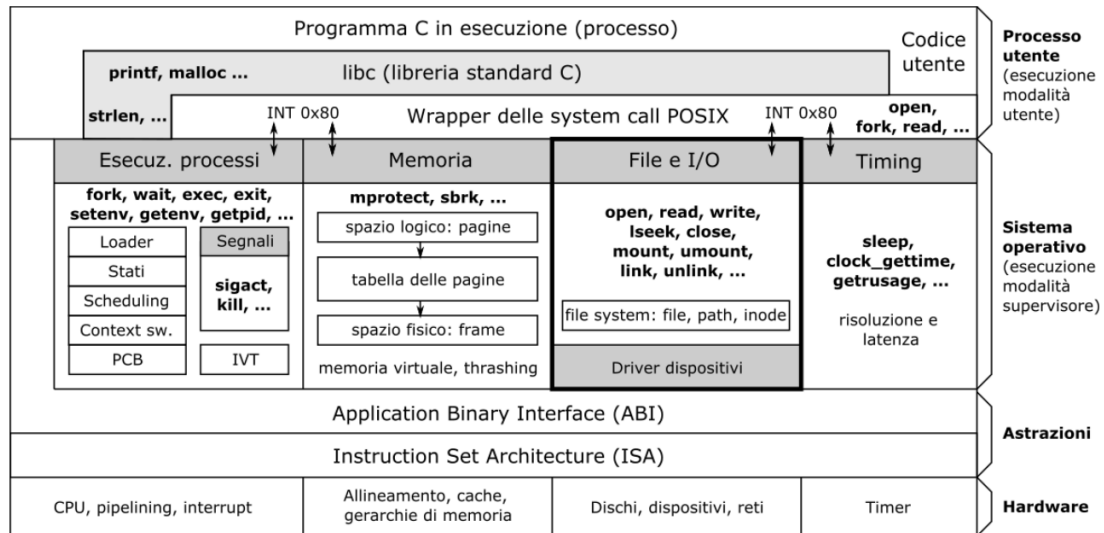


Figura 4.38: Architettura generale del File System

Analizziamo ora i principali elementi che compongono un file system.

### File e Directory

#### File

Un **file** è un contenitore di dati, generalmente rappresentati come una sequenza ordinata di byte. Oltre al contenuto vero e proprio, ogni file possiede una serie di **metadati**, utilizzati dal Sistema Operativo per gestirlo correttamente.

Tra i principali metadati troviamo:

- **Nome** del file;
- **Dimensione** in byte;
- **Permessi di accesso**;
- **Timestamp** (creazione, modifica, ultimo accesso).

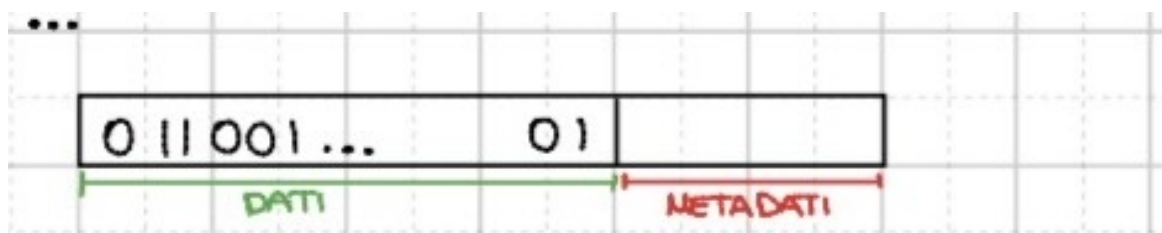


Figura 4.39: Esempio di file con metadati

## Directory

Una **directory** è un contenitore logico utilizzato per organizzare file e altre directory.

Le directory permettono di strutturare i dati secondo una gerarchia, tipicamente ad albero, facilitando la navigazione e la gestione delle informazioni all'interno del file system.



Figura 4.40: Organizzazione gerarchica delle directory

## Struttura gerarchica dei File System

Nei sistemi UNIX/Linux, il contenuto del file system è organizzato secondo una struttura gerarchica ad albero. Alla base di questa struttura si trova una directory speciale chiamata **root directory**, identificata dal simbolo:

/

Tutti i file e le directory presenti nel sistema discendono da questa directory principale.

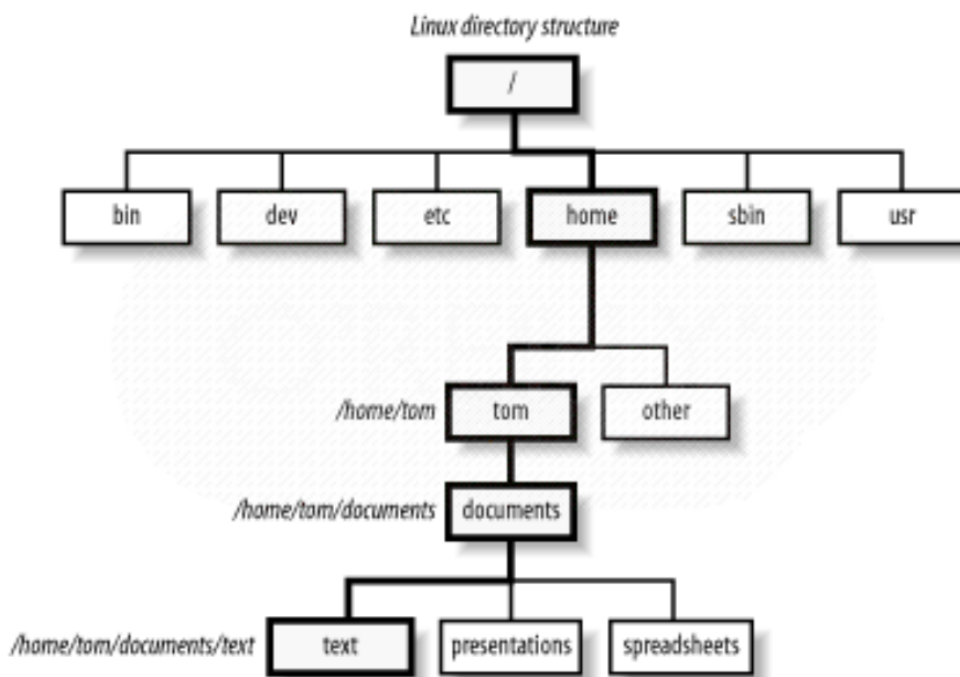


Figura 4.41: Struttura gerarchica del file system UNIX

Ogni directory, ad eccezione della root directory, possiede una **directory genitore**.

Se una directory è contenuta all'interno di un'altra directory, allora viene detta **directory figlia**.

Ad esempio, nella figura:

- tom e other sono directory figlie di home;
- home è la directory genitore di tom e other.

I file system UNIX-like sono generalmente **case-sensitive**: lettere maiuscole e minuscole vengono considerate differenti.

Ad esempio:

```
/home/tom/documents/text
```

e

```
/home/tom/documents/Text
```

identificano due file distinti.

Il **Filesystem Hierarchy Standard (FHS)** è uno standard che definisce l'organizzazione e il significato delle principali directory nei sistemi UNIX/Linux, garantendo uniformità tra le diverse distribuzioni.

## Path / Percorso

Nel file system, ogni file o directory è identificato univocamente tramite un **path** (percorso).

Esistono due tipologie principali di path:

- **Path assoluto** → rappresenta il percorso completo che parte dalla root directory e arriva fino al file o directory desiderato.

```
/home/Tom/Documents/Text
```

In questo esempio:

- **Text** è il file/directory identificato;
- **home**, **Tom** e **Documents** rappresentano le directory attraversate;
- il simbolo iniziale **/** identifica la root directory;
- gli altri caratteri **/** fungono da separatori.

### Attenzione:

```
/      -> path valido  
//     -> generalmente non considerato un path standard
```

- **Path relativo** → rappresenta il percorso rispetto a una directory di riferimento (tipicamente la directory corrente).

Per costruire path relativi si utilizzano alcuni riferimenti speciali:

- **.** → directory corrente;
- **..** → directory padre;
- **~** → home directory dell'utente corrente.

## Esempio

Supponiamo di trovarci nella directory:

```
/home/tom/documents
```

Per raggiungere la directory **other**:

```
../../other
```

Il percorso:

- torna indietro di due livelli tramite **..**;
- entra successivamente nella directory **other**.

## Organizzazione interna del File System

Per descrivere l'organizzazione interna di un file system consideriamo i file system della famiglia **ext** (ad esempio **ext2**, **ext3**, **ext4**), tipici dei sistemi UNIX/Linux.

In questi file system, ogni file o directory è associato a una struttura dati chiamata **inode** (*index node*).

L'inode contiene tutte le informazioni necessarie alla gestione del file, ad eccezione del nome, ed è caricato e gestito dal Sistema Operativo.

I dati effettivi del file non sono memorizzati direttamente nell'inode, ma in blocchi separati di dimensione fissa presenti sul disco.

### Contenuto di un inode

Un inode contiene principalmente metadati relativi al file:

- **Numero inode** → identificatore univoco nella tabella degli inode;
- **Tipo di oggetto** → file regolare, directory, link simbolico, ecc.;
- **Permessi di accesso**;
- **Numero di hard link**;
- **User ID (UID)** → identificativo del proprietario;
- **Group ID (GID)** → identificativo del gruppo proprietario;
- **Dimensione del file**;
- **Timestamp**:
  - ultimo accesso;
  - ultima modifica del contenuto;
  - ultima modifica dell'inode;
  - creazione del file.

Queste informazioni permettono al Sistema Operativo di gestire il file e ricostruirne la cronologia delle operazioni effettuate.

### Permessi di accesso

I permessi definiscono quali operazioni possono essere effettuate sul file. Per ciascuna categoria di utenti sono disponibili tre diritti:

- **R** (*Read*) → lettura;
- **W** (*Write*) → scrittura;
- **X** (*Execute*) → esecuzione.

I permessi vengono specificati per:

1. utente proprietario;
2. gruppo proprietario;
3. tutti gli altri utenti.

```
RWX RWX RWX
|   |   |
|   |   +---> altri utenti
|   +-----> gruppo
+-----> proprietario
```

Esempio:

```
r-x r-x r-x
```

significa che:

- il proprietario può leggere ed eseguire;
- il gruppo può leggere ed eseguire;
- tutti gli altri utenti possono leggere ed eseguire.

Per modificare i permessi si utilizza il comando:

```
chmod
```

mentre per modificare proprietario o gruppo si utilizza:

```
chown
```

Esempi:

```
chmod a+r text.txt
```

```
chmod g-x file.txt
```

### Notazione ottale dei permessi

I permessi possono essere rappresentati anche in forma ottale (base 8):

| Permesso | Valore |
|----------|--------|
| R        | 4      |
| W        | 2      |
| X        | 1      |

Ad esempio:

```
755
```

corrisponde a:

```
rwX r-x r-x
```

Quando un file viene creato da un programma, i permessi iniziali possono essere specificati tramite la system call:

```
open()
```

## Puntatori ai dati

L'inode contiene anche i puntatori ai blocchi dati del file.

Tipicamente sono presenti:

- 12 puntatori diretti;
- 1 puntatore indiretto singolo;
- 1 puntatore indiretto doppio.

I puntatori diretti referenziano direttamente i blocchi contenenti i dati.

I puntatori indiretti, invece, referenziano blocchi che contengono ulteriori indirizzi di blocchi dati.

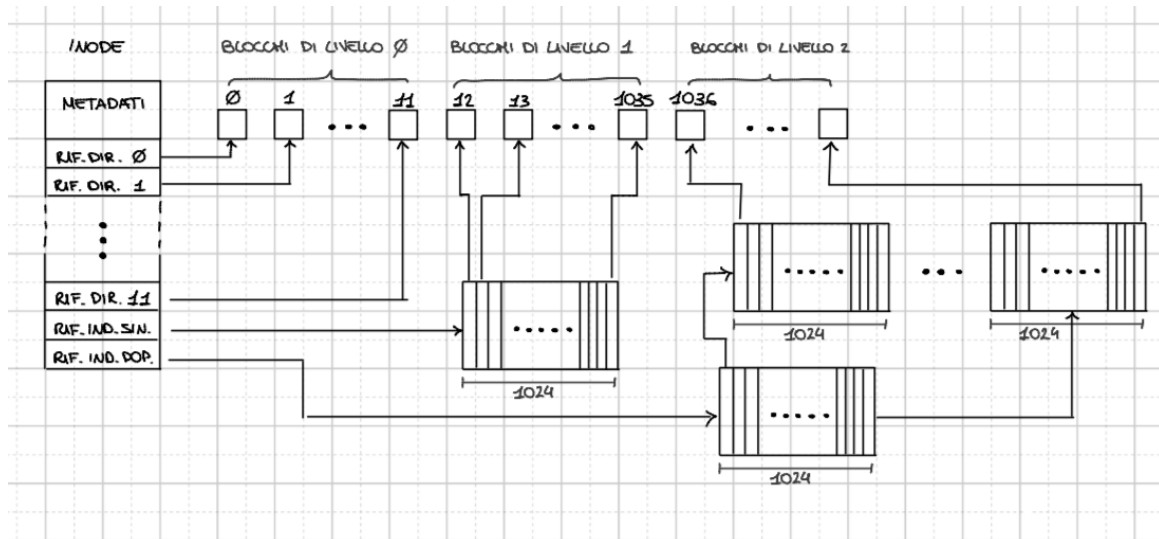


Figura 4.42: Schema di un inode

Assumendo blocchi da 4 KB:

Dimensione massima file =  
 $12 * 4096$   
 $+ 1024 * 4096$   
 $+ 1024^2 * 4096$   
 $= 4 \text{ GB}$

Lo spazio occupato dagli indici è:

$(12 + 2) * 4 + 4096 + (1024 + 1) * 4096$   
 $= 4 \text{ MB}$

## Directory

Anche le directory sono rappresentate tramite inode. I dati contenuti in una directory consistono tipicamente in associazioni:

nome file → numero inode

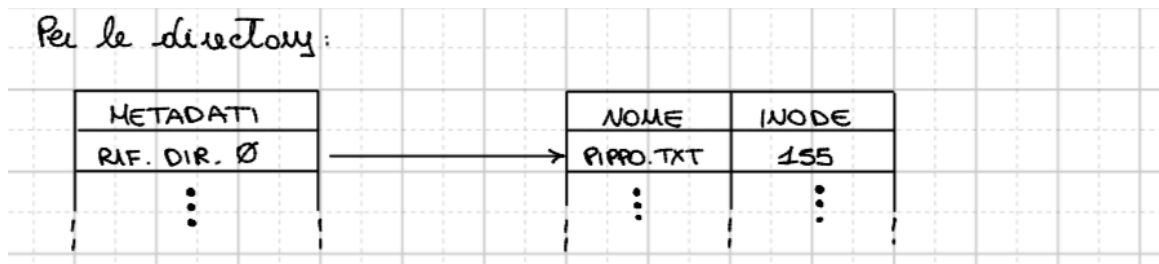


Figura 4.43: Struttura interna di una directory

## Esempi di occupazione su disco

### File da 265 byte

Occupi 1 blocco  $\rightarrow$  4 KB

### File da 16000 byte

$16000 / 4096 = 4$  blocchi

Occupi quindi 16 KB

### File da 4 MB

4 MB = 4194304 byte

$4194304 / 4096 = 1024$  blocchi

12 blocchi  $\rightarrow$  indirizzati direttamente

1012 blocchi  $\rightarrow$  tramite indirizzamento indiretto singolo

+ 1 blocco aggiuntivo per contenere i puntatori indiretti

Totale spazio occupato:

$(12 + 1012 + 1) * 4096$

## Tabella dei descrittori

Per ogni processo, il sistema operativo mantiene una **tabella dei descrittori** (File Descriptor Table), contenente l'elenco dei file e degli stream di I/O aperti dal processo.

Ogni voce della tabella associa un **file descriptor** (intero non negativo) a una struttura del kernel che rappresenta un file aperto, includendo:

- posizione corrente nel file;
- modalità di accesso (lettura, scrittura, append, ...);
- riferimento all'inode del file.

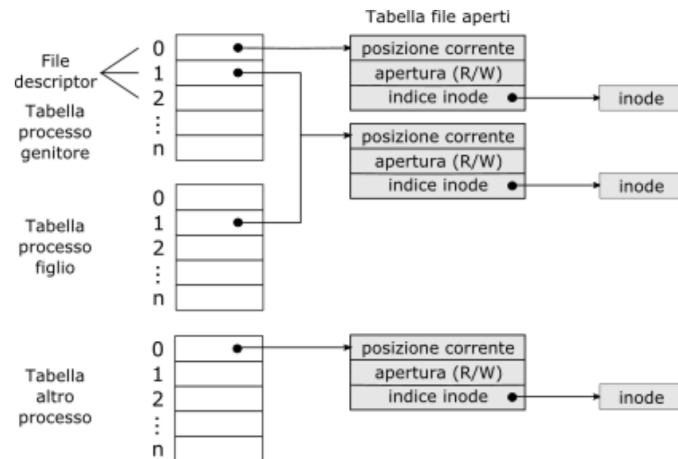


Figura 4.44: Tabella dei descrittori dei file aperti

Esempio:

```
x = fopen("a.txt");

if (fork() == 0)
    y = fopen("b.txt");
else
    z = fopen("c.txt");
```

Dopo la chiamata a `fork()`, il processo figlio eredita una copia della tabella dei descrittori del padre. Inizialmente, quindi, padre e figlio condividono gli stessi file aperti.

**ATTENZIONE:** in presenza di thread multipli, la tabella dei descrittori è condivisa tra tutti i thread dello stesso processo.

## Gestione (Mount) dei Dispositivi

L'operazione di `mount` rende accessibile il file system di un dispositivo all'interno della struttura gerarchica unica del file system UNIX.

In pratica, il contenuto del dispositivo viene associato a una directory detta **punto di mount** (*mount point*).

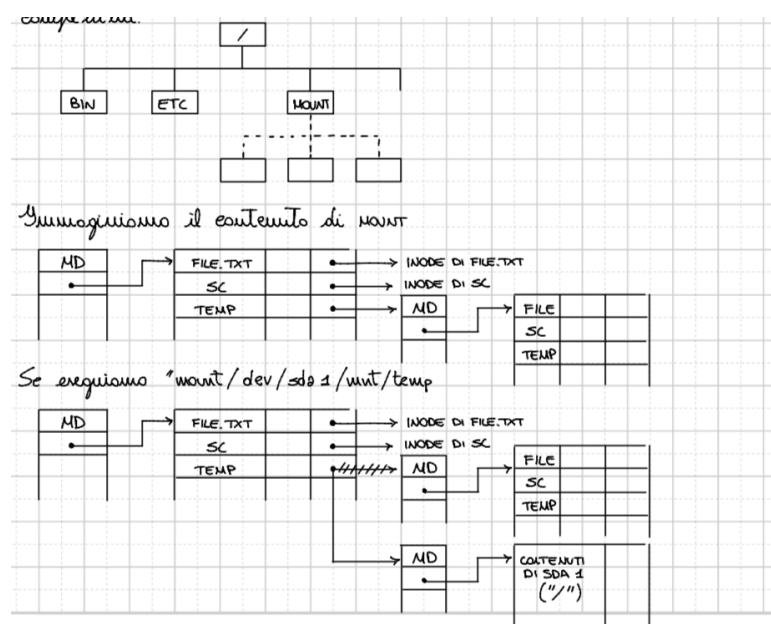


Figura 4.45: Operazione di mount

Esempio:

```
/dev/sdb1 --> montato su --> /mnt/usb
```

Dopo il mount, i file del dispositivo risultano accessibili tramite il path:

```
/mnt/usb/...
```

## Link

Un **link** è un collegamento a un file o a una directory. Ne esistono due tipologie principali:

- **Link simbolico (symbolic link o symlink)** → è un file speciale che contiene il path del file o della directory di destinazione.

VANTAGGI

- + portabile
- + può puntare a directory
- + può collegare file system differenti

SVANTAGGI

- si rompe se il file originale viene spostato o eliminato

- **Hard link** → è un nuovo riferimento diretto alla stessa inode del file originale.

VANTAGGI

- + robusto rispetto a rinomina o spostamento del file
- + accesso diretto alla stessa inode

SVANTAGGI

- non può attraversare file system differenti
- normalmente non può riferirsi a directory

**ATTENZIONE:** un file viene realmente eliminato solo quando:

- tutti gli hard link sono stati rimossi;
- nessun processo mantiene il file aperto.

## Variabili di ambiente

Le **variabili di ambiente** sono coppie **nome=valore** mantenute dalla shell e utilizzate dai processi per configurare il proprio comportamento.

- **env** → stampa le variabili di ambiente definite;
- **export NOME=valore** → crea o modifica una variabile di ambiente;
- **unset NOME** → elimina una variabile di ambiente.

Esempio:

```
export PATH=/usr/bin
echo $PATH
unset PATH
```

## 4.9 Domande d'esame

### Domanda 2 (cache)

Si consideri una cache associativa a 2 vie con 4 linee da 32 byte ciascuna e politica di rimpiazzo LRU, inizialmente vuota. Potendo scegliere fra più linee vuote, si usa la linea con indice più basso. Si ha inoltre un processo che accede in sequenza ai seguenti indirizzi di memoria (senza interruzioni): 105, 782, 384, 1356, 11, 324, 265.

Alla fine della sequenza di accessi, quali sono gli indici dei blocchi contenuti nelle 4 linee di cache? Il trattino indica che la linea di cache rimane vuota.

|   |              |   |             |
|---|--------------|---|-------------|
| A | 10, 8, 3, -  | B | 10, 8, -, - |
| C | 0, 10, 8, 42 | D | 3, -, 10, 8 |

Motivare la risposta nel file M2.txt. Risposte non motivate saranno considerate nulle.

Figura 4.46: Domande 1

Es.: 25/7/2019 domanda 2

$k=2$  line=4 dim=32 byte

| ADDR | BLOCK | CACHE |       |      |   |          |
|------|-------|-------|-------|------|---|----------|
|      |       | ①     | ②     | ③    | ④ |          |
| 105  | 3     | -     | -     | 3(0) | - | COLD     |
| 782  | 24    | 24(0) | -     | 3(1) | - | //       |
| 384  | 12    | 24(1) | 12(0) | 3(2) | - | //       |
| 1356 | 42    | 42(0) | 12(1) | 3(3) | - | CONFLICT |
| 11   | 0     | 42(1) | 0(0)  | 3(4) | - | //       |
| 324  | 10    | 10(0) | 0(1)  | 3(5) | - | //       |
| 265  | 8     | 10(1) | 8(0)  | 3(6) | - | //       |

risultato: 10, 8, 3, -

Figura 4.47: Svolgimento 1

### Domanda 1 (ottimizzazioni)

Supponiamo di essere in grado di ridurre di un fattore 3.5x il tempo di esecuzione di una porzione di un programma mediante delle ottimizzazioni. Che percentuale del tempo complessivo deve prendere quella porzione per ottenere uno speedup complessivo per il programma pari a 1.4x?

|   |     |   |     |
|---|-----|---|-----|
| A | 25% | B | 33% |
| C | 40% | D | 50% |

Motivare la risposta nel file M1.txt. Risposte non motivate saranno considerate nulle.

Figura 4.48: Domande 2

Es. 25/7/2019 domanda 1

$$A \rightarrow K = 3,5x$$

$$P \rightarrow S = 1,4x$$

$$S = \frac{1}{\frac{\alpha}{K} + 1 - \alpha}$$

$$1,4 = \frac{1}{\frac{\alpha}{3,5} + 1 - \alpha} \Rightarrow \frac{3,5}{\alpha + 3,5 - 3,5\alpha} = \frac{3,5}{3,5 - 2,5\alpha}$$

$$2,5\alpha \approx 3,5 - \frac{3,5}{1,4} \approx 0,4 \rightarrow \alpha \approx 40\%$$

Figura 4.49: Svolgimento 2

### Domanda 3 (processi)

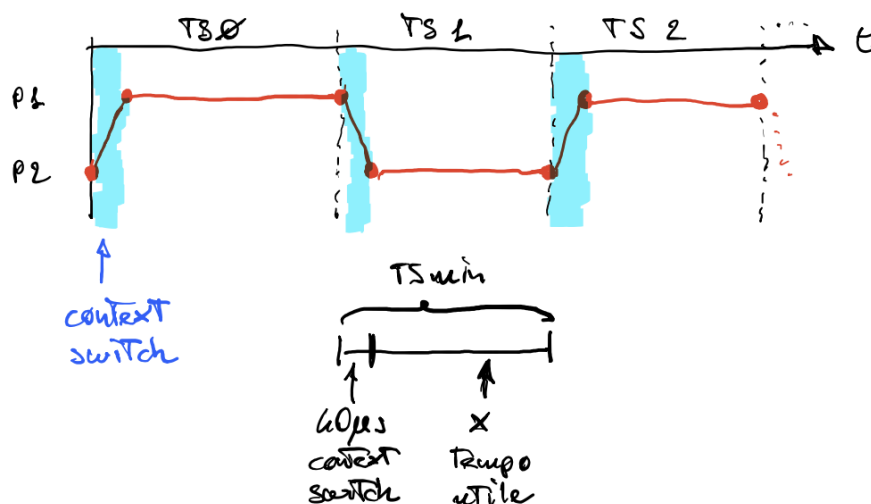
Un sistema operativo time-sharing è progettato per dare l'illusione all'utente che più di un processo progredisca nella computazione sullo stesso core allo stesso tempo. Si supponga che la gestione di un context switch richieda 40 usec (microsecondi). Qual è la minima durata di time slice affinché il tempo speso nei context switch non superi il 2% del tempo totale impiegato dal sistema per eseguire processi in time-sharing?

|   |           |   |           |
|---|-----------|---|-----------|
| A | 4 msec    | B | 1.96 msec |
| C | 2.44 msec | D | 800 usec  |

Motivare la risposta nel file M3.txt. Risposte non motivate saranno considerate nulle.

Figura 4.50: Domande 3

Es. 25/7/2019 domanda 3



$$40 = 0,02 * TS_{min} = 0,02 * (40 + x)$$

$$x = \frac{40}{0,02} - 40 = 1,96 \text{ msec}$$

Figura 4.51: Svolgimento 3

#### Domanda 4 (paginazione)

Si consideri un sistema di calcolo con spazio logico dei processi a 20 bit. Quanto deve essere grande una pagina affinché la tabella delle pagine occupi 2 KB? Si assuma che le entry della tabella delle pagine siano grandi ciascuna 32 bit.

|   |      |   |      |
|---|------|---|------|
| A | 1 KB | B | 2 KB |
| C | 4 KB | D | 8 KB |

Motivare la risposta nel file M4.txt. Risposte non motivate saranno considerate nulle.

Figura 4.52: Domande 4

ES. 25/7/2019 domanda 4

spazio indirizzabile dei processi  $\rightarrow 2^{20}$

Tabella delle pagine  $\rightarrow 2\text{KB} = 2048\text{ byte} = 2^{11}$

ogni entry  $\rightarrow 32\text{ bit} = 4\text{ byte} = 2^2$

numero righe nella Tabella =  $\frac{\text{dim. Tabella}}{\text{dim. entry}} = \frac{2^{11}}{2^2} = 2^9$

dimensione pagina =  $\frac{\text{dim. RAM}}{\text{num. pagine}} = \frac{2^{20}}{2^9} = 2^{11} = 2\text{KB}$

ATTENZIONE

~~2KB~~  $\neq$  2048 byte

| POTENZE DI 10 |          |    |
|---------------|----------|----|
| $10^3$ byte   | kilobyte | KB |
| $10^6$ =      | megabyte | MB |
| $10^9$ =      | giga ... | GB |
| $10^{12}$ =   | Tera ... | TB |
| $10^{15}$ =   | peta =   | PB |
| $10^{18}$ =   | exa =    | EB |
| $10^{21}$ =   | zetta =  | ZB |
| $10^{24}$ =   | yotta =  | YB |

| POTENZE DI 2  |          |          |
|---------------|----------|----------|
| $2^{10}$ byte | kibibyte | KiB      |
| $2^{20}$ =    | Mibibyte | MiB      |
| $2^{30}$ =    | Gibibyte | GiB      |
| $\vdots$      | $\vdots$ | $\vdots$ |

Figura 4.53: Svolgimento 4

#### Domanda 4 (Allineamento in memoria)

Si consideri la seguente struct C:

```
typedef struct S {  
    char x;  
    char *y;  
    short z;  
} S;
```

Quale delle seguenti affermazioni è **falsa**?

|          |                                   |          |                                                                      |
|----------|-----------------------------------|----------|----------------------------------------------------------------------|
| <b>A</b> | sizeof(S) = 12 ✓                  | <b>B</b> | Sono necessari 5 byte di padding ✓                                   |
| <b>C</b> | Il campo z si trova a offset 12 ✗ | <b>D</b> | Cambiando l'ordine dei campi, è possibile ottimizzare la sizeof di S |

Motivare la risposta nel file M4.txt. **Risposte non motivate saranno considerate nulle.**

Figura 4.54: Domande 5

Es 14/5/2023 Domanda 4

```
typedef struct S {  
    char x;  
    char *y;  
    short z;  
} S;
```

|   |   |   |   |
|---|---|---|---|
| x | . | . | . |
| x | x | x | x |
| x | x | . | . |

sizeof(S) = 12    byte di padding = 5

ordine che ottimizza dimensione

```
typedef struct S {  
    char *y;  
    short z;  
    char x;  
} S;
```

|   |   |   |   |
|---|---|---|---|
| x | x | x | x |
| x | x |   |   |
|   |   | x | . |

sizeof(S) = 8    byte di padding = 1

Figura 4.55: Svolgimento 5

## Domanda 2 (pipelining)

Si consideri la seguente sequenza di istruzioni:

```
movl $5, %eax
addl %esi, %edx
subl $10, %ecx
incl %edx
movl $3, %edi
```

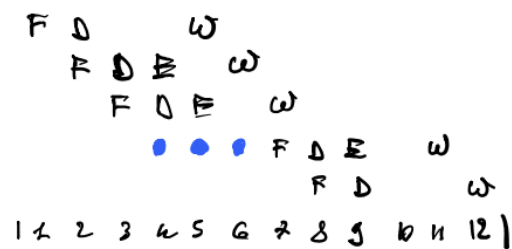
Riordinando le istruzioni (senza cambiare la semantica), è possibile ottimizzare il codice in modo da ridurre il numero di cicli di clock richiesti?

|          |                                            |          |                                                                        |
|----------|--------------------------------------------|----------|------------------------------------------------------------------------|
| <b>A</b> | No, non è possibile                        | <b>B</b> | Sì, è possibile ridurre il numero di cicli di clock a 9, ma non a meno |
| <b>C</b> | Sì, è possibile ridurre il numero di ciclo | <b>D</b> | Sì, è possibile ridurre il numero di cicli                             |

Figura 4.56: Domande 6

Es. 16/9/2023 Domanda 2

```
movl $5, %eax
addl %esi, %edx
subl $10, %ecx
incl %edx
movl $3, %edi
```



```
movl $5, %eax
addl %esi, %ecx
subl $10, %ecx
movl $3, %edi
incl %edx
```

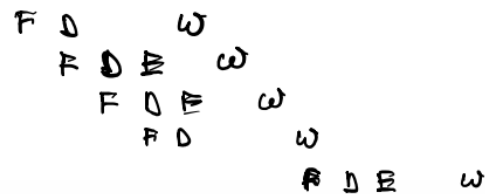


Figura 4.57: Svolgimento 6